Market analysis predictor – Mandar Shringari NEA -project

Centre number - 51419
candidate number - 2196

Shringari, Mandar (13Br)

Table of Contents

Analysis *5*
Problem Identification **5**
Survey and problem research **6**
Features Making the Problem Computationally Solvable **8**
Why is computational method suitable **10**
Stakeholders **12**
Solutions already present and other approaches **13**
Different approaches for forecasting potential closing price **16**
Sentiment Analysis 16
Deep Learning Models 17
The approach chosen **18**
Hardware requirements **22**
Software requirements **23**
Success Criteria **24**
Evaluation Metrics (For Prediction Model) **25**
Design *27*
User interface design **27**
The start page **28**
The login and sign-up page **30**
The portfolio page **32**
Individual stock page **35**
Stakeholders' input on GUI **38**
Structure Algorithm: **39**
System Decomposition and Work Breakdown Structure 39
GUI management 40
Authentication and security 49
Data insertion and retrieval 54
Data retrieval and portfolio search algorithm 54
Adding Stock to Portfolio Algorithm 55
Validation and Integrity Checks 55
Machine learning prediction 59
Data Preprocessing 59
The model training 60
Database for the backend 64
User Portfolio Historical Data 70
Implementation *72*
Database 72
Insertion of OHCLV and other data for user chosen stock to database table 79

Stock plotting 96
Changes and Justifications 102
Future trend of closing price 106
Changes and Justifications 110
Volatility calculator closing price 113
Linking volatility calculator to data retrieval 115
Hashing algorithm and validation 119
GUI initialization 126
Base page 127
Start page 129
Login page/ sign up page 131
Sign up page 134
Portfolio page 137
More detailed stock page 147
Final backend code with comments 154

Evaluation ***178***

Post development testing **178**

Justification of Results ***180***

Justification of Results ***182***

Overview: **183**

184
Overall: 185

Usability features **190**

Appendices ***195***

Analysis

Problem Identification

Problem Statement

Investors and traders must decide which stocks to buy, sell, or hold based on complex and fluctuating financial data on an hourly basis. However, analysing market trends manually would often be time-consuming, difficult, and sometimes incorrect, especially for investors who lack advanced financial knowledge. This requires a dependable system that is automated to analyse historical stock data and identify trends and patterns, which gives an integrated tools that combines statistical analysis, graphical visualisation, and key financial data in one place, and provides another layer for informed decision-making.

Background

The market price of a stock is not influenced by a single parameter, but by many factors which include historical trends, market sentiment, company performance, and economic conditions. All these causes a large volume of financial data to be generated every day this makes it harder for investors to interpret the trends without a computational tool. While professional investors do have access to advanced software financial tools, many other investors rely on basic charts or their intuition, which may not be consistent investment decisions, as they are based on uninformed financial information.

With the increasing availability of financial data, it is possible to use machine learning techniques to develop software that analyses historical stock data and provides a clear representation of the overall price direction. This software tool processes the stock data and generates the trend line automatically, which makes it more accessible and consistent to make trend analysis.

Possible Causes of the Problem

- Large and complex financial datasets that are difficult to interpret manually
- Reliance on visual chart inspection rather than mathematical analysis
- Lack of integrated tools combining statistical analysis with clear visualisation
- Limited understanding of measures such as volatility and regression trends
- Fragmentation of financial data across multiple sources

Impact and Significance

without any structured analytical tool, where users may misinterpret market behaviour, underestimate the risk, or overlook any important patterns in price movement. This would reduce confidence in decision-making especially if they are new to the field and further affecting their financial knowledge.

Developing a market analysis system that focuses on financial data with graphical visualisation, trend analysis, and volatility measurement would allow users to analyse stocks more objectively. This project would demonstrate the statistical computational method that can be applied to real world financial data, which improves the accessibility to analytical tools

Scope of the Project

This project will develop a software application that:

- Retrieves and displays historical stock market data
- Produces graphical representations of price movement
- Calculates linear regression trend lines
- Measures volatility to quantify price variation
- Presents key financial information for analysis

The program would support market analysis rather than precise future prediction, as it is affected by many external factors such as geopolitical events and other sentimental views of investors. This project focuses more on statistical analysis and mathematical analysis of historical financial data.

Survey and problem research

In a survey conducted for this project, the following questions were asked to better understand the needs of investors.

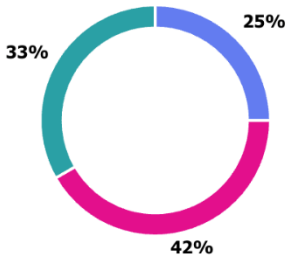
1. Have you ever invested in stock market? If yes, what level would you be?
2.Do you use any digital platform to invest (e.g. - app or website)
3.Are you satisfied by your experience of using the platform
4.Are there any features or experience you did not like and want to improve? if never used digital investing platform is there anything pushing you away

The responses -

1. Have you ever invested in stock market ? If yes what level would you be?

More details

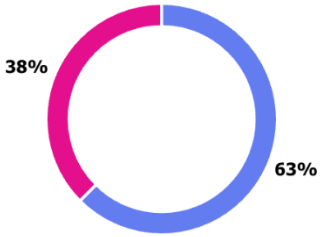
Yes - beginner	3
Yes - I have experience	5
No - I have never	4



2. Do you use any digital platform to invest (eg - app or website)

More details

Yes	5
No	3



3. Are you satisfied by your experience

More details

Yes	3
No	3

4. Are there any features or experiences you did not like and want to improve? if never used digital investing platform is there anything pushing you away.

“It could never visualise trends of multiple stocks & shares on the stock market”

“I could't not see market volatility in the stock market which is potential to higher return”

"Stockbrokers give an indication and through their experience where the market is further providing educational source for stock investment"

"I feel secure investing through brokers as I never know who would hack my digital account"

"There are many resources available while I am planning and researching a stock in google finance but there it feels sometimes overwhelmed"

"A simpler research system, which some do provide, often lacks depth or reliability. Many platforms that try to make research easy end up limiting the detail of information available, which makes it harder to make accurate decisions. To get the most value, investors often must use more advanced tools, which can be overwhelming. In addition, unclear or inconsistent data can cause confusion, leading to repeated analysis or misinterpretation of trends — which can be damaging if not noticed in time."

In summary the users/ investors want a digital market research platform which is -

- Safe and secure preventing any unauthorised access
- Insightful has enough data and not to overwhelming
- Can help visualise or inform the market trend so an application that generates market forecast
- Inform market volatility for their selected stock allowing user to decide the right time to invest

Features Making the Problem Computationally Solvable

1. Fast processing of large datasets:
   Stock market contains thousands of data such as price records from prolonged periods of time which might take long time manually analyse.

   Justification: This would help investors because they could analyse a market data quickly instead of spending time manually reviewing price. It would save them time as it would give the opportunity to focus on decisions rather than processing data.

2. Automatic calculations using fixed formulas:
   The program doe athematic calculations to find values for volatility and regression using well defined mathematical formulas

   Justification: This would benefit investors who are not confident with their mathematical abilities or new to the complex formulas. The program would perform this calculation for them reducing chance of mistake occurring and ensuring the analysis is reliable

3. Easy comparison between different stocks or time periods:

Investors can select different stocks and view the historical data from different data range to analyse

Justification: This would allow investors to compare the performance of different stocks at the same time or the stocks historical performance in different period. This would support a better decision making as they could evaluate multiple option quickly

4. Organised storage of financial data:
   The system stores and manages the historical price data and other financial information in structured way

   Justification – This would help investors by allowing quick access the data they needed in single place without searching through multiple sources. It makes analysis more efficient and reduces confusion

5. Clear visual graph generated from numerical data:
   The program converts the historical data and trend calculations into charts and graphs

   Justification: This would help users to understand market behaviour more easily as visual displays makes trends and price movement more clearly, which would support a more confident interpretation of the results.
6. User - controlled analysis:
   The program takes inputs such as stock ticker and the data range to decide what analysis to perform they want.

   Justification: Different users based on their preference run analysis on the stock ticker they want more information on. For example, short term traders and long-

Why is computational method suitable

Making the stock market visual, predictable, and secure is perfectly suitable through computational methods.

Problem recognition

The main problem is helping users to understand the stock market trends using large financial data however which is also clear, secure, and useful for decision making. users should be able to see the market trend, understand volatility, and view the financial market in a way that would not overwhelm them by too much information.

However, the underlying problem would be collecting the historical stock data, while storing it securely, analysing it with statistical methods, and presenting the results in a simpler and more meaningful way. The system must generate visual trend information, provide market insights, and show volatility so users can decide the right time to invest. It must also prevent unauthorised access to protect user accounts and data.

Justification – problem recognition would ensure the programs is fully understood before the development begins and the stakeholder's requirement are met. This not only includes analysing market data but implementing the solution in a clear ans secure way as stakeholders requested.

Problem decomposition
The overall problem could be broken down into a smaller manageable sub task such as data collection, statistical analysis, visual presentation, and secure data access. Where each of them could be developed separately
Justification – the braking down of the program into modules would make the development easier as it is more organised when developing as each part can be built and tested separately which improves the reliability of the program as errors are prevented. For stakeholders, the program is more trusted and stable while also ensuring features work properly without interfering with each other.

Abstraction

Abstraction focuses only essential information while ignoring the unnecessary, specific details simplifying the overall system. The system should include measurable financial data such as historical prices, trends, and volatility because which are essential to make investment decisions.

Information that does not directly support analysis, such as unrelated news or excessive financial indicators, is excluded to prevent the system from becoming confusing or overwhelming.

Justification - For stakeholders such as investors and students learning market analysis, abstraction ensures the system provides clear and useful insights rather than too much information. This helps users focus on understanding trends, forecasting behaviour, and judging volatility without distraction from irrelevant data.

Data Mining

The program uses data mining to find patterns in the financial data instead of just displaying raw data but in display in such a way investors can identify any insights such as trends, volatility patter, and market behaviour. Data mining involves collecting financial data, filtering it, and analysing it to discover relationships or patterns that are not immediately obvious.

Justification –

For students or beginners studying the financial markets, data mining helps them understand how the real-world data can be analysed to produce meaningful insights. The system simplifies complex datasets and presents clear results without overwhelming the user.

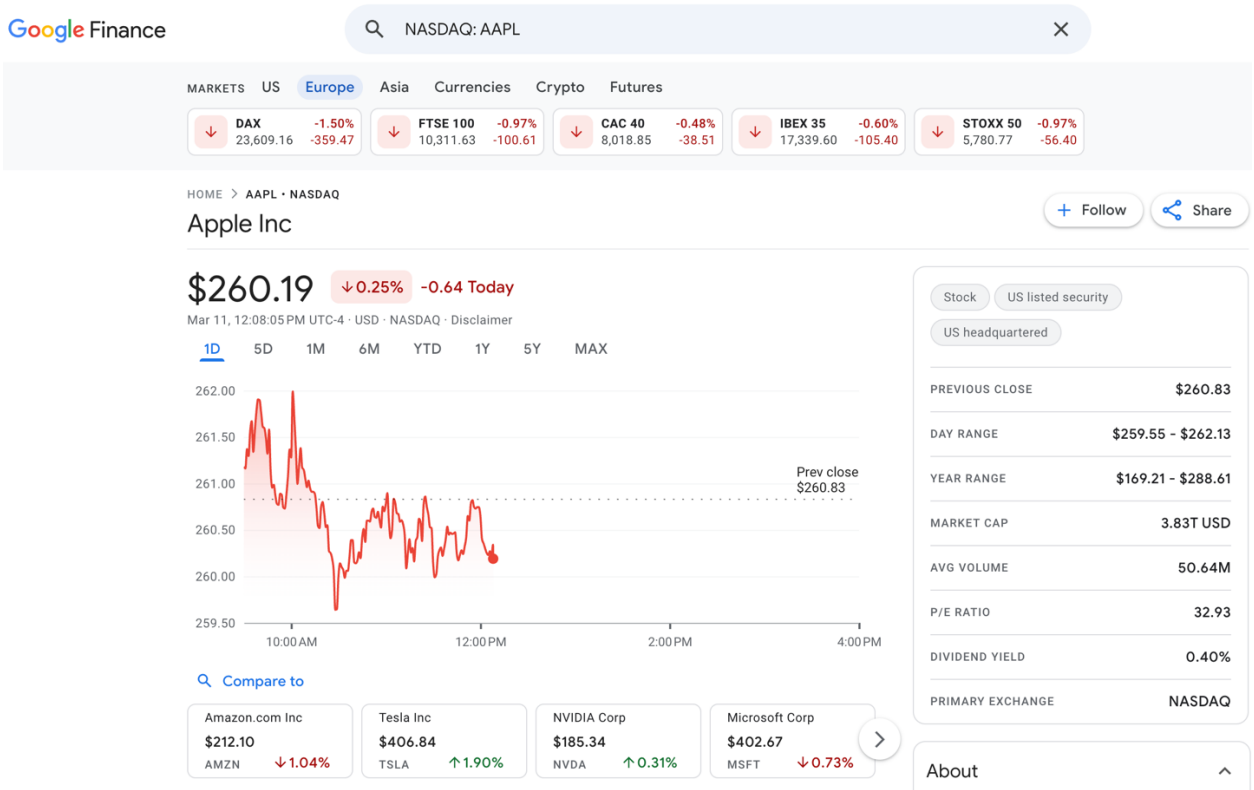
Stakeholders

The stakeholders of this project are newcomers to the stock market, financial students/educators, and casual investors.
Newcomers in the stock market are one of the stakeholders, as it is difficult to understand complex financial market information, particularly in market research. With the project being abstracted, the users can understand what is essential for every investor to understand and make insights and develop their knowledge and experiment to see if their insight or decision was right and compare it with the predictive model before investing. With tools like volatility, it also teaches them how to correctly time when buying or selling a stock.
Financial students/educators can learn and teach about the role of computing systems in the finance market; they could also learn similarly to new investors about trends and more financial terms in a visual way, such as 'open', 'close', and 'moving average'. The programme also highlights that the stock market is not truly predictable but rather heuristic and estimates where it could be based on current market conditions, showing the importance of research about external factors such as news and sentiment and the importance of well-rounded research.
Casual investors benefit from a secure market research account where unauthorised access cannot see their investment, as it can pose a huge financial security risk. They also can gain access to tools such as volatility measurers and market forecasts without doing any market research, as it easier, as no knowledge or experience is required, leading to potentially lucrative outcomes.

Solutions already present and other approaches.

To understand the current market for stakeholder's research was conducted to understand existing solution and gain ideas to incorporate.

Google finance



Overview

Google finance is an online financial information platform from the company Google which provides a real time stock market data with financial information, news, and investment tracking tools. It provides users to monitor the current stock price, analyse historical trends and follow the market trends using an easy-to-use interface. The website is widely

used by both beginners and experienced investors as it is accessible through a web browser and is integrate with many tools through google services.

Advantages

- Easy to use interface – the overall layout of the website is simple and how it is designed is kept all type of investors in mind such as beginner investors.
- Reak time market price - Users have access to live updated market price and trend movements.
- Portfolio tracking – investors can follow a specific stock which are kept as watchlist the website informs the user when there is a update to the stock through notification.
- Integrated financial news – the website provides real time news about the organisation and other external factors effecting the market price due to sentimental effect to investors.
- Free access – the platform is free to use to all investors who have access to internet

Limitations

- Limited analysis tool – It lacks any technical analysis tools compared to specialised trading platform for which experienced investors have access.
- Minimal customization – the website prevents any significant customization to the dashboards and adding other complex indicators.
- Charts use a simple line historical market price instead of using a candlestick, mountain and other templates used by professional trading software which give much more information and insights.

Features to incorporate

- Interactive price chart for visualising for historical data and trends
- A portfolio page like watchlist functionality so a user can track the added stocks.
- Slider or button which filters the historical data based on time ranges and change the historical stock data price chart
- A minimalistic and abstracted GUI interface which is suitable all the stakeholders of the program

Justification

Incorporating all these features would make the improve the overall usability and the functionality of the projrct program.

Trading View

Trading view similarly is also an online financial analysis platform mostly used by traders and investors to analyse financial market by providing tools such as charting, technical indicators, and real time financial data to a wide range of financial assets such as cryptocurrencies, index funds, foreign exchange market and currencies. The platform is mostly used by experts who have extensive knowledge about trading the financial market.

Advantage

- Trading View provides a highly detailed and a interactive charts with multiple viewing template such as heikin ashi and renko & kagi
- Users can apply many built in indicators such as moving averages, RSI and MACD for more in-depth technical analysis

- Trading view has its own custom coding language for users to make their custom indicator called pine script
- It supports multitude of assets from cryptocurrencies to index funds in a single platform
- Trading view allows investors to invest using fake currency and test how well it would perform as if they put in their real money

Limitations

- Most tools are complex for inexperienced users and may overwhelm them with options
- Advanced tools are paid features and only available with premium plans
- The platform focus is on charts and technical indicators rather than fundamental financial data or some kind of predictive insight

Features to incorporate

- Candlestick template chosen as the graphing of stock price movement.
- Multiple time frame viewing option such as daily or yearly
- Technical indicators to find the trend and other trend analysis
- User friendly data visualisation to help user to understand complex market data more easily.

Justification

Candlestick including some of these features would make the analytical capability of the program to improve such as the interactive charts and the technical indicator would allow the stakeholders to identify the trends by themselves in a more approachable and insightful way. By incorporating simplified versions of these tools, the system can provide meaningful insights without overwhelming users. This would allow the system to combine ease of use with powerful analysis features, improving its overall effectiveness for users researching stocks.

Different approaches for forecasting potential closing price

Sentiment Analysis

Overview

Sentimental analysis, where web scraping is implemented on a financial news website or a general social media page for any news for a given stock. It stores the stock information in a data structure and utilizes natural language processing (NLP) to understand sentiment. It uses news articles and social media (e.g., Twitter, Reddit) influenced by public perception and investor confidence rather than technical indicators. The NLP works by tokenizing the data (splitting sentences into words) and removing stop words (the, etc.) and giving sentiment scoring (positive, negative, or neutral). Further feature engineering is used to convert sentiment scores into numerical values (between –1 and +1). This approach is powerful because it captures real-world events and emotions that purely numerical models cannot.

Advantages

One of the advantages of sentimental analysis it captures multiple external factors that would influence stock price such as the public opinion or breaking news which doesn't purely rely on historical data but instead identifies impact of the event on the market reactions. This would allow for forecasting model to include qualitative information that would affect price movements before they affect technical indicators.

Limitations

Sentiment analysis is significantly more complex to implement compared to other forecasting methods. It requires reliable web scraping systems to continuously collect large amounts of textual data from multiple sources. In addition, NLP processing requires specialized libraries and significant computational resources. Text data also introduces challenges such as sarcasm, misleading headlines, spam content, and biased reporting, which can reduce the accuracy of the sentiment score. Furthermore, financial sentiment models typically require large

Deep Learning Models

Overview

Another approach to stock market prediction is the use of deep learning. By utilising recurrent neural networks (RNNs) and versions of its long-term memory networks. The model is specifically designed to handle sequential data, making use of well suited for stock market prediction, as past market values strongly influence the future outcomes in the stock market. The model works by taking past stock prices and processing them step by step, finding a learning pattern and dependencies. Using LSTM makes it go one step further, as it remembers important long-term patterns while discarding irrelevant details using the computational method of abstraction and backtracking. The drawbacks are that deep learning requires substantial amounts of historical and substantial computing power. which cannot be due to the huge resources of money, and storage space is limited. It is also harder to learn and understand for stakeholders such as students who want to learn the link between finance and computing.

Advantages

One if the main advantage if a deep learning model is the ability of the model to detect non-linear trend and pattern in large financial dataset, they can automatically learn patterns in the data without requiring extensive manual feature engineering. A LSTM model a particular effective at long term dependencies in time series data which can improve the accuracy of the stock market predictions given their sufficient data is available.

Limitations

- Technical indicators, which are mathematical formulas derived from stock price and volume data. Examples include moving average (MA), relative strength index (RSI), and moving average convergence divergence (MACD). Each of them measures different things and helps forecast the market –
    - MA smooths price fluctuations to highlight overall trends
    - RSI measures whether the stock is overbought or oversold

- MACD shows the relationship between two moving averages.

The formulae are combined and simulated in a program where the program can predict the direction the market is heading (falling or rising) and even the closing price. However, there are hundreds of different indicators, each with different parameters. Choosing which indicator to use is often arbitrary and may lead to biased results. Adding too many indicators can cause the model to fit historical noise. reducing its ability and can give false or misleading signals

The approach chosen

The approach selected for the project includes tkinter as the main GUI, as it allows users to interact easily with the application by entering stock tickers, viewing historical data, and seeing the predicted closing price in a clear format due to its seamless integration with Python integration, such as matplotlib. Features handled by tkinter -

- Text Entry for Stock Tickers – Users can type the symbol of the stock they want to analyse in a plain text box, making the input process straightforward.
- Action Buttons (Fetch, Predict, Clear)—Clearly labelled buttons allow users to retrieve data, generate predictions, or reset inputs, guiding them through each step of the workflow.
- Visualisation of Historical and Predicted Prices – Matplotlib graphs embedded in the GUI display past stock performance alongside predicted closing prices for the next seven days, helping users easily interpret trends.
- Shows volatility in the market and ranks it. between user-seen stocks and uses sorting algorithm to rank with most volatility
- Display of Model Accuracy (R^2 Score) – Users can see a numerical score indicating how accurate the predictions are, providing transparency and building trust in the model.
- Error Messages and Feedback – The program notifies users of invalid tickers or data retrieval issues, preventing confusion and making the application more robust.
- Logical Layout – The interface is organised with clear sections for input, output, and plots, ensuring a natural workflow and reducing cognitive load.

A sign-in page is made where users can create an account where the user enters a username and password. Checks if the username already exists in the database. If not, a hashing algorithm is used to hash the password, and the username and hashed password are stored in SQLite. While signing in, the user enters and searches the database for the

username; if found, hash the input password and compare it with the stored hash. If they match, the login is successful.

For getting historical data and live current market information of the stock market, the program is utilising the Yfinance API. The API chosen is free, reliable, and easy to integrate with Python, enabling the program to automatically fetch data for any stock. ticker entered by the user. This ensures that both historical trends and current market conditions are readily available for analysis and prediction within the application.

The data structures involved in the programme include lists, dictionaries, tuples, and databases.

- **Lists** are used to store sequences of historical stock prices and predicted values, allowing easy iteration for plotting or calculations.
- **Dictionaries** store stock-specific information, such as ticker symbols and associated metadata, providing for user queries.
- **Tuples** are used to store fixed sets of related data, such as date-price pairs, ensuring the integrity of the data.
- A **database (SQLite)** is used to persistently store historical stock data locally, so users can access previously retrieved information without repeatedly downloading it.

The database allows the storage of stock information accessed by the user in the long term and prevents repeated downloads. It also prevents all need for all stocks to be present on the user's computer. By organizing the data using normalisation, different entities such as stock details, historical prices, and model predictions are stored in separate, logically structured tables. This ensures that data is not redundant, is consistent, and is easier to manage, while it also makes it simple to update or add new records. Referential integrity complements normalisation by enforcing valid relationships between tables. For example, each prediction in the Predictions table must reference a valid stock in the Stocks table. This prevents orphan records or inconsistencies, ensuring that the regression model always works with reliable and connected data.

The predictive model is based on a supervised machine learning regression approach and is implemented using the scikit-learn library. From raw data the module selects a feature which is further engineered. Such as the previous closing price, which is used as input x to train the regression model, whereas the closing price acts as the target output.

The training of the program works by splitting the labelled data dataset into training and testing sets. With 80% for training and 20% for testing the model, this means 80% of labelled data is used for supervised training; then, the ordinary least squares algorithm is used to find a linear equation between the historical closing price (x) and the forecast price, which is labelled for training purposes this means scikit-learn is finding weights (coefficients) for each input variable that best explain the target; the other 20% is used to test with already available data and evaluate how close it is, allowing it to be tweaked.

Once trained, the model can make predictions on future stock data. In the program, this allows the user to view the forecasted closing prices for the next seven days alongside historical values. These predictions are then plotted using Matplotlib, enabling the user to visually compare the historical "mountain" of prices with the projected trend.

Libraries required for the prediction-

- **scikit-learn**
  - Provides the regression model.
  - Includes tools for splitting data (80% training, 20% testing).
  - Provides evaluation metrics like the R^2 score.
- **pandas**
  - Stores stock data in Data Frames.
  - Makes it easy to clean, manipulate, and engineer features
- **NumPy**
  - Handles efficient numerical calculations and array operations.
  - Used internally by scikit-learn for regression maths.
- **finance**
  - Retrieves historical and livestock market data.
  - Supplies the raw OHLCV dataset (Open, High, Low, Close, Volume).
- **matplotlib**
  - Visualizes historical prices and predicted trends.
  - Allows overlay of actual vs predicted values for comparison.
- **sqlite3**
  - Stores historical stock data locally in a database.
  - Prevents repeated downloads from finance.

The use of linear regression is suitable for the project, as it does not require vast amounts of training data, reducing memory usage and training time. Additionally, the outputs can be easily visualised alongside historical data to aid user interpretation. However, linear regression has limitations; it assumes linear relationships while real markets often behave non-linearly, is sensitive to outliers like sudden market crashes, and cannot incorporate external factors such as news sentiment or investor psychology.

One of the core features of the project is to calculate and display the market volatility of the stock. Volatility is a measure of how stock prices fluctuate over time. The more the stock fluctuates, the higher the level associated with holding the stock. A highly volatile stock is considered riskier because its price can change rapidly, whereas a stock with low volatility is more stable.

The method implemented to calculate market volatility is known as historical volatility. Which is dependent on readily available historical data, the process begins from the closing price. for stock stored in the database collected from the API. Logarithmic returns are used instead of using simple percentage change. Because it is mathematically

convenient, they are scaled independently and added together over time, making it better suited for financial modelling.

Once the return has been calculated, the rolling window is implemented for 30 days. Within the window an algorithm is used to calculate the standard deviation of returns, which measures how spread out the returns are from their mean. If the returns are tightly clustered, volatility will be low; if they vary widely, volatility will be high. Daily volatility is then annualised by multiplying by the square root of the number of trading days in a year (252 days).

The feature is also extended by ranking multiple seen stocks by their volatility. In this case, the program calculates the volatility for several downloaded stock in database tickers, stores the values in a dictionary, and then sorts them in descending order using quick sort having space complexity of Olog(n) reducing the load on user's computer moreover a time complexity of Olog(n log n) in average case.

1. The user enters a stock ticker into the program.
2. Historical closing prices are fetched using the API.
3. Daily log returns are computed.
4. A rolling standard deviation of returns is calculated over the selected window.
5. The result is annualised to produce a percentage figure for annual volatility.
6. The program displays the volatility value in the graphical user interface.

Historical volatility is suitable for the programme. because it is straightforward and computationally efficient. It is efficient because it uses original price data, which requires only basic statistical computations; it also can be updated incrementally with a rolling window, avoiding complex assumptions, and provides a practical measure of risk. This translated to faster execution and accurate insights for decision-making

There are, however, some limitations. Historical volatility is entirely backward-looking: it only shows how volatile the stock has been in the past and cannot predict sudden changes caused by external factors such as news events or market shocks. The choice of window size can also affect results. A shorter window may be too noisy, while a longer one may lag current market conditions. Nevertheless, despite these drawbacks, historical volatility provides an accessible and practical measure of risk, making it an appropriate choice for this project.

Limitation

#	Limitation	Description	Impact	Justification
1	Data Reliability from yfinance API	The prediction model relies entirely on historical data retrieved from the yfinance API. If this data is missing, delayed, or	High – directly affects the accuracy of all stock price predictions.	yfinance is a widely used, free, and well-maintained library that retrieves data from Yahoo Finance, a reputable financial data source. For an educational project, this

		contains errors, the model will produce unreliable and potentially misleading predictions.		level of reliability is considered sufficient. Additional validation checks and error messages are implemented to alert users when data cannot be retrieved.
2	Internet Connectivity Dependency	Since stock data is fetched live from the yfinance API over the internet, the application cannot function without an active network connection. Offline use is not supported.	High – the application is non-functional without an internet connection.	SQLite local storage is used to cache previously fetched stock data, reducing repeated downloads and allowing limited offline access to historical data that has already been retrieved. A clear error message is displayed when the connection fails.
3	Limited Scope: No Investment or Portfolio Tools	The application only provides historical and predicted stock prices. It does not offer investment strategies, risk analysis tools, or portfolio management features.	Medium – limits the application to educational or observational use rather than practical financial decision-making.	The project scope is intentionally focused on demonstrating machine learning concepts and data visualisation. Adding portfolio management or financial advisory features would significantly increase complexity and introduce regulatory considerations. Stakeholders such as new investors and hobbyists can still gain meaningful insight into market trends.

Hardware requirements

Requirement	Explanation	Justification
Processor (CPU)	The CPU executes all program logic including stock searching, volatility calculations, sorting and rendering graphs using Matplotlib.	Minimum: Dual-core 2.0 GHz (e.g. M series MAC). stocks using quicksort has average time complexity O(n log n), which is efficiently handled by modern CPUs. Multi-core CPUs also improve responsiveness when GUI and background tasks run simultaneously.
RAM (Memory)	RAM stores active datasets such as stock price history, portfolio data, and temporary arrays used in	Minimum: 4 GB RAM, Recommended: 8 GB+. A dataset of ~5 years of daily stock data for 50 stocks can exceed 10–50 MB in memory when processed for

	calculations. Libraries like Pandas and Matplotlib load data into memory.	database. Insufficient RAM leads to swapping, causing noticeable slowdowns during analysis and graph rendering.
Storage	Storage is used for the application, SQLite database, and cached historical stock data.	Minimum: 500 MB free space, as logs/backups are added, storage requirements increase. SSD storage significantly improves database read/write speeds compared to HDD improving the user experience giving accurate representation while testing .
Internet Connection	Required to fetch live and historical stock data using APIs (e.g. yfinance).	Minimum: 1 Mbps, Recommended: 5 Mbps+ stable connection. API calls typically return data in JSON/CSV format (~50–500 KB per request). A stable connection ensures low latency (<200 ms) and prevents timeouts

Software requirements

Requirement	Explanation	Justification
Programming Language – Python	Python is used to build the core logic of the program and handle financial data processing.	Python has strong support for data analysis and is widely used in financial modelling due to its simplicity and powerful libraries.
Development Environment – Visual studio code	Provides tools for writing, editing, and debugging the program.	An IDE improves development efficiency by providing features such as debugging tools, syntax highlighting, and integrated terminals.
Data Processing Libraries – pandas and NumPy	These libraries are used to manipulate datasets and perform mathematical calculations.	They enable efficient handling of large financial datasets and provide optimized numerical operations required for analysis.
Visualizations Library – Bokeh	Used to create interactive graphs and charts displaying stock market trends.	Interactive charts allow users to explore financial data more easily, improving understanding of trends and patterns.

Success Criteria

ID	Success Criterion	How It Will Be Measured / Tested	Target Outcome	Justification
SC1	The system must allow users to securely create an account and log in.	Attempt login with correct and incorrect credentials. Inspect database to confirm passwords are hashed.	Only correct credentials allow access. Passwords stored as hashed values with salt.	Security was a major concern raised by stakeholders ("I feel secure investing through brokers..."). Implementing hashing with SHA-256 and salted passwords ensures protection of financial-related data and prevents unauthorised access.
SC2	The program must fetch real historical stock data for any valid ticker using an API.	Enter valid and invalid tickers in GUI and observe response.	Valid ticker retrieves historical OHLCV data. Invalid ticker shows clear error message.	Stakeholders wanted reliable and insightful data without being overwhelmed. Using the yFinance API ensures real market data is used rather than simulated values.
SC3	The system must visually display historical stock data clearly.	Compare plotted graph with raw dataset values. Conduct user feedback testing.	At least 30 days of historical data displayed clearly using candlestick or line graph.	Users stated difficulty visualising trends. Visual representation through matplotlib reduces abstraction and improves interpretability for beginners and students.
SC4	The program must generate a 7-day market forecast using regression.	Evaluate predictions using R^2 score and compare predicted vs actual test data.	$R^2 \geq 0.6$ on test dataset.	Investors wanted market trend indication. A supervised regression model provides a computational forecasting solution while remaining computationally efficient.
SC5	The system must calculate and	Manually verify volatility formula using sample data.	Volatility matches manual	Stakeholders wanted visibility of market risk and volatility. Historical

	display annualised historical volatility.		calculation within ±0.01 tolerance.	volatility provides a practical, computationally efficient risk indicator.
SC6	The portfolio must rank stocks by volatility automatically.	Add 3–5 stocks and verify sorting order.	Stocks correctly sorted highest → lowest volatility.	Ranking simplifies complex financial data and supports decision-making without overwhelming users. Quick sort ensures efficiency (O (n log n)).
SC7	The GUI must be intuitive for beginner users.	Conduct user testing with at least 3 non-technical users.	≥ 80% of users can search, predict, and interpret results without assistance.	Stakeholders include beginners and students. Interface simplicity and abstraction are core project goals.
SC8	The system must handle errors gracefully without crashing.	Test invalid input, duplicate stock entries, and other parts.	Clear error messages displayed. Program continues running.	Prevents frustration and increases reliability. Error handling improves robustness and user trust.
SC9	Previously accessed stock data must persist using a database.	Close and reopen program; check if stored stocks remain available.	Portfolio and user credentials remain stored between sessions.	SQLite prevents repeated API downloads and improves efficiency while maintaining data organisation via normalisation.
SC10	The system must respond efficiently under normal usage.	Measure execution time for single stock fetch + prediction.	Execution time < 5 seconds per stock.	Performance modelling ensures system responsiveness, meeting stakeholder expectations for real-time interaction.

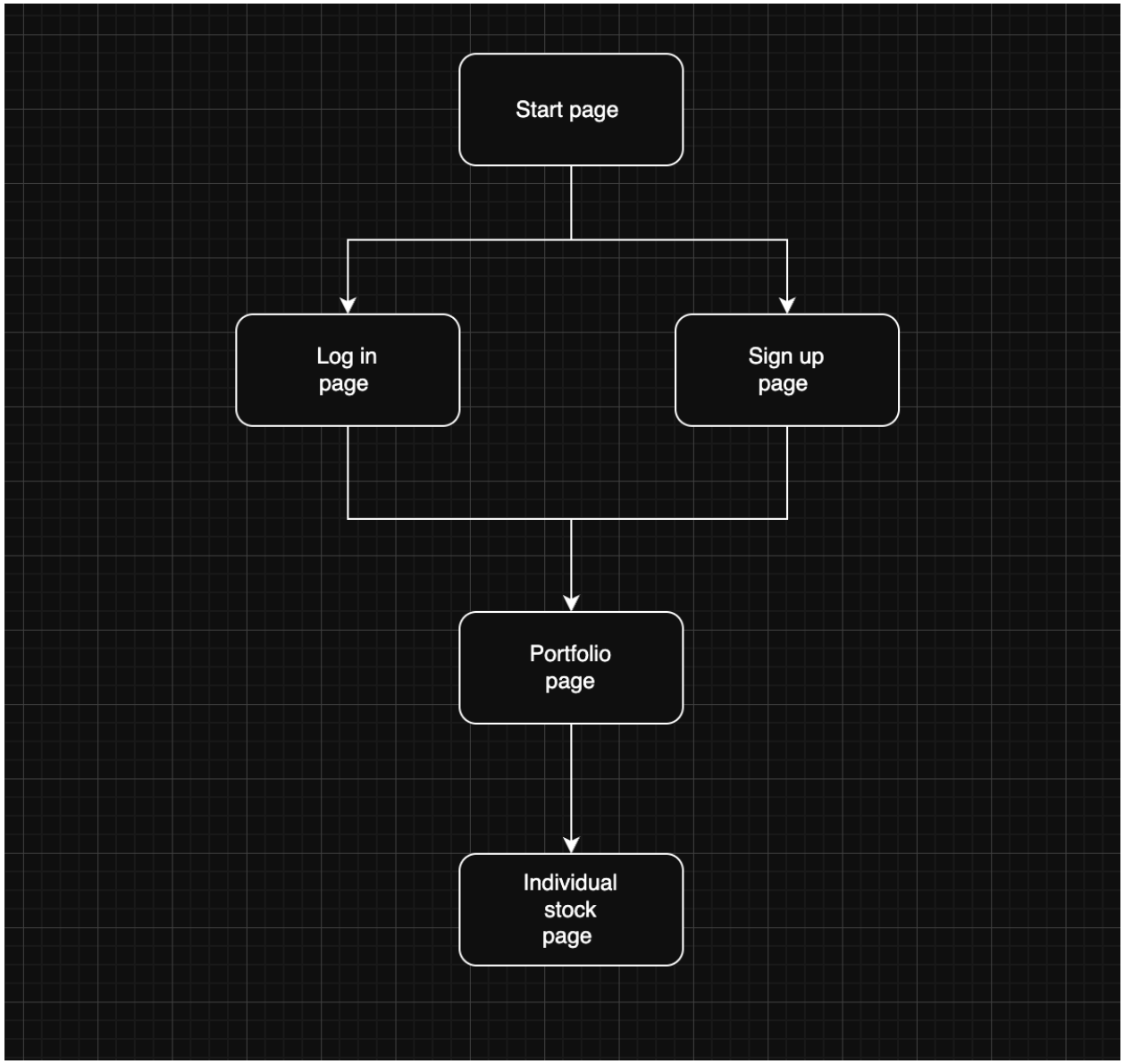
Evaluation Metrics (For Prediction Model)

Metric ID	Metric Name	Application	Success Threshold	Justification

ME1	R^2 Score	Measures proportion of variance explained by regression model.	≥ 0.6	Ensures model explains majority of price movement patterns.
ME2	RMSE	Measures average squared prediction error.	≤ 5% of average stock price	Ensures predictions remain useful.
ME3	MAE	Measures average absolute error.	≤ £2 (for medium-priced stocks tested)	Keeps prediction error within acceptable investor tolerance.

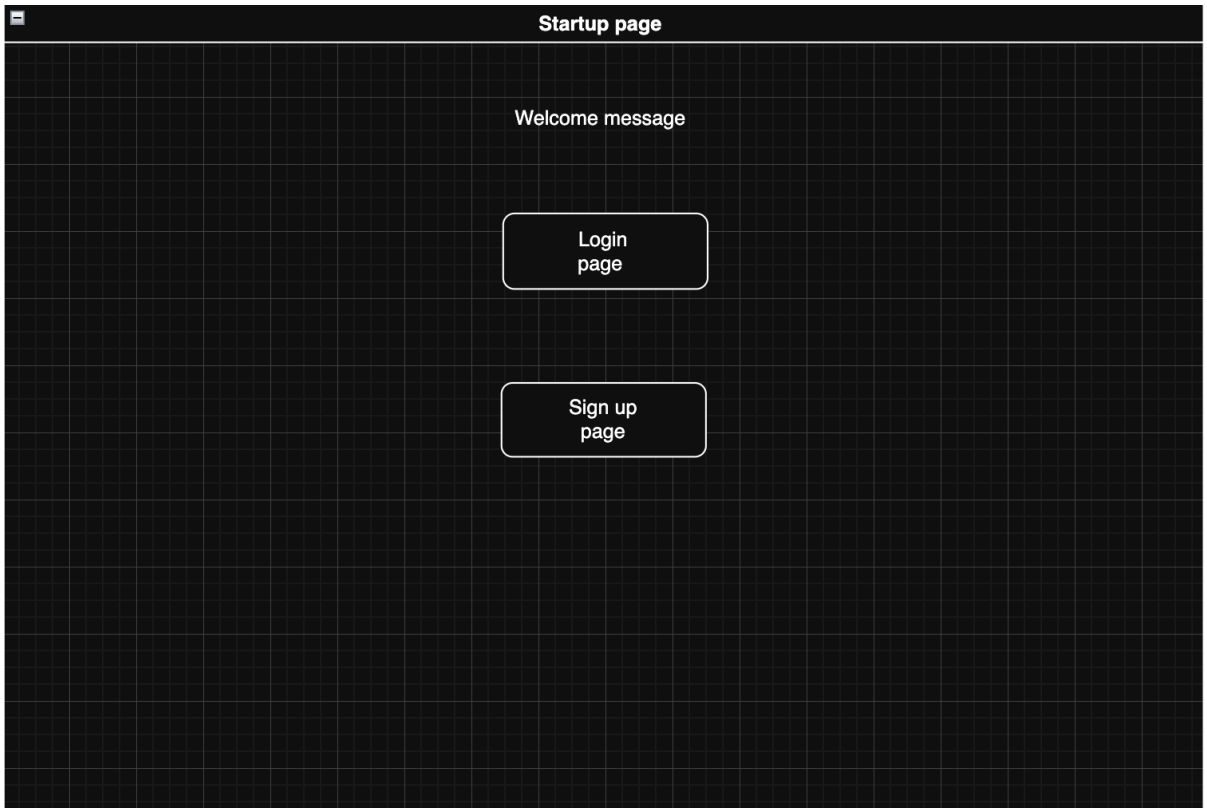
Design

User interface design



The diagram above shows the sequential flow of all the GUI pages the user interacts with.:

The start page



The welcome message

The start page is the first point at which the user interacts with the program. The use of a welcome message provides the user with a friendly and clear introduction explaining the features of the program and how it is utilised. The main purpose of the programme is to create a sense of confidence for beginner investors and students by giving a brief introduction of features in an abstracted manner.

Stakeholders' justification and user usability – improves the user experience by introducing the functionality of the program prior to experience. It makes the program more approachable to stakeholders by building trust by explaining the program's nature and educational purpose. It also aligns with the success criteria making sure that almost 80% of users can use the system without requiring guidance

The login button and sign-up button

The sign-in and login page buttons are placed on the start page to provide the users with a practical and intuitively navigated path. The buttons provide an essential component of the program's usability design function, which allows either the user to create the program account or, if they already have an existing one, to log in. The overall design creates a common convention used by most common modern applications, further improving

accessibility for the inexperienced users to a structure they are mostly familiar with, which has similar interfaces.

- Sign-up button – takes the user to a registration page where users can create a secure account.
- Login button – takes users to the login page provided they already have an account, allowing them to access their saved financial data.

Justification – the use of both buttons will support user usability and security criteria. The buttons are more simplified, making it less stressful for users to navigate through the program. By directing the user to the authentication page, making sure only users who are authorised to can enter the program.

The closing of the program

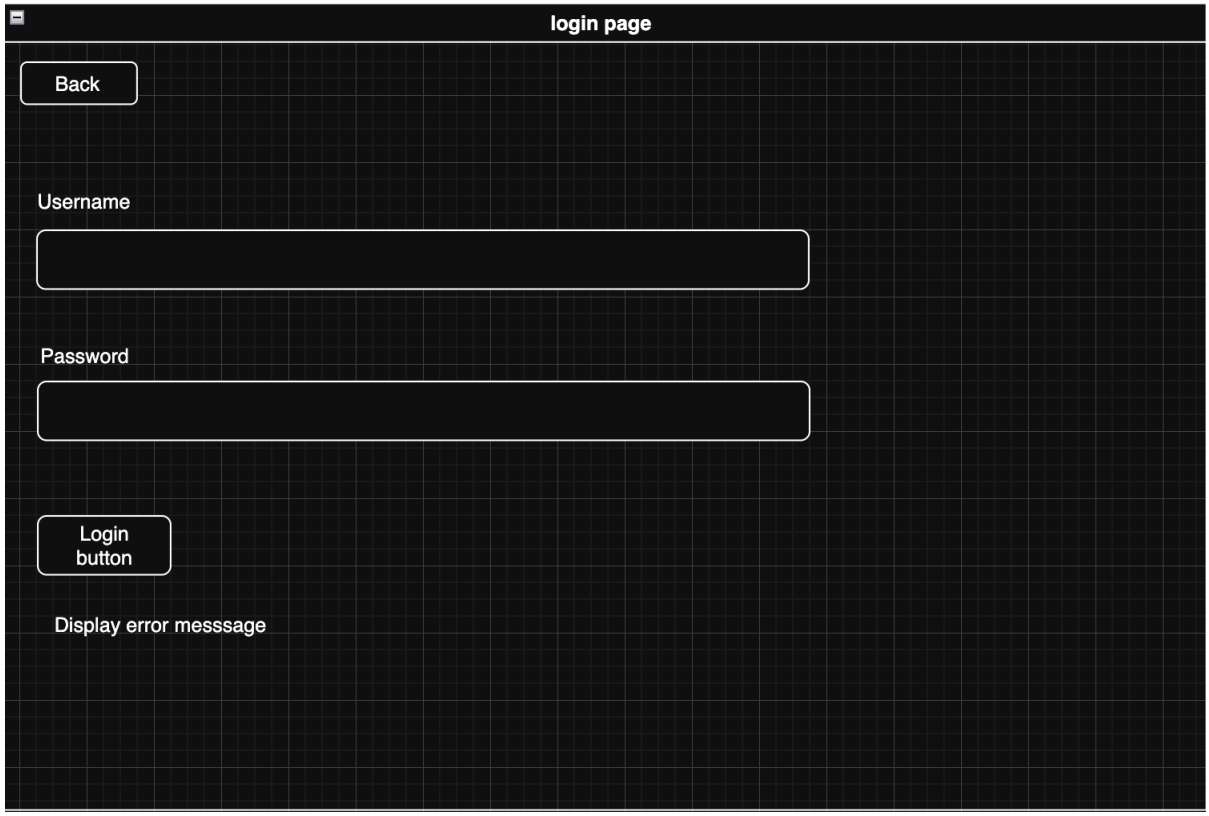
The program does not include a particular exit or close button inside the program to terminate the program. However, it is dependent on the default provided closing button using the operating system. This ensures the user can close the program consistently through every platform equally.

When the user has clicked the X button, the tkinter main loop will be destroyed. This ensures that all the important backend processes are stopped and closes database connections. That would end the user's session is ended; that would mean the user is automatically logged out of the program when it is closed

Justification – A separate exit button would lead to redundancy and limited professional interface. The basic system-level close is universally accepted and understood by everyone, making it more usable for beginner users.

The automatic closing also adds another layer of security, making sure that users' credentials and portfolios are not active in memory, which could be vulnerable in attacks

The login and sign-up page



The back button

The back button is used in multiple pages inside the program shown above; it is also further implemented in the login, sign-up and portfolio pages. The buttons' function allows users to return to the previous screen page frame without the user having to revert and restart the program. This ensures that there is a smooth navigation while making the program state stable and user experience consistent

User usability – The button is clearly labelled "Back" and positioned in familiar location to most users (top left), recreating conventional layouts

Justification – the back button prevents unnecessary repetition, such as re-entering login details if the user opens the wrong page. which causes system load and potentially crashes the user's program. The current interface maintains the right level of responsiveness and reduces the memory load, as only one frame is displayed at a single time.

The input field for username and password

The login page and sign-up page both include two input field widgets, the first one for username and the second for password. To further protect the user's privacy, the password field is masked using the attribute show="*. Preventing anyone nearby from seeing the user's password promoting confidentiality

After fields are submitted, input validation is performed; the program would check whether both the fields meet the required structure and formatting. If either of them is invalid, the respective field boxes are highlighted in red.

Justification – using a masked password would increase the security of sensitive data, which is never displayed in normal text, reducing the overall risk of unauthorized viewing.

The additional validation with red error highlighting would improve the usability by clear feedback, making it easier for the user to correct and review.

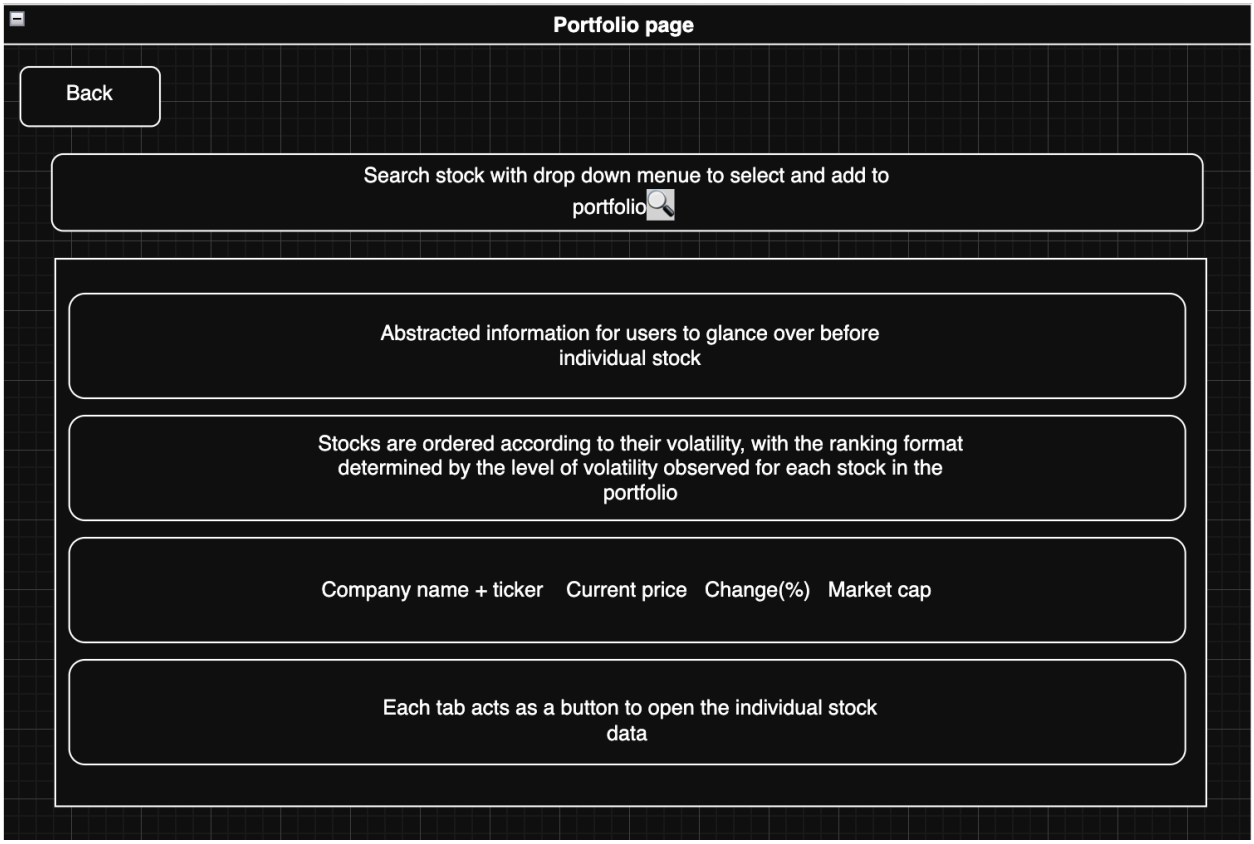
The error message display

The error message provides real-time feedback to users about the errors in the information they entered; this is situated just below the submit button. When a user submits an invalid input example – leaving fields blank, entering mismatched passwords, or typing an incorrect username – the program would trigger an error function. Which would update the user using label text in a user-friendly message?

Justification – the inclusion of errors would improve the usability for the users. By displaying a message immediately after an invalid action, users would be prevented from being frustrated but in turn would be directed to the process of fixing their mistake in the input field.

The inclusion of both visual and text-based indications would provide support to more inclusive groups by providing different accessibility styles. From a security point of view, error messages would prevent the program from going forward with incorrect and incomplete data.

The portfolio page



The search bar

The search function within the portfolio section would allow users to find and track the stocks they are interested in, which would ensure the user is updated on the market performance for the stocks they are interested in. When a user would enter a company name or ticker symbol in the search input field of the search bar and press or click cs the search button, the program would connect to the API to check if the stock is right and exists and to retrieve the historical and current live data.

If the API successfully found the stock the user was looking for, the API would then retrieve key stock information about the given stock example – company name, ticker symbol, current price are displayed on the program.

To make sure the user has selected the intended stock they wanted, a drop-down menu is generated below the search bar. The drop-down would display closely matched results; for example, if user types “apple,” subsequent result such as AAPL – Apple Inc. and APLE – apple hospitality REIT would appear

Once the user crosses the correct stock, they can click the “add” option from the drop-down menu situated inside the correct result. This would immediately add the stock to

their portfolio database. Where it would store the given stock's live data permanently until acted upon by the user and automatically update the portfolio for the stock using an API whenever the portfolio page is open.

If no stock is found or the user enters an invalid ticker, an error message would be displayed:

“Stock was not found. Please try again.”

Justification - the inclusion of the search and add function would allow users to easily locate and track the companies they are interested in without requiring any previous financial knowledge

The drop-down would prevent any confusion to the user between any similarly named companies and prevent them from entering any incorrect stock they did not intend to be added to the portfolio. Moreover, the API would be used for validation to ensure that only verified market data is stored. This would create an interactive user experience that would mimic a typical trading platform for users while keeping it abstracted enough and educational for new investors.

From a technical point of view, the chosen approach would be efficient, as the API only fetched relevant data on demand, which minimized the load for processing. The results are then dynamically updated whenever the user accesses their portfolio, ensuring reliability

Overall, this would make the design a secure and beginner – friendly to build and manage a personalized stock portfolio, supporting the overall project goal with simplified digital market analysis and financial learning.

The interested stock list

The portfolio section will display all the list of the stocks the user has chosen to be updated about and tracked. There are stock entries used that include key information for users, such as ticker symbols, company name, current price, and volatility. The chosen layout is designed for a clear and interactable interface which is scrollable and allows the user to view more stock and add more to the stock portfolio.

A search bar just for users quickly finds the stocks they have already added to the portfolio and is implemented at the top of portfolio section. A search bar just for users to quickly find the stocks they have already added to the portfolio is implemented at the top of the portfolio section. When a user types a company or ticker name into the input field, the program performs a binary search that would go through the stored list and selects a stock.

If a match is found, the stock is brought into view and entry highlighted however, if not found error message is shown.

Additionally, the portfolio would rank and sort the displayed stock based on their volatility, which is the measure of how much the stock price has fluctuated over time. This ranking would users easily identify which of their stocks traded are most volatile and potentially carry potentially more risk or opportunity

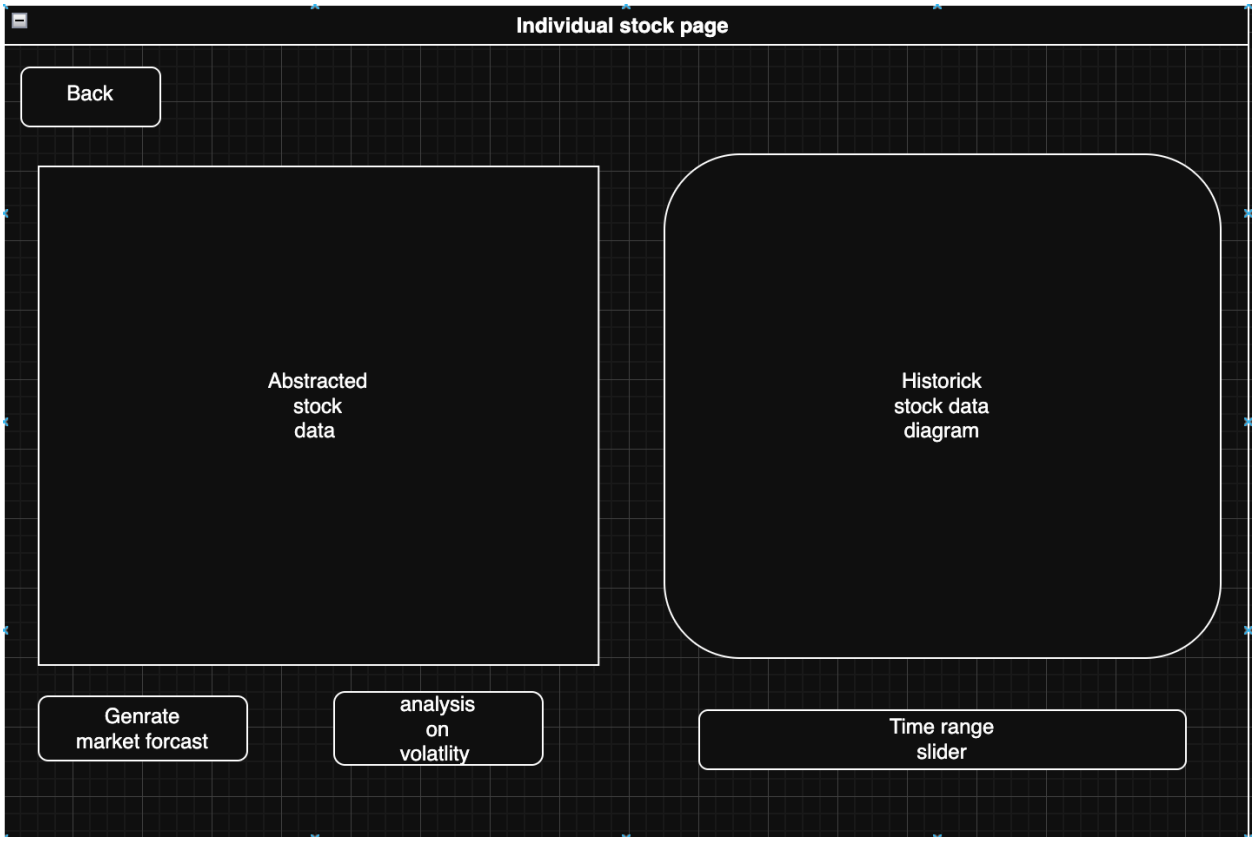
To accommodate a growing number of entries using a scroll bar is integrated, allowing users to smoothly navigate and go through the whole full list without overloading the display window.

The individual stock tab has small, abstracted information of the stock to have quick glance for the user to keep track of the stock.

Justification – the chosen design supports both useability and performance objectives of the project:

- The search feature which allows users to quickly access and find to locate the individual stocks in the stock portfolio page improving accessibility
- The volatility-based ranking would provide an insightful analytical take on the opportunity and risk assessment in real time reinforcing the educational goal of the overall program
- The use of scrollable interface would maintain a clean design while allowing preventing clutter and overwhelming to users as users can properly format and add more stock to the portfolio

Individual stock page



Abstracted stock data

The individual stock page provides a focused insight of the selected stock from the user's portfolio section. It provides the function of presenting abstracted stock data which would give an easy and simplified information of the chosen stock using historical stock data.

When user clicks on a stock in their portfolio section list. The program would retrieve it would retrieve all its data through yfinance API. The data would be then processed and displayed in a proper structured format, showing key abstracted metrics needed shown below:

Abstracted Stock Data
Company: Tesla Inc. (TSLA)
Current Price: $248.32
Open

High
Low
Close
Volume

Justification – The abstracted stock data would improve the usability and understanding to users by presenting complex financial data in an easy and simple abstracted way without overwhelming the users with raw financial data.
From an educational point of view the approach chosen mirrors professional financial dashboard's while also maintaining simplicity for new investors. The page would encourage uses to analyse and compare of how stock metrics are related to market performance.

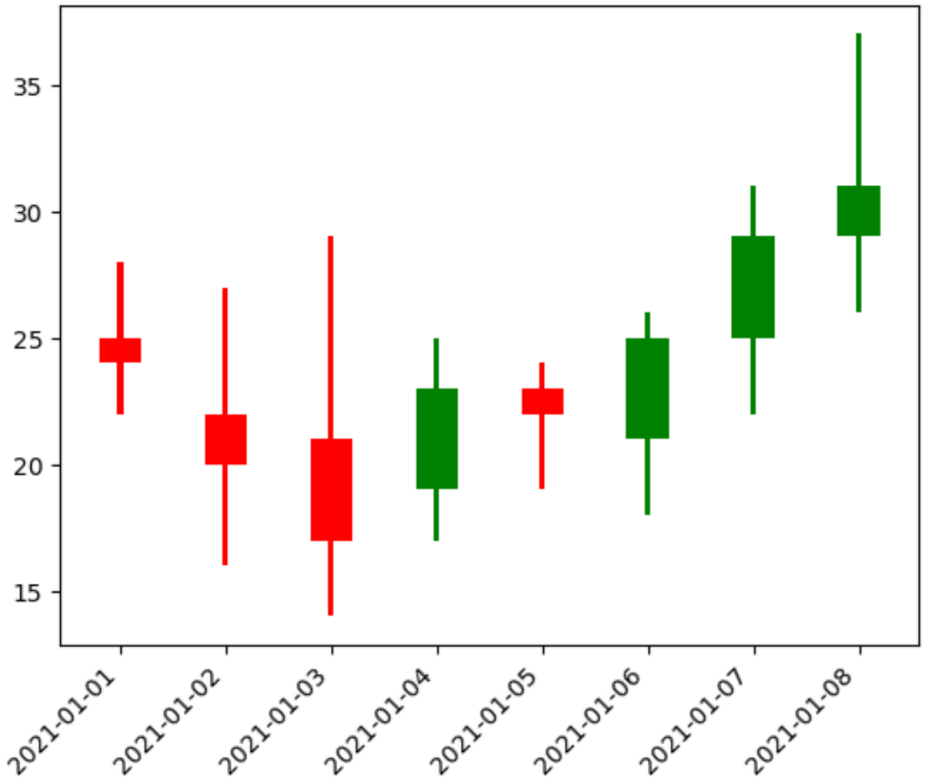
Historical stock data

The historical stock data are plotted using python library matplotlib it plots the trends, fluctuation, and long-term performance patterns.

The data is historical data which is displayed as candle stick using matplotlib the x- axis represent times(dates), and y axis to represent the stock price users can toggle between varying time intervals such as (months, year) to change the data displayed in time scale. Additional visual elements – such as coloured markers for price peaks and dips – are used to enhance inheritability. For example,

- Green markers indicate upward price movement
- Red markers indicate downward movement

The historical stock data diagram will use a candlestick chart because it provides a clear and detailed visual representation of the stock price movement over time, allowing users to observe market trends and volatility effectively, particularly beginners and students who are main stakeholders.



The generated market forecast would generate a predictive closing price in terms of a moving line graph alongside the historical stock data.
The slider could be adjusted to show different timescales of the historical data, such as 1M, 1Y and YTD.

Justification – the historical stock data diagram would transform large amount of numerical data into an abstracted representation improving clarity and understanding. It would allow users to spot performance pattern, volatility, and growth trends without needing to analyse raw numbers

From a learning perspective the diagram would help investors to connect the key ideas showing how market volatility, trading volume, and price fluctuation

It also meets SC7 (performance and efficiency) because only relevant data is retrieved from the API and plotted on demand, reducing unnecessary computation and memory use.

Stakeholders' input on GUI

To ensure that the program GUI is designed to meet the need of the stakeholder and their preferences overall:

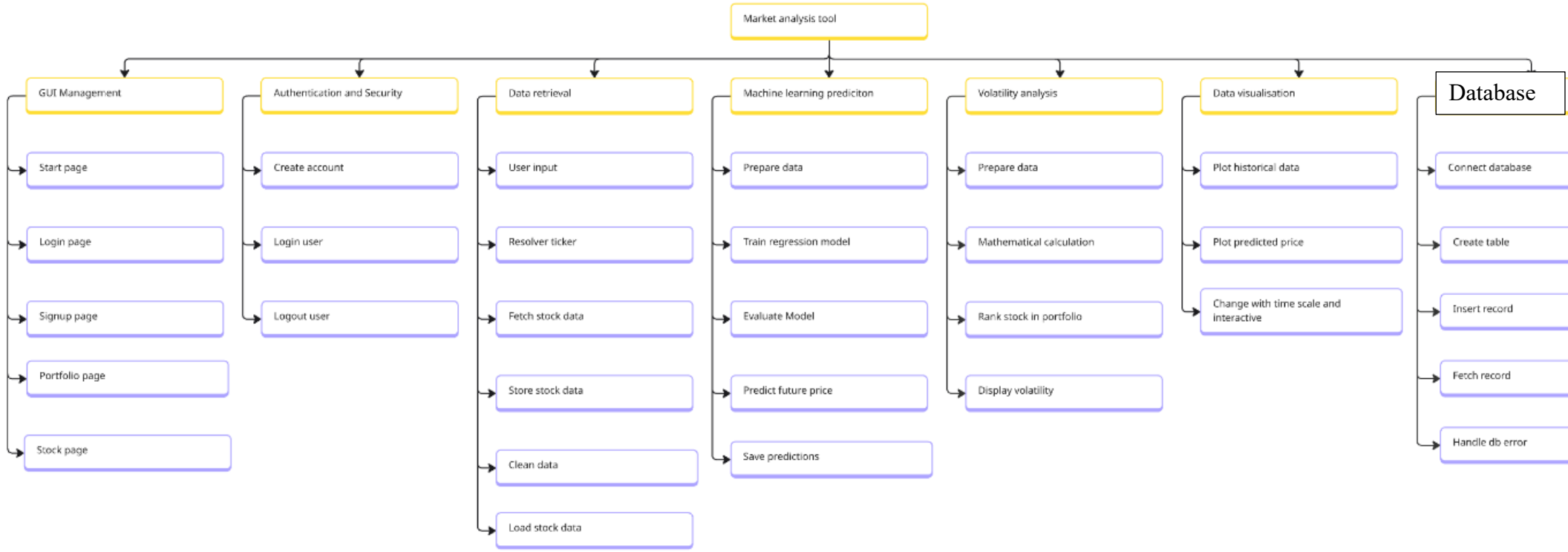
Person 1 – Sarah M. (Beginner Investor)
Sarah found the start page is welcoming and easy enough to navigate through. She also appreciates the brief program explanation and said that it would “make the system feel less intimidating and more approachable.” She also liked features used such as the sign-up and login processes which she described as clearly labelled and straightforward enough, stating that “the layout feels professional and secure.” Sarah also mentioned that the inclusion of helpful error messages, such as incorrect password warnings, made her feel confident that the application would handle mistakes safely.

Person 2 – Priya S. (Casual Investor)
Priya overall said her favorite feature is volatility ranking due to its usefulness of the volatile, as it would help quickly identify which stocks were most risky. She said, “The ranking system may save time — as I don’t have enough time to check it myself each stock’s details manually.”
She also appreciated the visual consistency and color choices as she said with experience with other sites it improves readability and gives sense of security, noting that the interface “felt clean and modern, and the buttons were easy to understand.” Priya suggested adding small icons (such as arrows or symbols) to indicate whether volatility was rising or falling, which was later incorporated into the GUI plan.

Structure Algorithm:

System Decomposition and Work Breakdown Structure

The following is the brief decomposition of the features program for each section there is a sub section there is more in detail view of the decomposition when further exploring each subsection, each subroutine is shown.



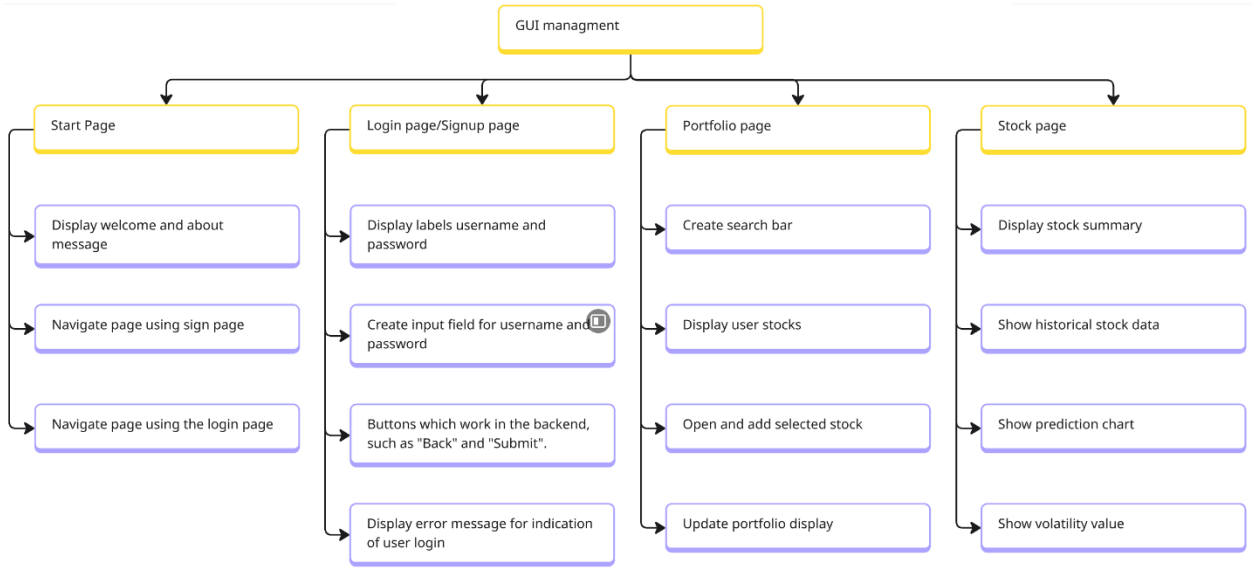
.

GUI management

The GUI is mostly done by the library of Tkinter, which allows rapid development of graphical user interfaces without requiring dependencies. Its object-orientated structure supports modular page management

GUI management handles all user interactions through the graphical user interface. It allows users to navigate and switch through pages such as the start page, login page, signup, portfolio page, and stock analysis. It ensures all the outputs from modules are displayed while making sure the interface is intuitive and easy for beginner stakeholders.

This all the breakdown of all the objects of page and the potential subroutine for each object.



Algorithm Name	Purpose	Algorithm Description	Justification
Login/sign up Validation Algorithm	Authentication of the users input for login username and password before granting access to the system	The algorithm takes the entered username and password, checks if fields are empty, verifies whether the user exists, and then validates the password. If all checks pass, the user is logged in and redirected to the portfolio page.	A step-by-step decision-based algorithm is necessary to ensure security and prevent unauthorized access. Using sequential validation ensures errors are caught early, improving efficiency.
Linear Search Algorithm	locate and search for specific stock within the user's portfolio list where they have added to their stock portfolio	The algorithm would iterate through each item in the portfolio list and compares it with the search query. If a match is found, the corresponding item is highlighted or ranked.	Linear search is appropriate because the dataset is relatively small (user portfolio). It is simple to implement and efficient enough without needing more complex algorithms like binary search.
GUI page update	To update the displayed data in a dynamic manner such that when changes occur	The algorithm is triggered when a user action clicks on the button or other event. It first clears the existing GUI elements such as list boxes or charts to prevent duplication. It then retrieves the updated data and checks for errors or empty datasets. If valid data exists, it iterates through each item, formats it appropriately, and inserts it into the GUI components. This ensures the display reflects the most recent data.	This algorithm ensures consistency between the interface and underlying data. By clearing and repopulating elements, it prevents outdated or duplicated information. Iteration allows efficient handling of multiple data entries, improving clarity and the user experience.
Button Event	To execute specific functions	The algorithm continuously waits for	This algorithm is essential for event-driven

Handling Algorithm	when a user interacts with buttons	user-generated events within the GUI. When a button is clicked, the system identifies which button triggered the event and maps it to a predefined callback function. The corresponding function is then executed, and the system may update the GUI based on the result. After execution, the system returns to a waiting state for further user interactions.	programming, allowing the system to respond dynamically to user actions. It improves responsiveness and interactivity, making the application intuitive and efficient to use.

Algorithm 1

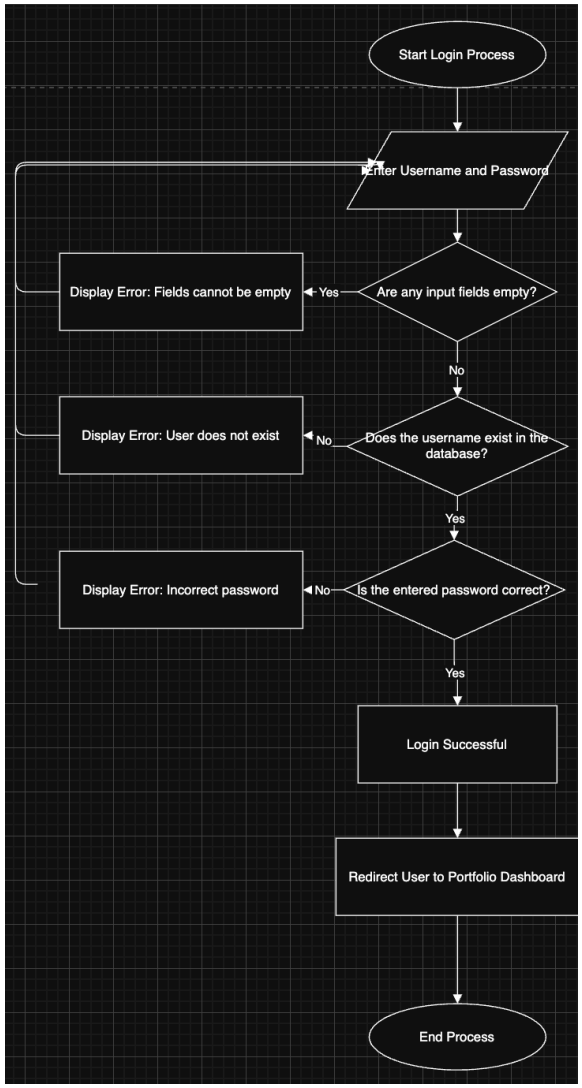
At first the user types in a username and password These are entered through Tkinter Entry widgets (for example: entry username, entry password, entry confirm).

Input validation takes place where the program checks if the user's input are valid this ensures the user entered acceptable and safe data.

Validation	**What it checks**	**Error message shown in GUI**
Empty fields	Username or password box is not filled	“Please fill in all fields.”
Username format	Username which only contains letters/numbers/underscores	“Username can only contain letters, numbers, and underscores.”
Username uniqueness	The username is already present in the database	“This username is already taken.”

Password length	Password is shorter than 8 characters	“Password must be at least 8 characters long.”
Password strength	Password does not contain mix of characters	“Password must include letters and numbers.”
Password match	“Password” and “Confirm password” do not match	“Passwords do not match.”

Flow chart

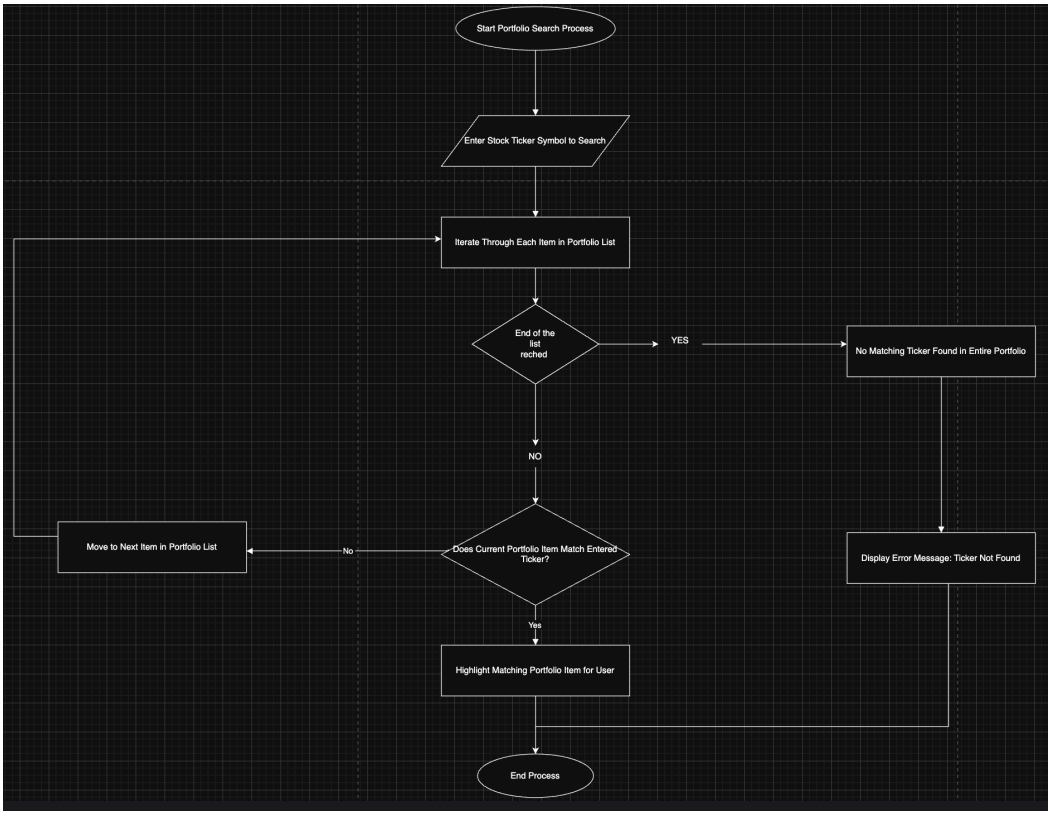


Pseudocode

```
'''
BEGIN LoginProcess

    LABEL PromptInput
    INPUT username, password

    // Check 1: Empty Fields
    IF username IS EMPTY OR password IS EMPTY THEN
        DISPLAY "Error: Fields cannot be empty"
        GOTO PromptInput
    END IF

    // Check 2: Username Existence
    IF username DOES NOT EXIST in database THEN
        DISPLAY "Error: User does not exist"
        GOTO PromptInput
    END IF

    // Check 3: Password Verification
    IF password IS NOT CORRECT THEN
        DISPLAY "Error: Incorrect password"
        GOTO PromptInput
    ELSE
        // Success Path
        DISPLAY "Login Successful"
        REDIRECT user TO Portfolio Dashboard
        END Process
    END IF

END LoginProcess'''
```

Algorithm 2

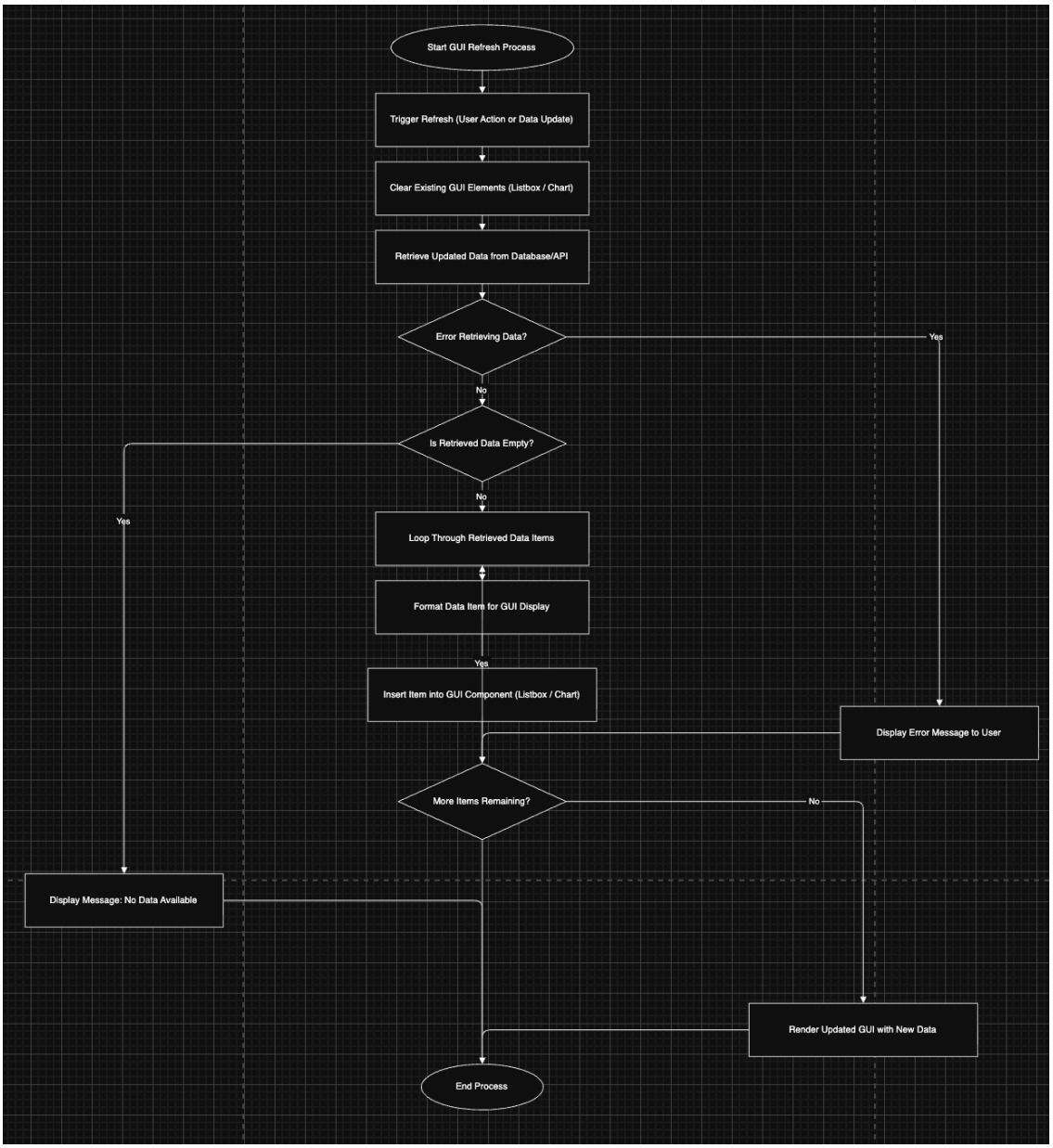
Flow chart



Pseudocode

```
"""
BEGIN PortfolioSearchProcess

    INPUT searchTicker

    // Start of the loop
    FOR EACH item IN portfolioList

        // 1. Check if we have more items to look at
        IF all items have been checked (End of list reached) THEN
            DISPLAY "Error: Ticker Not Found"
            EXIT Process
        END IF

        // 2. Compare the current item
        IF currentItem.ticker EQUALS searchTicker THEN
            HIGHLIGHT item FOR user
            EXIT Process
        ELSE
            // 3. Move forward if no match
            MOVE to next item in list
            REPEAT loop
        END IF

    END FOR

END PortfolioSearchProcess"""
```

Algorithm 3


Start GUI Refresh Process
Trigger Refresh (User Action or Data Update)
Clear Existing GUI Elements (Listbox / Chart)
Retrieve Updated Data from Database/API
Error Retrieving Data?
Yes
No
Is Retrieved Data Empty?
Yes
No
Loop Through Retrieved Data Items
Format Data Item for GUI Display
Yes
Insert Item into GUI Component (Listbox / Chart)
Display Error Message to User
More Items Remaining?
No
Display Message: No Data Available
Render Updated GUI with New Data
End Process

Pseudocode

```
'''
BEGIN GUIRefreshProcess

    // Step 1: Initialize Refresh
    EVENT RefreshTriggered (User Action or Data Update)
    CLEAR GUI elements (Listbox, Chart, etc.)

    // Step 2: Fetch Data
    TRY
        RETRIEVE updatedData FROM Database/API
    CATCH Error
        DISPLAY "Error Message to User"
        GOTO EndProcess
    END TRY

    // Step 3: Validate Data
    IF updatedData IS EMPTY THEN
        DISPLAY "Message: No Data Available"
        GOTO EndProcess
    END IF

    // Step 4: Process and Display Data
    FOR EACH item IN updatedData
        FORMAT item FOR GUI display
        INSERT item INTO GUI Component (Listbox/Chart)

        IF more items REMAINING? THEN
            CONTINUE LOOP
        ELSE
            EXIT LOOP
        END IF
    END FOR

    // Step 5: Finalize
    RENDER Updated GUI with New Data

    LABEL EndProcess
    END GUIRefreshProcess'''
```

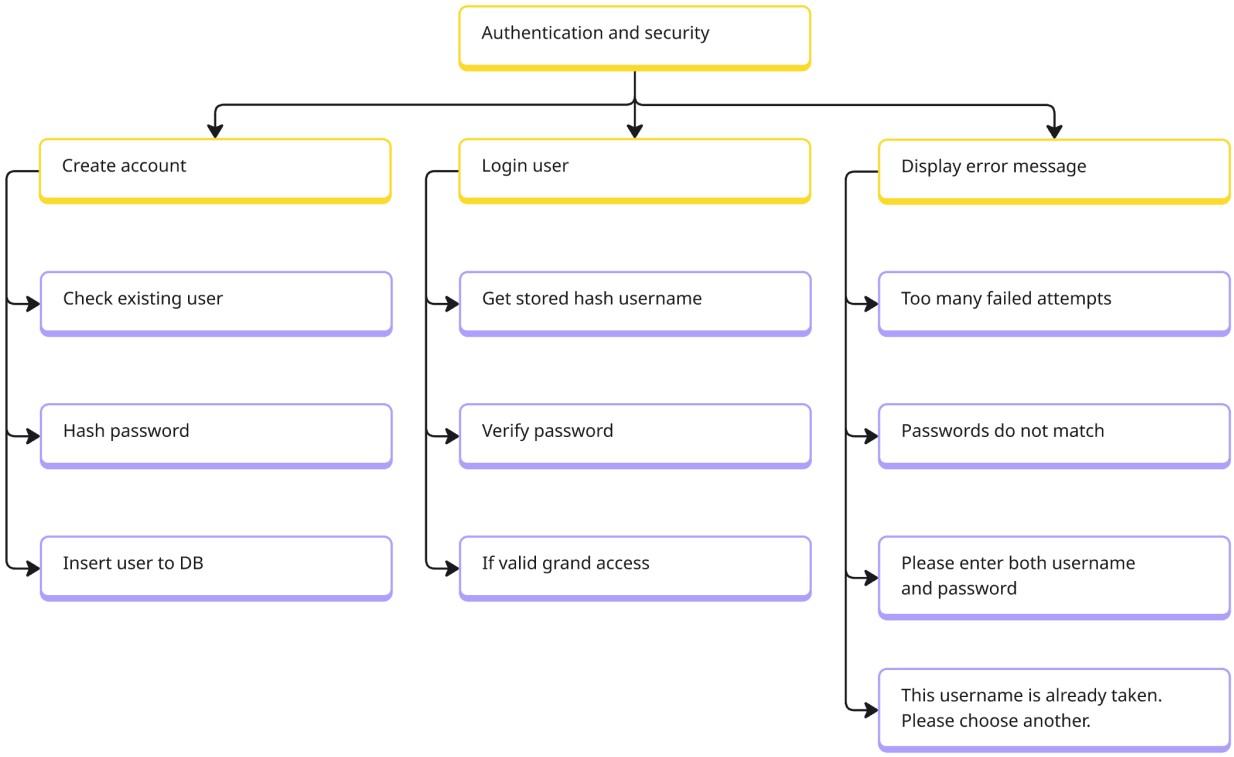
Classes and their function for GUI management

Class Name	Purpose / Description	Validation or Functionality
Market Analysis	The main application is used in each child classes and the main class control all pages and transitions. Inheriting it from Tk.	Controls which page is visible and which page to change using the show page () function. Ensures only one of the pages is active at a time to maintain usability and avoid GUI overlap.
Start Page	The first page displayed to the user. Provides access to login or signup page and gives a brief introduction the program.	No input validation is used. Due to the use of buttons which are validated for correct linkage to navigation functions
Login Page	Allows existing users to log in securely.	Validation ensures both fields are not empty before submission. Displays error messages if input is missing or if incorrect credentials are entered in the submission.
Sign up Page	Allows creation of new accounts and stores credentials securely.	Checks for blank inputs, password confirmation matches, and if the username already exists in the database. Implements password hashing for secure storage.
Portfolio Page	Displays all stocks the user is tracking with volatility ranking.	Validates that stock tickers or company names are entered before adding. Prevents duplicate entries using database lookup. Displays error for invalid or non-existent stocks.

Stock Page	Displays detailed information and graphs for an individual stock.	Validates that selected stock data exists before graph generation. Time slider is restricted to valid options (1M, 6M, 1Y). Displays warning if no data available.

Authentication and security

The decomposition of authentication and security to create login and sign -up page:



Hashing algorithm

The hashing algorithm works by first creating a random salt value, which is a randomly generated 16-bit value made up of random characters and numbers. The salt value is then combined with the user password. Justification – make every password unique and no two users ' passwords stored are equally the same, stopping rainbow table attacks

I chose to use the SHA-256 algorithm to hash user passwords in my program because it provides an elevated level of security and ensures that sensitive data is never stored in plain text. When a user creates an account, their password is combined with a randomly generated salt and then hashed using SHA-256. This converts the password into a fixed-length 256-bit string that cannot be reversed to reveal the original password. The resulting hash, along with the salt, is stored in the database instead of the actual password. During

login, the entered password is hashed again using the same salt, and the output is compared to the stored hash to verify the user's identity.

Justification- The use of SHA-256 is justified because it is a cryptographic-grade hashing algorithm widely recognised for its security and reliability. It has the property of being one-way, meaning it is computationally infeasible to retrieve the original password from the hash. It also demonstrates the avalanche effect, where even a single character change in the password produces a completely different hash, reducing the risk of collisions. By combining SHA-256 with a unique salt for each user, the system ensures that even if two users choose the same password, their stored hashes will be different. This prevents attacks such as dictionary or rainbow table lookups. Overall, implementing SHA-256 helps maintain user trust and data protection by ensuring that passwords remain secure even if the database is accessed by an unauthorised user.

After SHA-256 bitwise manipulation through every character of the input string, mix it with the state using bitwise operations. The first bitwise operation is XOR, which combines the byte and changes bits in an unpredictable way. The second bitwise manipulation done is rotation of bits around, so changes are spread across the whole number. Multiplication introduces more randomness and ensures large bit changes. Together the steps ensure minor changes leads to big differences in the output, called the avalanche effect.

The entire process is repeated multiple times using a parameter called rounds. Each additional round makes sure that the hash is slower to compute, which makes it harder to scramble for the project. 6 rounds are moderately secure.

Justification - XOR manipulation is used in the hashing algorithm because it efficiently increases randomness and data mixing. It compares binary values and changes the output whenever bits differ, ensuring that small input changes produce large variations in the hash. This enhances security by preventing patterns while remaining fast and lightweight for repeated operations.

Finally, the integer state is turned into a fixed-length hexadecimal string of 16 hex digits, which is the final value stored in the database, so later when the user logs in, the program only retrieves the salt and rounds and rehashes the entered password again and compares the result with the result from the database; if they match login successful; if not, the password is invalid.

The pseudocode for the hashing algorithm:

```
'''
# Hashing the password
FUNCTION hashPassword(password)
    salt = generateRandomString(16)
    combined = concatenate(password, salt)
    hashValue = SHA256(combined)
    storedValue = concatenate(salt, "$", hashValue)
    RETURN storedValue
END FUNCTION

# Verifying the password
FUNCTION verifyPassword(inputPassword, storedValue)
    splitValues = split(storedValue, "$")
    salt = splitValues[0]
    storedHash = splitValues[1]

    combined = concatenate(inputPassword, salt)
    newHash = SHA256(combined)

    IF newHash = storedHash THEN
        RETURN True
    ELSE
        RETURN False
    END IF
END FUNCTION'''
```

Login page

The user enters their username and password into input field widget. The input validation would make sure that not field are not blank before it goes through. The program queries the database to locate the username field in the user database using SQLite. Each of the user record would contain the salt, hashed password, and rounds. The password entered by the user would be the program password concatenates the password to find the salt value. The salt string value then processed with SHA-256 hashing algorithm. The same

hashing value would be repeated the same number shown in the stored database rounds such as 6 rounds. The newly calculated hash is compared with the hash value stored in the database it the two given hash value are the same it could mean wither two things which could be that the user has entered correct password for given username or it could be that the salt and rounds of hashing algorithm created same hash value. The login is processed and if successfully confirmed and could be redirected to portfolio page. The program would a temporary session token just to keep track if the user has logged securely during interacting with program itself.

```
'''if entered_hash == stored_hash:
    login_successful()
else:
    display_error("Invalid username or password")'''
```

Classes and subroutines their function for Authentication and security

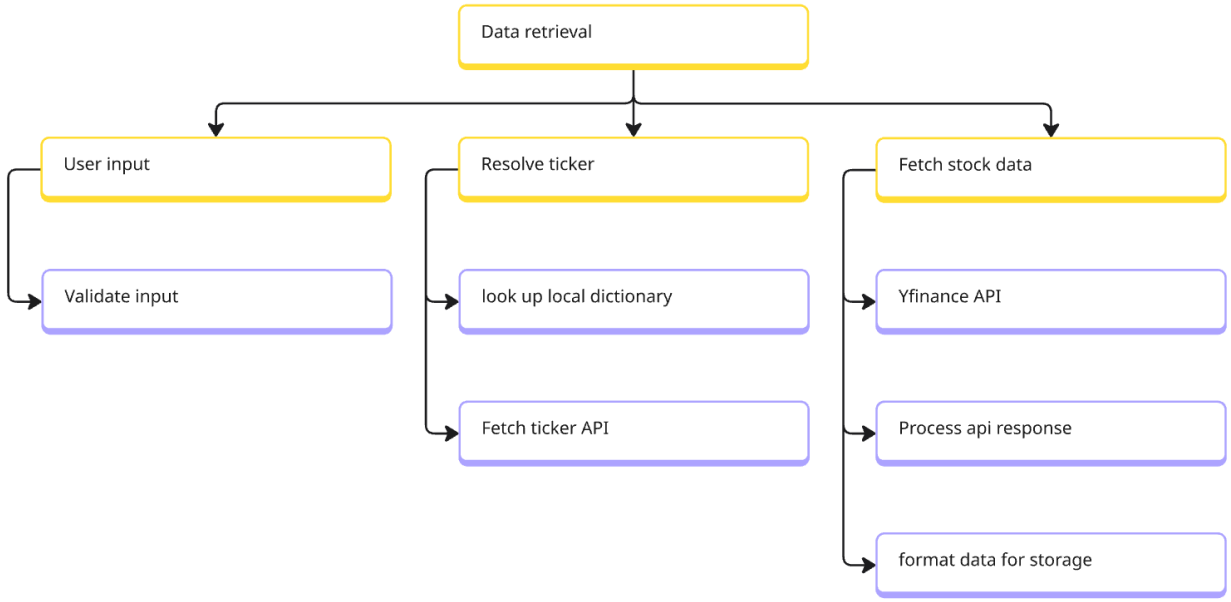
Class Name	Purpose	Key Methods	Justification
Hashing Algorithm	Creates and verifies password hashes using SHA-256 and salt	generate salt (), hash password (), verify password ()	Protects data through one-way encryption and prevents hash-based attacks
User Authentication	Manages login and registration process securely	register user (), login user (), validate input ()	Ensures only validated, securely hashed credentials are processed

Key variables and data structure for GUI management

Variable / Structure	Data Type	Purpose	Justification

user data	Database	Stores usernames, salts, and hashes	Allows fast and secure user lookup
salt	String	Random value added before hashing	Ensures unique hashes, prevents rainbow table attacks
hashed password	String	Final SHA-256 output	Keeps password irreversible and secure
stored value	String (salt + hash)	Combines salt and hash for storage	Simplifies retrieval and verification
input password	String	Temporarily stores entered password	Enables verification, then cleared for safety
Hashing Algorithm	Object	Handles hashing and verification	Improves modularity and separates security logic

Data insertion and retrieval



Data retrieval and portfolio search algorithm

The data retrieval and search mechanism in the given portfolio section would allow user to search a given stock for users using API calls. The algorithm would ensure the relevant and efficient use to fetch relevant instead of caching it locally on the user's personal computer for later use.

1. Search and add algorithm – would retrieve data via API
2. Portfolio data management algorithm – Handles functions such as sorting and memory management

Search and add algorithm

The stock searching algorithm works by first simply taking the user's input which could be either a ticker symbol or company name. A query-based table is created in the database which is independent to any other table designed to hold the most used stock ticker with its name and symbol such that when performing a search, the algorithm just the query's the database using pattern matching. The input can be formatted in any order so that partial matches can be found which would give multiple results for their input and choose the best result. The query then searches the table using the SQLite operators, comparing the input against both the ticker name and company name fields.

If matching results are found, the algorithm retrieves the first valid ticker from the database and uses it to create a yfinance. Ticker object. This ensures that only valid, database-approved tickers are used for the API fetching the historical data and other financial data.

Justification – This approach would ensure that user input is always validated against a trusted dataset before an API request is followed through. By using the SQL wildcard approach the searching, system becomes more user-friendly as it would allow partial or approximate searches instead of requiring exact matches. Additionally, parameterized SQL queries are used, which prevent SQL injection attacks and improve overall security

The algorithm also improves reliability by separating user input from actual data retrieval. Even if a user enters an invalid or misspelled ticker, the system will only proceed if a valid match exists in the database. This prevents errors when fetching livestock data and ensures that the system remains stable.

Adding Stock to Portfolio Algorithm

The algorithm for adding a new stock to the portfolio page begins by first checking whether the stock is already present for the given user. This is done by querying the database using the SQLite and checking for the given user ID and ticker symbol. If the stock does not exist, a new portfolio entry is created and updated to the database respectively.

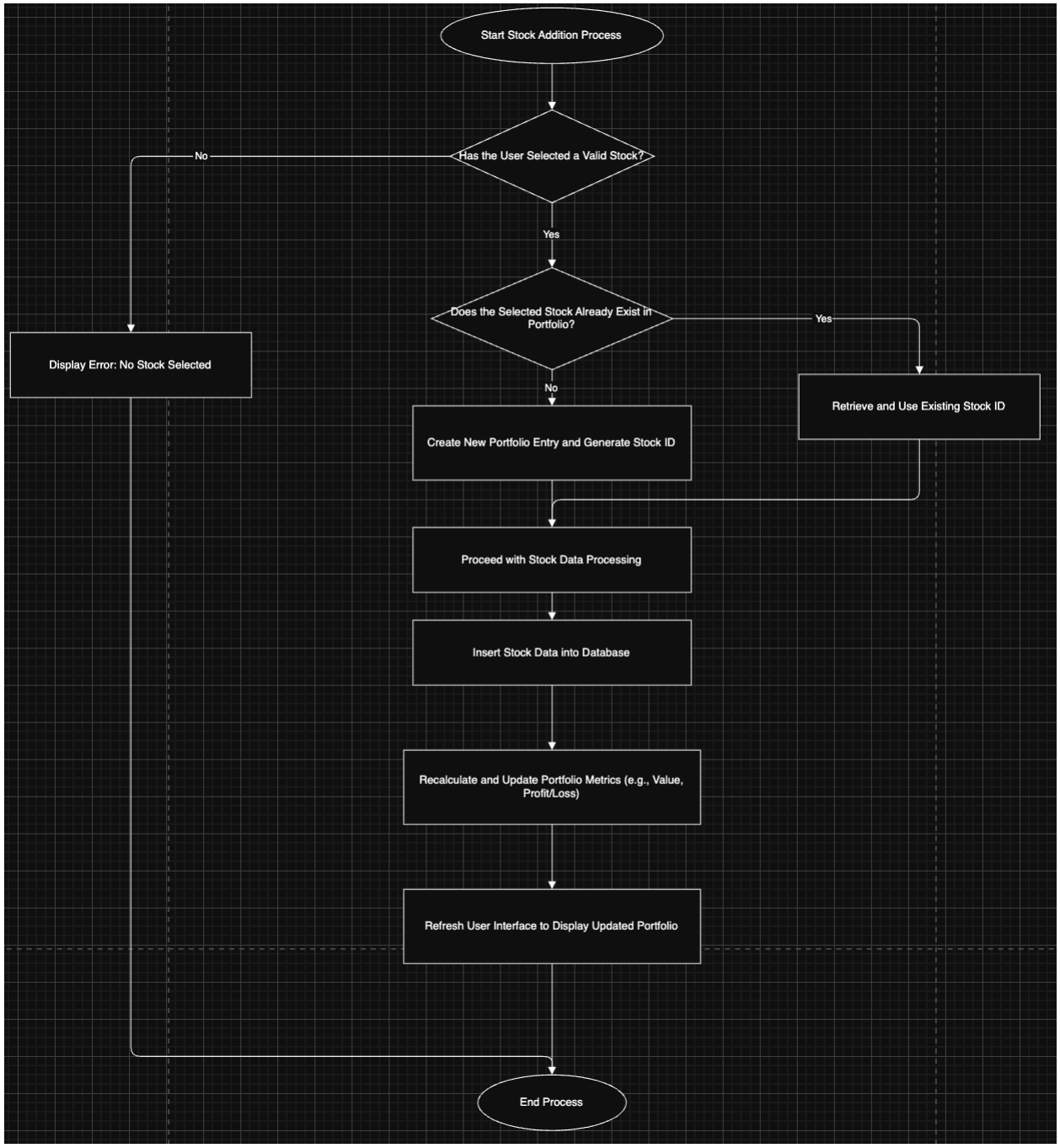
Once the portfolio entry is confirmed or created, the system retrieves historical stock data using the yfinance API. This data includes daily values such as open price, close price, high, low, and volume over a fixed time.

Validation and Integrity Checks

Before inserting any data, the algorithm verifies that the correct portfolio stocks are linked with user. If a mismatch is detected, an error is raised to prevent incorrect data linkage.

Justification – This prevents logical errors and protects user data by ensuring that one user cannot accidentally or maliciously modify another user's portfolio. It also maintains the integrity of relationships within the database

Flowchart:


Start Stock Addition Process
Has the User Selected a Valid Stock?
No
Yes
Display Error: No Stock Selected
Does the Selected Stock Already Exist in Portfolio?
Yes
No
Retrieve and Use Existing Stock ID
Create New Portfolio Entry and Generate Stock ID
Proceed with Stock Data Processing
Insert Stock Data into Database
Recalculate and Update Portfolio Metrics (e.g., Value, Profit/Loss)
Refresh User Interface to Display Updated Portfolio
End Process

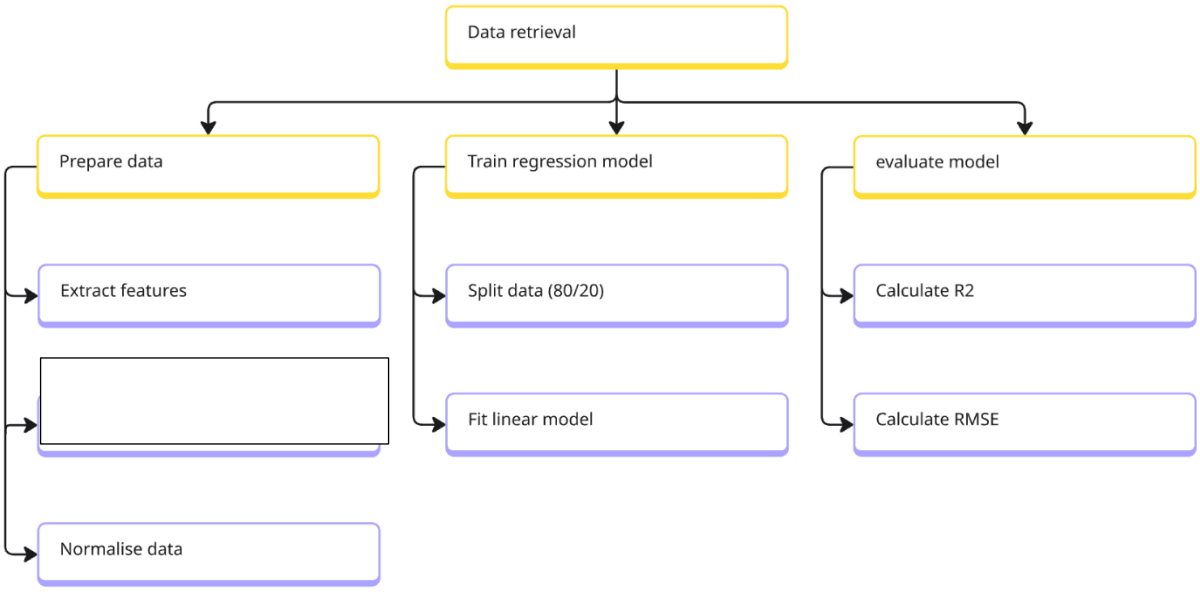
All the key variables and data structures

Name	Type	Purpose	Justification
user input	String	Stores the text typed into the search field.	Allows beginner investors to input either company name or ticker symbol without needing prior knowledge of exact ticker formats.
Api response	Dictionary	Holds the raw data returned by the API after a search.	Enables efficient data retrieval from external APIs in a structured format, allowing validation of user entries.
stock info	Dictionary	Stores extracted information from Api response such as ticker, name, price, and change.	Keeps only necessary details for display and portfolio tracking, reducing complexity for new investors.
portfolio dB	Database Table	Stores all the stocks added by the user to track performance over time.	Ensures beginner investors can easily manage multiple stocks in one place, improving usability and organization.
valid ticker	Boolean	Indicates whether the API has confirmed the stock as valid.	Prevents invalid or non-existent stocks from being added, reducing user error.
search button	Button Object	Triggers the API call when the user clicks “Search.”	Makes the process simple and intuitive for beginners by linking input to action directly.
search entry	Entry Object	Tkinter widget for user input of stock/ticker.	Provides an accessible interface for entering queries; familiar layout helps novice users navigate easily.

suggestion box	List box Object	Displays the autocomplete suggestions or error messages.	Improves user guidance and prevents confusion by confirming valid inputs and showing relevant stocks.
stock_data_frame	or Data Frame	Temporarily stores fetched stock data (price, volatility, etc.) before it is displayed.	Helps manage multiple stocks' information efficiently without cluttering the interface for beginner users.
selected_stock	Dictionary	Stores details of the user-selected stock from suggestions or portfolio.	Makes it easy to pass the selected stock's details between functions for analysis or display.
error message	String	Holds error prompts such as "Stock not found, try again."	Provides clear feedback to beginner investors, enhancing usability and minimizing frustration.

Classes

Machine learning prediction



The predictive component of the market analysis was developed using a machine learning model of linear regression implemented with the algorithm of the Scikit-learn library, while libraries such as Pandas and NumPy are used to handle preprocessing and mathematical operations.

The regression algorithm works by first retrieving the historical stock data from the database using a SQL query. The query then selects the date and closing price for a specific portfolio and orders the data chronologically. This ensures that the dataset reflects the real-time sequence of stock prices, which is essential for time-series prediction.

Justification – Ordering the data by date ensures that the model learns from past trends in the correct sequence. Without this, the regression model would not capture the temporal relationship between stock prices, reducing prediction accuracy.

Data Preprocessing

The algorithm then preprocesses the data by converting the dates into a datetime format which is the format the model requires to come up with the regression while also removing any invalid or missing values. A new feature called Prev Close is created by shifting the closing price column by one position. This means each row now contains the previous day's closing price, which is used as the input feature for the model.

Justification – Creating a lag feature allows the model to learn the relationship between consecutive days, which is a simple but effective method for time-series forecasting. Removing invalid or missing values ensures data quality, preventing errors during model training.

The model training

The feature-engineered data is split into 80% training and 20% subsets. This ensures the model achieves the computational method of data mining, and finding a pattern between

the data to allow for prediction can occur while also allowing some labelled data to be used to test and evaluate the model and allow users to see how to improve the model and how accurate the prediction is.

The regression algorithm used is ordinary least squares (OLS). The training algorithm computes the optimal coefficients (weights) w and an intercept b for an accurate linear regression model.

$$\hat{y} = wX + b$$

The y represents the predictive closing price, and the algorithm minimises the mean square errors between predicted and actual values:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The scikit-learn library internally uses NumPy using linear algebra operations to optimise problems efficiently, providing a closed-form analytical solution for w and b.

Justification - scikit learn library is used as the primary machine learning framework due its scope of providing prebuilt algorithms and data processing tools allowing efficient use of structured stock data the scikit – learn supports supervised learning while working seamlessly with liberies such as panda and NumPy which makes data handling easier smoothly transitioning from raw data to predictive data.

Once the model is trained the model's accuracy is evaluated using the test data set using

- R^2 measures how well the regression line fits the data
- RMSE quantifies average prediction errors in same unit as the output

MSE measures the average squared error, indicating how far predictions are from actual values, while R^2 measures how well the model explains the variance in the data. These metrics allow the programmer to assess the accuracy and reliability of the model.

Justification - This regression-based model, combined with Pandas and NumPy, ensures efficient data processing and accurate predictions. It allows beginner investors to visualise potential trends in a clear, data-driven manner without requiring financial expertise. By using supervised learning with real stock data, the model enhances user confidence, supports informed decision-making, and makes complex machine learning concepts accessible through a simple, interactive interface.

Recursive Forecasting

To generate the future predictions, the algorithm also uses the recursive forecasting object/function. It starts with the most recent closing price and uses the trained model to predict the next day's price. This predicted value is then fed back into the model as the input for the next prediction.

This process is repeated for a specified number of future days.

Justification – Recursive forecasting allows the model to generate multiple future predictions using only a single trained model. Although errors may accumulate over time, this method is efficient and suitable for short-term forecasting, making it appropriate for this project.

Machine learning variables and data structures needed

Variable Name	Data Type	Purpose / Description	Justification
stock data	Data Frame	Stores all retrieved stock data (Date, Open, High, Low, Close, Volume).	Central dataset for feature generation and model training.
Time	NumPy Array	Store independent variables (e.g., moving average, volume, RSI, volatility).	Input data for model training and prediction.
Closing price	NumPy Array	Stores dependent variable — typically next-day price or price change.	Uses as ground truth for the supervised learning.
Prediction model	Object	Holds the trained ML model (e.g., regression or neural net).	Allows consistent prediction and retraining without reloading architecture.

predicted price	Float	Stores the model's predicted output value for future stock prices.	Used for portfolio visualization and feedback to users.
volatility index	Float	Represents the computed volatility level of the stock based on recent price data.	Helps assess risk and inform sorting algorithms in the portfolio.

Volatility analysis

The historical data is first loaded from the database where after retrieval is completed an algorithm logarithmic return which is a method of calculating an investment's return using the natural logarithm of the ratio of the ending price to the beginning price.

The mathematical formula is following:

$$r = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Where Pt is the current price and Pt-1 is the previous price.

Justification – Logarithmic return is utilized of a alternative model os simple percentage returns as the current approach is more time additive and more suitable for financial modelling.

Rolling Volatility Calculation

The algorithm then calculates volatility using a rolling window approach. A range of data is used to take subsets of the log returns. For each window, the standard deviation of returns is calculated using a statistical function.

This standard deviation is then annualised using the formula:

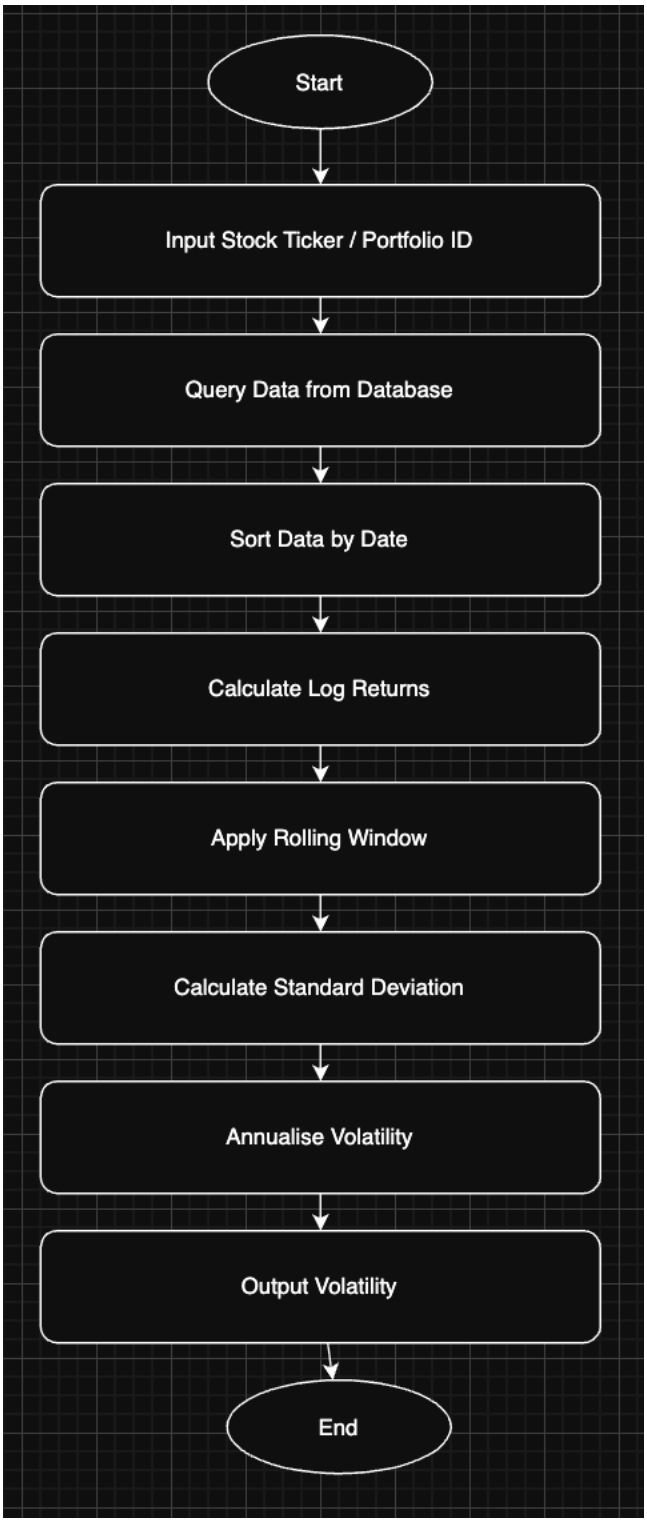
$$\text{Volatility} = \sigma \times \sqrt{252}$$

Where 252 is the number of trading days in a year.

Justification – A rolling window would make the algorithm able to track how the volatility changes over time rather than introducing a single static value. Standard deviation is used because it measures how spread-out returns are, which directly represents market risk.

Annualizing the volatility makes the result easier to interpret and compare across different stocks.

Flow chart:



when the volatility is calculated the stocks are ranked in the portfolio section using quicksort where the greatest volatility in the stocks is at the top of the portfolio table.

Justification – Quick sort has average time complexity of O(nlog n) making it very efficient in terms of time compare to other similar sorting algorithms comparatively helping to achieve the success criteria of

Pseudocode:

```
"""
# The main function to start the sort
FUNCTION sortPortfolioByVolatility(portfolioList)
    IF length of portfolioList is less than or equal to 1 THEN
        RETURN portfolioList
    END IF

    # Step 1: Pick a pivot (middle item)
    pivot = portfolioList[middle index]

    # Step 2: Create three groups
    higher = all stocks with volatility GREATER than pivot
    equal = all stocks with volatility EQUAL to pivot
    lower = all stocks with volatility LESS than pivot

    # Step 3: Recurse and combine for descending order
    # (Higher goes first to put the most volatile at the top)
    sortedList = join(sortPortfolioByVolatility(higher), equal, sortPortfolioByVolatility(lower))

    RETURN sortedList
END FUNCTION"""
```

Database for the backend

The database acts as the centre for storage and fetching system throughout the program it ensures the user information such as:

- stocks – stores static stock information such as ticker, company name, and sector.
- historical data – holds time-series stock prices fetched via the API (Date, Open, High, Low, Close, Volume).
- predictions – stores predicted values generated by the regression model for future dates.
- user portfolio – tracks which stocks users have added to their watchlist or portfolio.

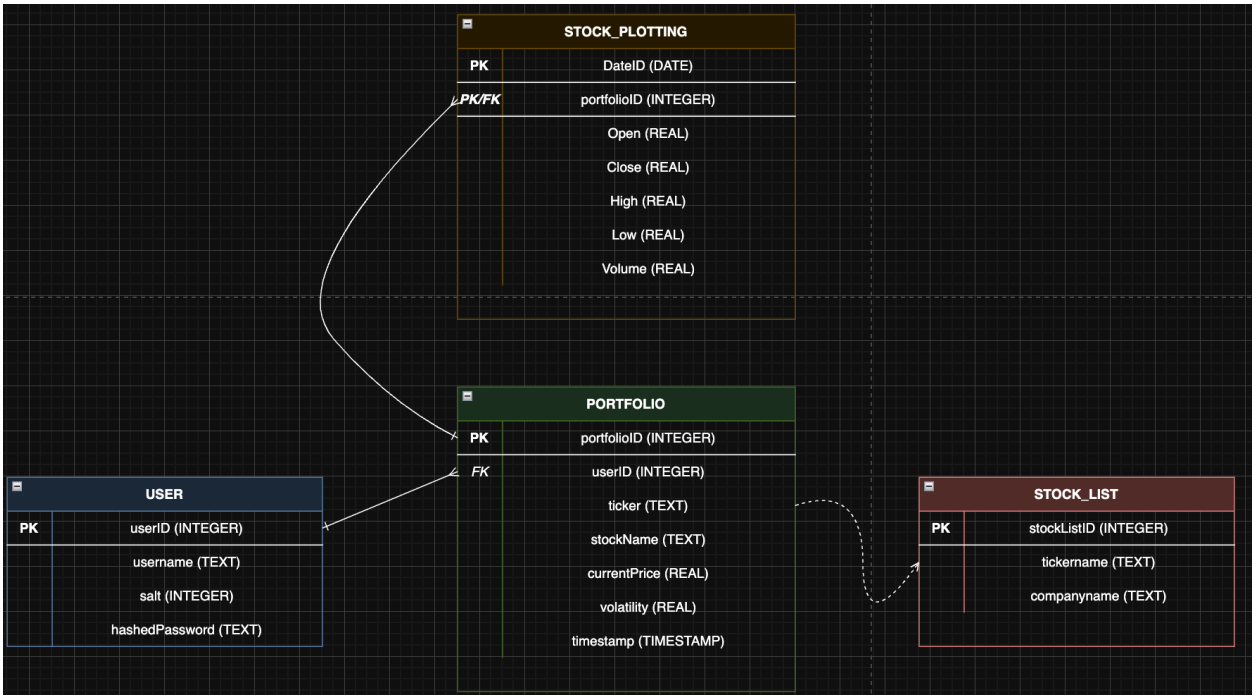
Are saved as securely and efficiently. It provides a backbone to the programs architecture unlike temporary program variables that are erased when the program is terminated causing users to potentially lose their data whereas database provides a persistent data management structure which retains all the information after it is reopened and

terminated. This builds confidence and trust to users that their information is not lost and stored in a reliable way,

SQLite is used as the programs database management system (DBMS) it would store all data locally in. dB file which can be accessed through Python's sqlite3 library. It is responsible for storing, retrieving, and managing data without requiring external server

Technical Justification:

SQLite model does not require further hardware such as dedicated server setup which is expensive. Making SQLite more suitable due no further hardware required achieving the same function. Moreover, it supports to any hardware allows the program to be run on any stakeholder's computer which can run Python increasing the accessibility of the program to any users. SQLite also ensures that all the transactions of the database are ACID compliant (Atomic, consistent, isolated, and durable) which is the critical for data to be reliable. The built-in module sqlite3 python module also allows program to achieve CRUD function (Create, Read, Update, Delete) operations directly within the program. SQLite also parameterises the program queries protecting the users from SQL injection attacks increasing user security under sensitive data.



Pseudocode for the ERD

```
"""CREATE TABLE Users (
  user_id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE,
  salt TEXT,
  hashed_password TEXT
);

CREATE TABLE Portfolio (
  stock_id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER,
  ticker TEXT,
  company_name TEXT,
  price REAL,
  volatility REAL,
  timestamp DATETIME,
  FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);"""
```

Test data for iterative development

During the iterative development process, different and several types of test data are used to verify the correctness, robustness, and reliability for both the database and algorithmic components. The chosen test data ensures that all features, such as data retrieval, storage, user authentication and prediction function, work under different scenarios

Valid Test Data

Purpose:

To verify that the system correctly handles valid, expected inputs, and returns accurate outputs.

Example:

Feature Tested	Example Test Data	Expected Output	Justification
User registration and login	username="mandar", email="mandar@example.com", password="Stock123"	User successfully registered and logged in.	Ensures that the authentication system functions correctly under normal use.
Stock search	ticker="AAPL"	Returns Apple Inc. data from yfinance.	Confirms correct API integration and data fetching.

Justification – ensure the system performs the expected work under normal user interaction when interacting with beginner investors who depend on accurate and intuitive process

Invalid test data

Purpose:

To check how the program handles incorrect or unexpected inputs without crashing.

Feature Tested	Example Test Data	Expected Output	Justification
User registration	username="", email="invalidemail", password="123"	Error message displayed, registration rejected.	Ensures input validation and error handling for form fields.
Stock search	ticker="XXXX"	Displays “Invalid ticker, please try again.”	Tests API response handling and error feedback.

Justification – prevents the program from crashing and data corruption This is beneficial for beginner investors, who benefit from clear error messages to guide them to correct mistakes.

Boundary test data

Purpose:

To test edge cases and the system’s response to extreme or limit values.

Feature Tested	Example Test Data	Expected Output	Justification
Portfolio limit	100 stocks added	System still performs efficiently.	Ensures scalability of database and GUI responsiveness.

Stock price inputs	price=0 or price=999999	System handles data gracefully without overflow or crash.	Tests robustness of numerical storage and visualisation.
Password length	8–64 characters	Only within this range accepted.	Confirms validation rules work correctly.

Justification – A boundary ensures stability of the program under extreme cases while maintaining reliability and useability as dataset grows.

Iterative Testing Process

Testing was performed after each major implementation phase:

- Phase 1: Database creation and connection testing using dummy data.
- Phase 2: Stock retrieval testing from yfinance API.
- Phase 3: Portfolio insertion, update, and delete testing.
- Phase 4: Prediction model verification using real stock data.
- Phase 5: Plotting and the predictive model.
- Phase 6: Volatility calculator
- Phase 7: Integrated testing combining GUI, database, and API responses.

Further data for data to be used in post-development

After the initial development, more data is incorporated to improve the model's prediction accuracy and portfolio analysis. The additional data inputs will incrementally improve the program's robustness while also ensuring that market behaviour is making the application more user-friendly for beginner investors.

Extended historical data

Description: large datasets from the yfinance API giving more historical data, such as 5 and 10 years, not only for a few months, giving more data for making better predictions of closing price.

Justification

- A large dataset allows the model to detect higher long-term patters and trends not seen before
- Reducing overfitting while improving prediction accuracy
- Stakeholders receive more reliable prediction and ignoring recent contextual short-term fluctuations

Sentimental analysis data

Description: using web scraping on news websites to extract news headlines and tokenising the words and giving a sentimental score to the given stock, providing more analysis where the news plays a huge role in market fluctuation and is one of the weaknesses of the current program.

Justification

- Making sentimental data helps to identify a potential market volatility and fluctuation
- Stakeholder are given another parameter to affect their investments; it also allows them to see if the company they investing are ethically right and environmentally sustainable

User Portfolio Historical Data

Description:

Historical performance records of each user’s portfolio are stored in the local SQLite database.

Justification:

- Enables analysis of user investment patterns and risk tolerance.
- Can be used post-development to personalise recommendations (e.g., suggest safer or riskier stocks based on history).
- Stakeholder Impact: Improves the personalisation of the system, helping beginners learn investment strategies suited to their behaviour

Implementation

Database

The database is the first backend to be programmed due it being backend of data processing and storing.
The code for basic database using SQLite without any other connection:

Creating the table

Creating the tables according to the entity relationship diagram for the tables and their fields in the database: this code was worked on from 10th November 2025 12th November

Errors encountered and fixed:

- self. cursor_obj.execute("PRAGMA foreign keys = ON;") the following line of code was added as the following error was displayed: the cursor couldn't link the fields and table together as SQLite does not enforce foreign key constraints by default however after research the logic error was fixed by enabling the foreign key
- After research and testing later with insertion of data SQLite does not have the function FLOAT but rather uses the REAL so all the fields containing FLOAT.

Creating table final code:

```
import sqlite3
class DatabaseManager:
    def __init__(self):
        super().__init__()

        self.connect_db = sqlite3.connect('Marketanalysis.db')
        self.cursor_obj = self.connect_db.cursor()

        self.cursor_obj.execute("PRAGMA foreign_keys = ON;")

        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS USER(
            userID INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            salt INTEGER NOT NULL,
            hashedPassword TEXT NOT NULL
        );
        """)

        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS PORTFOLIO(
            stockID INTEGER PRIMARY KEY AUTOINCREMENT,
            userID INTEGER NOT NULL,
            ticker TEXT NOT NULL,
            stockName TEXT,
            currentprice REAL,
            volatility REAL,
            timestamp TIMESTAMP,
            FOREIGN KEY (userID) REFERENCES USER(userID)
        );
        """)

        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS STOCK_HISTORY(
            historyID INTEGER PRIMARY KEY AUTOINCREMENT,
            stockID INTEGER NOT NULL,
            currentPrice REAL NOT NULL,
            marketcap REAL NOT NULL,
            wrange REAL NOT NULL,
            closingprice REAL NOT NULL,
            volume REAL NOT NULL,
            date TEXT,
            predictedprice REAL,
            FOREIGN KEY (stockID) REFERENCES PORTFOLIO(stockID)
        );
        """)
```

```
self.cursor_obj.execute("""
CREATE TABLE IF NOT EXISTS STOCK_LIST(
    stockListID INTEGER PRIMARY KEY AUTOINCREMENT,
    tickername TEXT NOT NULL,
    companyname TEXT NOT NULL
);
""")
```

Inserting data and fetching data manipulation

The database gets inserted data in many ways throughout the program and many of this data are either. As the program is in its developmental stages other functions are not available to test with real data so just an overview of how the function would work is shown here: final code worked on between 13th to 14th November 2025

Changes made and log of changes:

1st

```
self.username = "mandar"
self.salt = 132
self.hashedpassword = "has"
self.cursor_obj.execute("""INSERT INTO USER(username, salt, hashedPassword) VALUES(?, ?, ?)""", (self.username,self.salt,self.hashedpassword))
```

This the first attempt to add data to the user table of the database the error encountered by this method:

- Every time the program is run the data is duplicated multiple times causing duplicate users and inconsistent data. Moreover, causing error when the foreign key is confused which user to link to causing a logic error.

```
    self.username = "mandar"
    self.salt = 132
    self.hashedpassword = "has"
    self.cursor_obj.execute("""INSERT INTO USER(username, salt, hashedPassword) VALUES(?, ?, ?)""", (self.username,self.salt,self.hashedpass

    existing = self.cursor_obj.fetchone()

    if existing is None:
        self.cursor_obj.execute(
            "INSERT INTO USER(username, salt, hashedPassword) VALUES(?, ?, ?)",
            (self.username, self.salt, self.hashedpassword)
        )
        self.connect_db.commit()

    self.cursor_obj.execute("SELECT * FROM USER;")
    rows = self.cursor_obj.fetchall()
    for row in rows:
        print(row)

DatabaseManager()
```

2nd

This was the attempt to remove duplication of the data in the user table which would be cross added to every field it also serves as user input validation.

- Error still occurring of the duplication as the self.cursor_obj still registering it to the database causing the user data to be added twice
- The existing function is not playing in the role to find duplication due to the different user id and other unique terms.

3rd

```
self.username = "mandar"
self.salt = 132
self.hashedpassword = "has"
self.cursor_obj.execute("""INSERT INTO USER(username, salt, hashedPassword) VALUES(?, ?, ?)""", (self.username,self.salt,self.hashedpassword))
existing = self.cursor_obj.fetchone()

if existing is None:
    self.cursor_obj.execute(
        "INSERT INTO USER(username, salt, hashedPassword) VALUES(?, ?, ?)",
        (self.username, self.salt, self.hashedpassword)
    )
    self.connect_db.commit()

self.cursor_obj.execute("SELECT * FROM USER;")
rows = self.cursor_obj.fetchall()
for row in rows:
    print(row)
```

Final change which successfully prevents data to be repeated as the program is run every time. It also achieves the fetching function test. More testing is done after integration with the data retrieval and other data counterparts.

User selection for interested stocks for portfolio page

This is an example of fetching required from the database from prebuilt records of all stocks in the database being searched based on what user searched for in the portfolio page so they can add it to their portfolio and return matching results from the database. 1st attempt on 17th November 2025 – 18th November 2025

```
venv > data_retrival.py > DataRetrieval > stock_search
1   import sqlite3
2   import yfinance as yf
3   import pandas as pd
4
5
6   class DataRetrieval:
7       def __init__(self):
8           self.connect_db = sqlite3.connect("Marketanalysis.db")
9           self.cursor = self.connect_db.cursor()
10          self.cursor.execute("PRAGMA foreign_keys = ON;")
11
12      def stock_search(self, ticker):
13          query = """SELECT * FROM STOCK_LIST WHERE tickername LIKE ? OR companyname LIKE ?;"""
14          like_value = f"%{ticker}%"
15          self.cursor.execute(query, (like_value, like_value))
16          self.comapany = yf.Ticker(ticker)
17          self.company_name = self.comapany.info['symbol']
18          return self.cursor.fetchall()
19
20  q = DataRetrieval()
21  results = q.stock_search("AAPL")
22  for row in results:
23      print(row)
24
```

Note – The company get info is used to search the database if the queries are valid in terms of the API by validating it to get results.

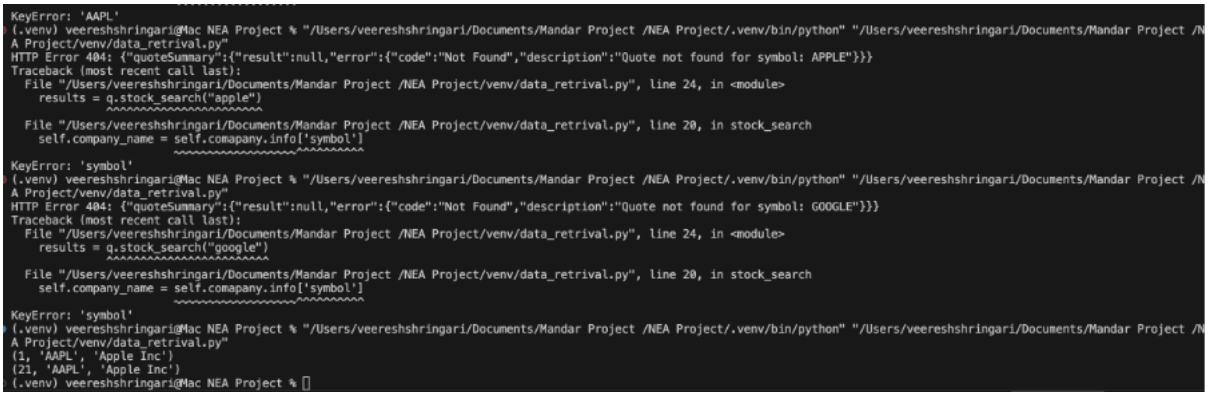
Errors encountered, fixed and still occurring:

- During testing the program, a KEYERROR occurred when accessing the metadata of the stock this was caused due to a wrong assumption of the API structure changed

from self.company_name = self.comapany.info['ticker'] to self.company_name = self.comapany.info['symbol'].

- The database is being queried before checking the API if such a ticker exists. As seen with testing with different queries outside the database or incorrect and more optimal approach was found to be use the queried database result as an input for the API. Test cases uses where = “AAPL,” “GGL,” “apple”
- No error handling is occurring when API fails to return the required stock data which caused the program to crash as seen from the previous test with incorrect tickers the program crashed for “apple” and “google” as they are not valid input for the API
- The search functionality does not give the best results seen with test cases it only accepts comparable results and not to distinct results mostly queried by stakeholders who are not particularly experienced. This will be fixed in future attempts using partial string match and ranked results.

Result of test case:



2nd attempt
Worked on from 19th - 25th November 2025

The program successfully addressed most of the previous errors and improvements needed to be made.

```python
import sqlite3
import yfinance as yf
import pandas as pd
class DataRetrieval:
    def __init__(self):
        #connects to the database
        self.connect_db = sqlite3.connect("Marketanalysis.db")
        self.cursor = self.connect_db.cursor()
        self.cursor.execute("PRAGMA foreign_keys = ON;")
    def stock_search(self, ticker):
        query = """SELECT * FROM STOCK_LIST WHERE tickername LIKE ? OR companyname LIKE ?;"""
        like_value = f"%{ticker}%"
        #Parameterised queries were used to improve security and prevent malicious SQL input.
        self.cursor.execute(query, (like_value, like_value))
        results = self.cursor.fetchall()
        # Use ticker from database result instead of raw user input
        if results:
            db_ticker = results[0][1]
            self.comapany = yf.Ticker(db_ticker)
            try:
                # prepares an API request and safely retrieves the ticker symbol
                self.company_name = self.comapany.info.get('symbol', None)
            except Exception:
                self.company_name = None
        else:
            self.company_name = None
        return results
q = DataRetrieval()
results = q.stock_search("GOO")
for row in results:
    print(row)
```

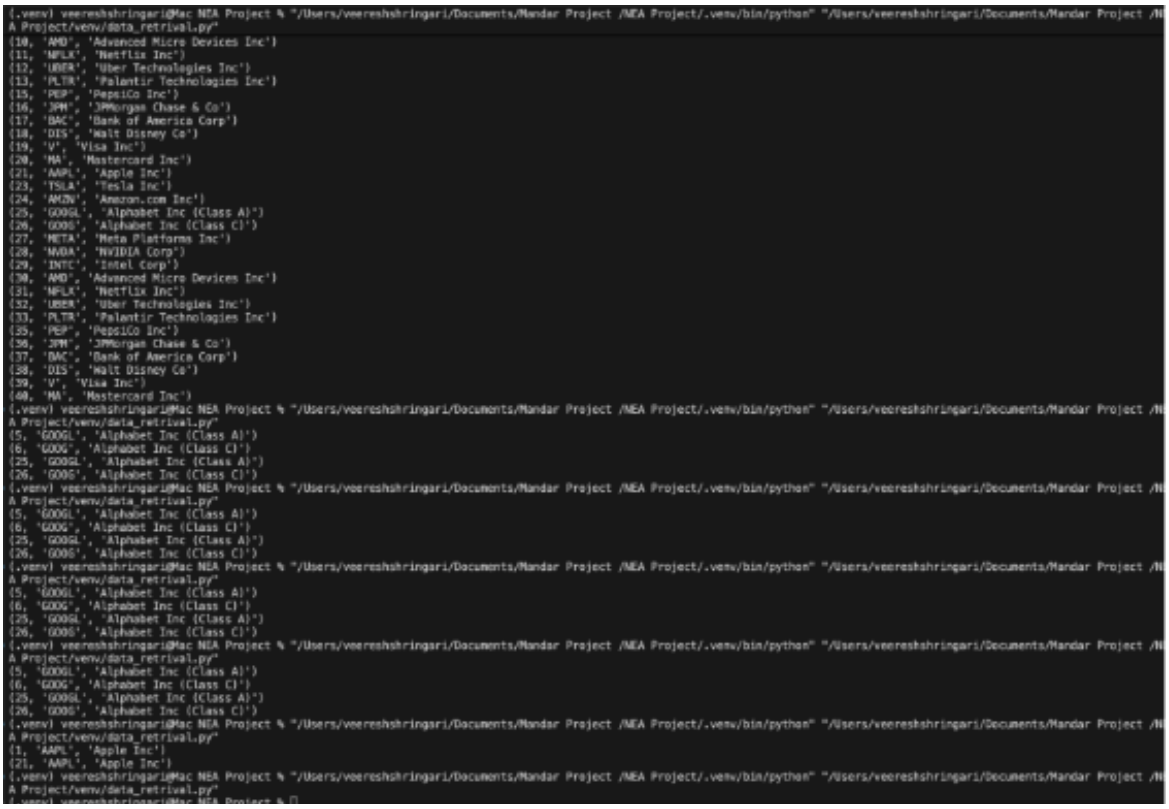
Test cases and result:

To test the database a test of data was added to the STOCK_LIST to acting as backend searching algorithm:

```python
self.cursor_obj.executemany(
    "INSERT OR IGNORE INTO STOCK_LIST (tickername, companyname) VALUES (?, ?);",
    [
        # Major US stocks
        ("AAPL", "Apple Inc"),
        ("MSFT", "Microsoft Corporation"),
        ("GOOGL", "Alphabet Inc"),
        ("AMZN", "Amazon.com Inc"),
        ("META", "Meta Platforms Inc"),
        ("NVDA", "NVIDIA Corporation"),
        ("TSLA", "Tesla Inc"),
        ("BRK-B", "Berkshire Hathaway"),
        ("JPM", "JPMorgan Chase"),
        ("V", "Visa Inc"),

        # UK / EU stocks
        ("BP", "BP plc"),
        ("HSBA", "HSBC Holdings"),
        ("SHEL", "Shell plc"),
        ("ULVR", "Unilever"),
        ("AZN", "AstraZeneca"),
        ("RIO", "Rio Tinto"),
        ("BHP", "BHP Group"),
        ("NESN.SW", "Nestlé"),
```

```
            # Indices
            ("^GSPC", "S&P 500"),
            ("^DJI", "Dow Jones Industrial Average"),
            ("^IXIC", "NASDAQ Composite"),
            ("^FTSE", "FTSE 100"),
            ("^N225", "Nikkei 225"),

            # Commodities
            ("GC=F", "Gold Futures"),
            ("SI=F", "Silver Futures"),
            ("CL=F", "Crude Oil"),
            ("NG=F", "Natural Gas"),

            # Forex
            ("GBPUSD=X", "GBP/USD"),
            ("EURUSD=X", "EUR/USD"),
            ("USDJPY=X", "USD/JPY"),

            # Crypto
            ("BTC-USD", "Bitcoin"),
            ("ETH-USD", "Ethereum"),
            ("BNB-USD", "Binance Coin"),
            ("SOL-USD", "Solana"),
            ("XRP-USD", "Ripple"),
            ("ADA-USD", "Cardano")
        ]
    )

    self.connect_db.commit()

DatabaseManager()
```

The ticker value was set to following test cases - ‘google’, ‘apple', 'APPL’, ‘appl’
The result:



Justification for solutions taken:

- The different approach chosen of sending request through database ticker data instead of user typed in search field is one of the main errors fixed as the user structure may not be compatible with the structure of the API with pre-defined tested request prevents error as shown in the test of the programs this also prevents other validation techniques and preventing beginner investors to memories structures.
- The use of try and except is validation technique implemented such that if the API could not find the company name for the ticker the program should not crash and just return nothing which would be further changed in the GUI section with right response. The validation was mentioned in design section but was forgotten to be implemented causing the error in the test cases

Insertion of OHCLV and other data for user chosen stock to database table

The first test if the API gives the right OHCLV data and experimenting with date is gave the desired results. With ticker value test case – AAPL and MSFT 25th November

```
29      def insert_historical_data(self,ticker):
30          print(ticker)
31          df = yf.download(ticker, period="1y", interval="1d")
32          df.reset_index(inplace=True)
33          print(df)
34
35
36
37
38
39  q = DataRetrieval()
40  results = q.stock_search("appl")
41  for row in results:
42      print(row)
43  results = q.insert_historical_data("AAPL")
44
45
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

(.venv) veereshshringari@Mac NEA Project % "/Users/veereshshringari/Document
A Project/venv/data_retrival.py"
  df = yf.download(ticker, period="1y", interval="1d")
[*********************100%***********************]  1 of 1 completed
Price        Date       Close        High         Low        Open     Volume
Ticker                   AAPL        AAPL        AAPL        AAPL       AAPL
0      2025-01-24  221.776947  224.614121  220.413120  223.767942  54697900
1      2025-01-27  228.025089  231.104771  222.971558  223.011386  94863400
2      2025-01-28  237.187256  239.108574  229.770802  229.810630  75707600
3      2025-01-29  238.282303  238.780052  232.956385  233.065890  45486100
4      2025-01-30  236.520264  239.705853  236.141985  237.595403  55658300
..            ...         ...         ...         ...         ...       ...
246    2026-01-16  255.529999  258.899994  254.929993  257.899994  72142800
247    2026-01-20  246.699997  254.789993  243.419998  252.729996  80267500
248    2026-01-21  247.649994  251.559998  245.179993  248.699997  54641700
249    2026-01-22  248.350006  251.000000  248.149994  249.199997  39708300
```

The module was further developed to insert the downloaded data from pandas' data frame to be inserted into the database.

```
    def insert_historical_data(self, stock_id, ticker):
        df = yf.download(ticker, period="1y", interval="1d")
        df.reset_index(inplace=True)

        df = df[['Date', 'Open', 'Close', 'High', 'Low', 'Volume']]

        for _, row in df.iterrows():
            self.cursor.execute(
                """
                INSERT INTO STOCK_PLOTTING
                (DateID, stockID, Open, Close, High, Low, Volume)
                VALUES (?, ?, ?, ?, ?, ?, ?)
                """,
                (
                    row['Date'],
                    stock_id,
                    float(row['Open']),
                    float(row['Close']),
                    float(row['High']),
                    float(row['Low']),
                    float(row['Volume'])
                )
            )
        self.connect_db.commit()

q = DataRetrieval()
results = q.stock_search("appl")
for row in results:
    print(row)
q.insert_historical_data(stock_id=1, ticker="AAPL")
```

Error:

- Using the first test case of stock_id = 1 and ticker = AAPL gave an error File "/Users/veereshshringari/Documents/Mandar Project /NEA Project/venv/data_retrival.py", line 58, in <module>q.insert_historical_data(stock_id=1, ticker="AAPL")

- File "/Users/veereshshringari/Documents/Mandar Project /NEA Project/venv/data_retrival.py", line 36, in insert_historical_data

- self.cursor.execute(sqlite3.ProgrammingError: Error binding parameter 1: type 'Series' is not supported)

What error is occurring explanation:

To debug the errors first had to understand the errors by searching the error code on the internet following conclusions were made and where it might be occurring.

- Error 1: Connection error is occurring as the data retrieval file is not connecting with database and creating duplicate database files which are empty causing other parts to fail. This could be occurring due either a failure in the database or insertion file.

- Error2: Schema creation errors the insertion of the OHCLV in the database the insertion function ran but the tables were not detected causing "sqlite3.OperationalError: no such table: STOCK_PLOTTING "error this might be due to the execution structure of the database where table creation was after the insertion function error in the database file.

- Error3: Referential integrity error as other data were missing the Insertion as the foreign key is enforced so no link was found to other table as the table are empty disrupting referential integrity. Also broke NOT NULL constraints as nothing was inserted.

- Error4: Panda's runtime error crash as. strip () was occurring on tuple which is static data structure and cannot change during runtime. The column rename DataID was being renamed by index and not by name. When inserting the data from the panda's data frame to SQLite database where columns are being mismatched and ignoring the constraints

- Error5: No verification if data is inserted to successfully and if the table are successfully created and more importantly if the file is connected to the database correctly.

Second attempt worked on to fix the errors shown above: worked on from 26th to 28th November:

```
import sqlite3
import yfinance as yf

class DataRetrieval:
    def __init__(self, db_path):
        self.db_path = db_path
        self.connect_db = sqlite3.connect(db_path)
        self.cursor = self.connect_db.cursor()
        self.cursor.execute("PRAGMA foreign_keys = ON;")
```

```
def ensure_portfolio_exists(self, portfolioID, ticker):
    # Check if portfolio exists
    self.cursor.execute(
        "SELECT 1 FROM PORTFOLIO WHERE portfolioID = ?;",
        (portfolioID,)
    )
    exists = self.cursor.fetchone()

    if exists:
        return

    # Ensure at least one user exists (minimal safe default)
    self.cursor.execute("SELECT userID FROM USER LIMIT 1;")
    user = self.cursor.fetchone()

    if user is None:
        self.cursor.execute(
            "INSERT INTO USER (username, salt, hashedPassword) VALUES (?, ?, ?);",
            ("default_user", 0, "placeholder")
        )
        user_id = self.cursor.lastrowid
    else:
        user_id = user[0]

    # Create the missing portfolio
    self.cursor.execute(
        """
        INSERT INTO PORTFOLIO (userID, ticker, stockName)
        VALUES (?, ?, ?);
        """,
        (user_id, ticker, ticker)
    )

    self.connect_db.commit()
```

```
def insert_historical_data(self, portfolioID, ticker):
        # Ensure FK target exists before inserting historical data
        self.ensure_portfolio_exists(portfolioID, ticker)
        # Use yfinance Ticker object (more stable than download)
        ticker_obj = yf.Ticker(ticker)
        # Get historical price data
        hist = ticker_obj.history(period="1y", interval="1d")

        # Guard against empty results
        if hist.empty:
            raise ValueError(f"No historical data returned for ticker: {ticker}")
        insert_query = """
            INSERT OR IGNORE INTO STOCK_PLOTTING
            (DateID, portfolioID, Open, Close, High, Low, Volume)
            VALUES (?, ?, ?, ?, ?, ?, ?);
        """
        rows = []
        for date, row in hist.iterrows():
            rows.append((
                date.date(),                 # DateID
                int(portfolioID),             # portfolioID (FK + PK)
                float(row["Open"]),
                float(row["Close"]),
                float(row["High"]),
                float(row["Low"]),
                float(row["Volume"])
            ))
        try:
            self.cursor.executemany(insert_query, rows)
            self.connect_db.commit()
            print("Inserted/ignored rows attempted:", len(rows))

            self.cursor.execute(
                "SELECT COUNT(*) FROM STOCK_PLOTTING WHERE portfolioID=?;",
                (portfolioID,)
            )
            print("Rows now in DB for portfolio:", self.cursor.fetchone()[0])

        except sqlite3.IntegrityError as e:
            raise RuntimeError(f"Insert failed due to constraints: {e}")
```

```
        except sqlite3.IntegrityError as e:
            raise RuntimeError(f"Insert failed due to constraints: {e}")

    def close(self):
        self.connect_db.close()



db_path = "/Users/veereshshringari/Documents/Mandar Project /NEA Project/marketanalysis_2.db"

# Tables MUST already exist before this runs
q = DataRetrieval(db_path)
q.insert_historical_data(portfolioID=1, ticker="AAPL")
q.close()
```

Justification of errors fixed

- Error 1 and 2: In the instillation of the data retrieval class this was added “self.cursor_obj. execute (""""CREATE TABLE IF NOT EXISTS STOCK_PLOTTING(...)""")”
  Justification – to ensure that the tables are created and the database is connected to the class and if not connected the program should not crash inform.
- Error 3: the function “ensure_portfolio_exists ()” was added to make sure portfolio exists in the portfolio table and is not empty by “SELECT 1 FROM PORTFOLIO WHERE portfolioID =?”. Furthermore, checks if the user table exists and not empty “SELECT userID FROM USER LIMIT 1” for now to test the structure the function creates a missing value “INSERT INTO PORTFOLIO (userID, ticker, stockName)” it also fixes the porflio_id NOT NULL constraint PRIMARY KEY (DateID, portfolioID) by implementing “INSERT OR IGNORE INTO STOCK_PLOTTING (DateID, portfolioID, Open, Close, High, Low, Volume)”
  Justification – These changes adhere to the SQLite structure and ensure all the data added are accurate and consistent.
- Error 5: Hidden constraints fail are highlighted to pinpoint where errors are occurring with use of try and except.
  Justification – the use of try and except in the database is to improve the reliability, robustness, and maintainability of the program. Using this idea system can detect error and prevent the system crashing but instead highlights the error instead of ignoring the error message such as inserting records to strict integrity constraints and all non – null field requirements.

Third attempt: 19th January 2026

A text-based prototype was developed as a Proof of Concept (PoC) to validate the backend logic before GUI integration. This CLI tool tests the Data Retrieval class's ability to manage SQLite foreign key constraints (specifically between PORTFOLIO and STOCK_PLOTTING) and ensures that finance API data is correctly parsed and mapped to the database schema. It also serves to verify search validation and the integrity of parameterized queries against SQL injection.

```
username = input("Enter username: ")
user_id = q.create_or_get_user(username)

search_term = input("Enter ticker or company name to search: ")
results = q.stock_search(search_term)

if not results:
    print("No matching stocks found.")
    q.close()
    exit()

print("\nSearch results:")
for i, row in enumerate(results):
    print(f"{i}: Ticker={row[1]}, Company={row[2]}")

existing_tickers = q.search_portfolio_tickers(search_term, user_id=user_id)
print("\nTickers already in your portfolio:")
if existing_tickers:
    for ticker in existing_tickers:
        print(f"- {ticker}")
else:
    print("No matching tickers found in your portfolio.")

choice = int(input("Select a stock by number: "))
selected_ticker = results[choice][1]

portfolio_id = int(input("Enter portfolio ID to use: "))

q.insert_historical_data(
    portfolioID=portfolio_id,
    user_id=user_id,
    ticker=selected_ticker
)

print("Stock added and historical data stored.")
q.close()
```

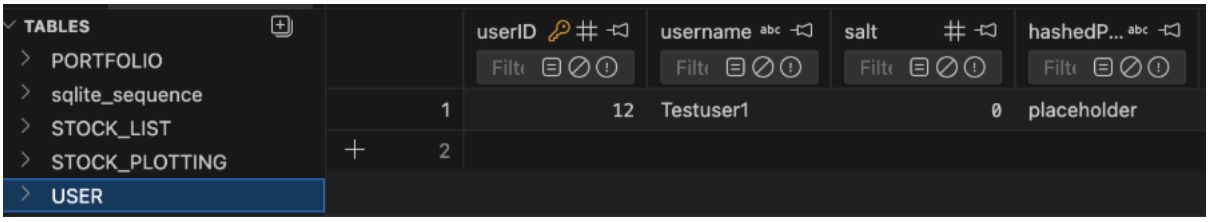
Test cases test

1st

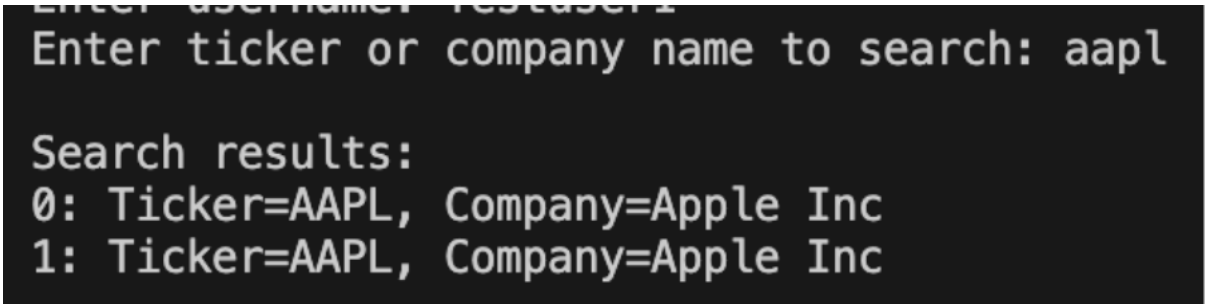
Adding inexperienced users to the database:

1. Testuser1

Results:

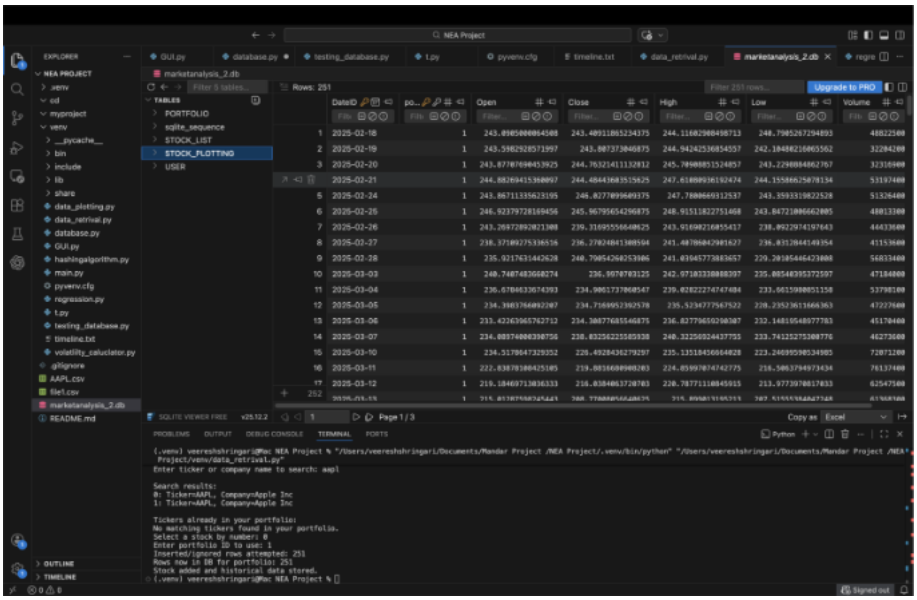


The CLI asks for which stock to add in the portfolio table:

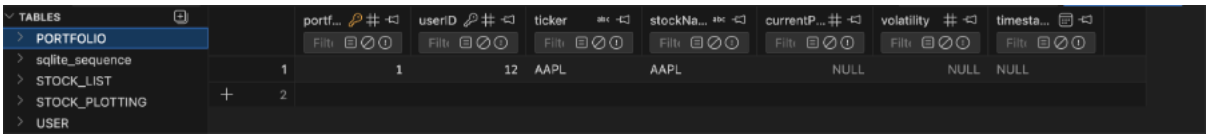


This shows it successfully returning right results which user search for then users select the stock they want add by example choosing the index they want to add ‘0’ means the apple inc is added to the portfolio.

Informs the chosen stocks are already in the portfolio section so it should not be added again.



Successfully adds data to the database with linking to the portfolio maintaining the relationship:

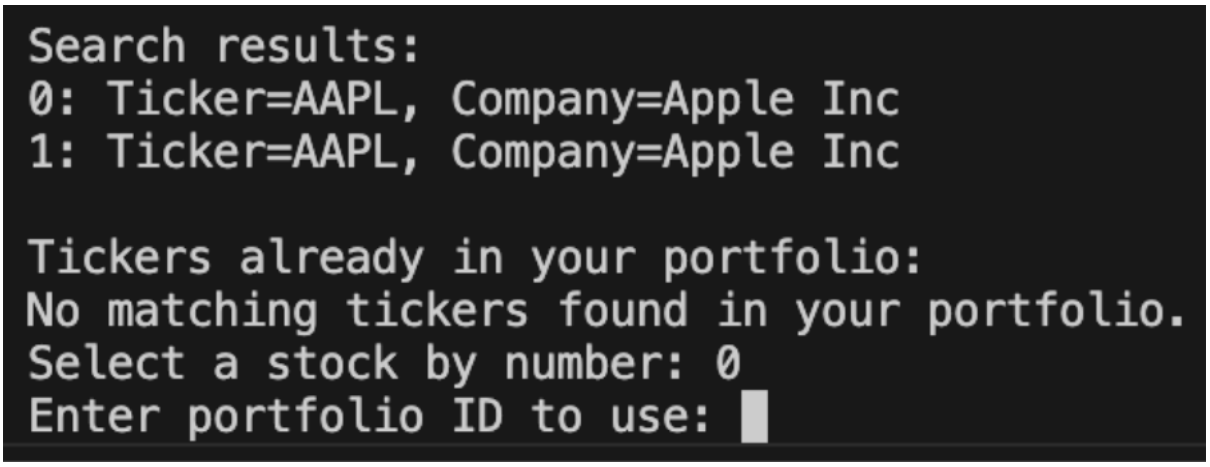


2nd

The same stock ticker is added to the for the same user to test duplication of results how it handles:

Username - Testuser1

Ticker – AAPL



Fail as the AAPL ticker is already added to portfolio and did not prevent adding the same ticker again data redundancy.

3rd

In the select a stock number chose number out range:

Test case – ‘5’

```
Search results:
0: Ticker=MSFT, Company=Microsoft Corporation
1: Ticker=MSFT, Company=Microsoft Corporation

Tickers already in your portfolio:
No matching tickers found in your portfolio.
Select a stock by number: 5
Traceback (most recent call last):
  File "/Users/veereshshringari/Documents/Mandar Project /NEA Project/venv/data_retrival.py", line 188, in <module>
    selected_ticker = results[choice][1]
                      ~~~~~~~^^^^^^^^
IndexError: list index out of range
(.venv) veereshshringari@Mac NEA Project %
```

Fail program crashes when out of range number is input.

4th

Empty field in the search ticker

```
Enter username: me
Enter ticker or company name to search:

Search results:
0: Ticker=AAPL, Company=Apple Inc
1: Ticker=MSFT, Company=Microsoft Corporation
2: Ticker=GOOGL, Company=Alphabet Inc
3: Ticker=AMZN, Company=Amazon.com Inc
4: Ticker=META, Company=Meta Platforms Inc
5: Ticker=NVDA, Company=NVIDIA Corporation
6: Ticker=TSLA, Company=Tesla Inc
7: Ticker=BRK-B, Company=Berkshire Hathaway
8: Ticker=JPM, Company=JPMorgan Chase
9: Ticker=V, Company=Visa Inc
10: Ticker=BP, Company=BP plc
11: Ticker=HSBA, Company=HSBC Holdings
12: Ticker=SHEL, Company=Shell plc
13: Ticker=ULVR, Company=Unilever
14: Ticker=AZN, Company=AstraZeneca
15: Ticker=RIO, Company=Rio Tinto
16: Ticker=BHP, Company=BHP Group
17: Ticker=NESN.SW, Company=Nestlé
18: Ticker=^GSPC, Company=S&P 500
19: Ticker=^DJI, Company=Dow Jones Industrial Average
20: Ticker=^IXIC, Company=NASDAQ Composite
21: Ticker=^FTSE, Company=FTSE 100
22: Ticker=^N225, Company=Nikkei 225
23: Ticker=GC=F, Company=Gold Futures
24: Ticker=SI=F, Company=Silver Futures
25: Ticker=CL=F, Company=Crude Oil
26: Ticker=NG=F, Company=Natural Gas
27: Ticker=GBPUSD=X, Company=GBP/USD
28: Ticker=EURUSD=X, Company=EUR/USD
29: Ticker=USDJPY=X, Company=USD/JPY
30: Ticker=BTC-USD, Company=Bitcoin
31: Ticker=ETH-USD, Company=Ethereum
32: Ticker=BNB-USD, Company=Binance Coin
33: Ticker=SOL-USD, Company=Solana
34: Ticker=XRP-USD, Company=Ripple
35: Ticker=ADA-USD, Company=Cardano
36: Ticker=AAPL, Company=Apple Inc
37: Ticker=MSFT, Company=Microsoft Corporation
38: Ticker=GOOGL, Company=Alphabet Inc
39: Ticker=AMZN, Company=Amazon.com Inc
40: Ticker=META, Company=Meta Platforms Inc
41: Ticker=NVDA, Company=NVIDIA Corporation
42: Ticker=TSLA, Company=Tesla Inc
43: Ticker=BRK-B, Company=Berkshire Hathaway
44: Ticker=JPM, Company=JPMorgan Chase
```

Test – Pass as the empty field gave options and program portfolio

Successful outcomes

User creation

Test ID	Feature	Input	Expected Result	Actual Result	Pass/Fail	Annotation
T1	Create new user	username = "Mandar123"	New user added to USER table	UserID returned, row added	Pass	Confirms INSERT query works and inexperienced users are stored correctly.
T2	Existing user login	username = "Mandar123"	Same userID returned	Same ID returned	Pass	Confirms duplicate users are not created.

Stock search creation

Test ID	Feature	Input	Expected Result	Actual Result	Pass/Fail	Annotation
T3	Valid ticker search	"AAPL"	Apple shown in results	Apple returned	Pass	Shows parameterised SQL query works correctly.
T4	Company name search	"Apple"	Apple ticker found	AAPL returned	Pass	Confirms LIKE search works.
T5	SQL injection test	"' OR 1=1 --"	No database breach	No extra results	Pass	Shows parameterised query prevents SQL injection.

Portfolio section:

Test ID	Feature	Input	Expected Result	Actual Result	Pass/Fail	Annotation
T6	New portfolio	portfolio=1	Portfolio row created	Row inserted	Pass	Confirms foreign key relationship works.

T7	Valid ticker history	ticker=AAPL	1 year data stored	Rows inserted	Pass	Shows yfinance data retrieval works.
T8	Duplicate insert	Run twice	No duplicate rows	INSERT OR IGNORE worked	Pass	Confirms primary key prevents duplicates.

unsuccessful outcomes

Test No	Feature Tested	Test Data	Expected Result	Actual Result	Pass/Fail	Justification
T9	Empty Field Handling	Portfolio ID = ""	Program rejects entry and shows error	Currently accepts empty input	Fail	Backend should validate that required fields are not empty before database insert to avoid corrupted data.
T10	Portfolio ID Type Check	Portfolio ID = "ABC123"	Program rejects non-integer input	Currently accepts string input	Fail	Portfolio ID should be integer because database relations depend on numeric ID. Prevents runtime errors in queries.
T12	Invalid Ticker Detection	Ticker = "FAKE123"	Program detects ticker not found and rejects	Empty DataFrame caught as ValueError	Partial Pass	Logic correctly detects invalid ticker using API response from Yahoo Finance, but error message should be clearer to user.
T13	Duplicate Ticker Prevention	Same ticker added	Program prevents	Duplicate rows created	Fail	Database should enforce UNIQUE

		twice for same username	duplicate entry			constraint (username + ticker). Prevents incorrect portfolio totals.
T14	Portfolio Matching Logic	Same ticker multiple times	Correct portfolio still displays ticker	Portfolio displays mismatched	Fail	Duplicate data breaks JOIN queries, so ticker does not match portfolio correctly. Logic must check existence before insert.

Updated program

Fourth attempt

```
import sqlite3
import yfinance as yf

class DataRetrieval:
    def __init__(self, db_path):
        self.conn = sqlite3.connect(db_path)
        self.cursor = self.conn.cursor()
        self.cursor.execute("PRAGMA foreign_keys = ON;")

    def stock_search(self, ticker):
        if not ticker.strip():
            return []

        query = """
        SELECT * FROM STOCK_LIST
        WHERE tickername LIKE ? OR companyname LIKE ?;
        """
        like_value = f"%{ticker.upper()}%"
        self.cursor.execute(query, (like_value, like_value))
        return self.cursor.fetchall()


    def search_portfolio_tickers(self, ticker_query="", user_id=None):
        like_value = f"%{ticker_query.upper()}%"

        query = """
        SELECT DISTINCT ticker
        FROM PORTFOLIO
        WHERE userID = ?
        AND UPPER(ticker) LIKE ?
        ORDER BY ticker;
        """
        self.cursor.execute(query, (user_id, like_value))
        return [row[0] for row in self.cursor.fetchall()]
```

```
def create_or_get_user(self, username):
    if not username.strip():
        raise ValueError("Username cannot be empty.")

    self.cursor.execute(
        "SELECT userID FROM USER WHERE username=?;",
        (username,)
    )
    user = self.cursor.fetchone()

    if user:
        return user[0]

    self.cursor.execute(
        "INSERT INTO USER(username, salt, hashedPassword) VALUES (?, ?, ?);",
        (username, 0, "placeholder")
    )
    self.conn.commit()
    return self.cursor.lastrowid

def ticker_exists_for_user(self, user_id, ticker):
    self.cursor.execute(
        "SELECT 1 FROM PORTFOLIO WHERE userID=? AND ticker=?;",
        (user_id, ticker)
    )
    return self.cursor.fetchone() is not None
```

```
def insert_historical_data(self, portfolioID, user_id, ticker):

    if self.ticker_exists_for_user(user_id, ticker):
        print("Ticker already exists in your portfolio.")
        return

    ticker_obj = yf.Ticker(ticker)
    hist = ticker_obj.history(period="1y")

    if hist.empty:
        print("Invalid or delisted ticker.")
        return

    # create portfolio row
    self.cursor.execute("""
        INSERT INTO PORTFOLIO (portfolioID, userID, ticker, sto
        VALUES (?, ?, ?, ?);
    """, (portfolioID, user_id, ticker, ticker))
    self.conn.commit()

    insert_query = """
    INSERT OR IGNORE INTO STOCK_PLOTTING
    (DateID, portfolioID, Open, Close, High, Low, Volume)
    VALUES (?, ?, ?, ?, ?, ?, ?);
    """

    rows = []
    for date, row in hist.iterrows():
        rows.append((
            date.date(),
            portfolioID,
            float(row["Open"]),
            float(row["Close"]),
            float(row["High"]),
            float(row["Low"]),
            float(row["Volume"])
        ))

    self.cursor.executemany(insert_query, rows)
    self.conn.commit()
    print("Historical data inserted.")

def close(self):
    self.conn.close()
```

```
try:
    # USERNAME
    username = input("Enter username: ").strip()
    if not username:
        raise ValueError("Username cannot be empty.")

    user_id = q.create_or_get_user(username)

    # SEARCH
    search_term = input("Enter ticker/company name: ").strip()
    if not search_term:
        raise ValueError("Search term cannot be empty.")

    results = q.stock_search(search_term)

    if not results:
        print("No matching stocks found.")
        exit()

    print("\nSearch Results:")
    for i, row in enumerate(results):
        print(f"{i}: {row[1]} - {row[2]}")

    # SAFE INDEX INPUT
    try:
        choice = int(input("Select stock number: "))
        if choice < 0 or choice >= len(results):
            raise IndexError
    except:
        print("Invalid selection.")
        exit()

    selected_ticker = results[choice][1]

    # SAFE PORTFOLIO ID
    try:
        portfolio_id = int(input("Enter portfolio ID: "))
    except:
        print("Portfolio ID must be a number.")
        exit()

    q.insert_historical_data(portfolio_id, user_id, selected_ticker)

finally:
    q.close()
```

Changes made and justifications

- T9: To prevent empty input in the input sections the following was added "if not username.strip(): raise ValueError("Username cannot be empty.")"

  Justification – prevents invalid database entries and increase the usability of the program

- T10: Users may add the same ticker multiple times in the portfolio database to prevent this a new function is added def ticker_exists_for_user(self, user_id, ticker):

  Justification - Which maintains the database integrity and prevents duplication of portfolio rows while also making sure the search function inside portfolio worked properly

- T11: to prevent ticker field being logically incorrect if the yahoo finance returns empty list using if hist.empty:

  Justification – prevents meaningless portfolio entries and incomplete historical data which could cause the program to crash in the plotting stage

- T12: using try and except:portfolio_id = int (input ("Enter portfolio ID: ")) is using this makes sure the user has input a valid integer and anything else causes them to try again

  justification - Ensures portfolioID matches database schema (INTEGER) while also preventing SQL type mismatch errors

- T13: Empty search would previously return everything or cause unwanted query behaviour. if not ticker. Strip (): return [] is added to prevent SQL search with blank search and returns the empty list safely

  Justification - Improves performance (no unnecessary query) as it

  avoids broad unintended database fetch this Ensures user intentionally searches

Review of the program

The program is designed to meet the success criteria SC2, and the program has partly met the goal of fetching the ticker and the respective historical data however as it uses a proof of concept using CLI and does not utilise GUI, yet the success criteria is still partially achieved.

Furthermore, the program is also designed to meet the success criteria of SC9 where the database should successfully store the stock ticker data and the ticker itself in the portfolio which it does achieve the function however as it is not completely linked with user table in the database the following success criteria is also partially met.

The program achieves SC8 as it has implemented with idea of validation to occur at every user interaction stage however most testing would be required to support the claim which would occur in post development testing.

Stock plotting

1st attempt 20th January 2026

Focused on to get the structure of the graph as first prototype. The prototype also serves the purpose of seeing how to get current market price using yfinance API and tested with the ticker 'APPL' and compared with real time market.

```
import time
import pandas as pd
import yfinance as yf
import plotly.graph_objects as go
import sqlite3
#basic idea of sketching would work
class Stockdata ():
    def __init__(self):
        con = sqlite3.connect('marketanalysis_2.db')
        self.df = pd.read_sql_query("SELECT DateID,Open,High,Low,Close FROM STOCK_PLOTTING",con)
        self.fig = go.Figure(data=[go.Candlestick(x=self.df['DateID'],
                    open=self.df['Open'],
                    high=self.df['High'],
                    low=self.df['Low'],
                    close=self.df['Close'])])
        self.fig.show()
    def updated_plotting(self):
        while True:
            time.sleep(1)
            target = yf.Ticker("AAPL")
            self.current_price = target.fast_info['last_price']
Stockdata()
```

Result:

The current price fetching worked as the price was matching with compared market price seen google finance however the sleep time was constantly optimised for the most accurate results.

Changes to be made

- Make sure the graph can be implement it in tkinter page.
- Allow time date range changing function for the graph and test it as PoC using CLI.
- To meet the SC8 a system to handle if the graph has missing data which cannot be plotted.
- Use the current price data fetched as shown above to be implemented in the graph logic such that stakeholder has the most updated data.

2nd attempt 20th – 23rd January 2026

```
import sqlite3
from datetime import datetime, timedelta

import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd
import yfinance as yf
from matplotlib.patches import Rectangle

class StockDataPlotter:
    def __init__(self, db_path="marketanalysis_2.db"):
        self.db_path = db_path
        self.df = pd.DataFrame()
        self.last_status = "Ready"

    def load_from_sqlite(self):
        try:
            con = sqlite3.connect(self.db_path)
            self.df = pd.read_sql_query(
                """
                SELECT DateID, Open, High, Low, Close
                FROM STOCK_PLOTTING
                ORDER BY DateID
                """,
                con,
            )
            con.close()
        except Exception as exc:
            self.df = pd.DataFrame()
            self.last_status = f"SQLite load failed: {exc}"
            return

        if self.df.empty:
            self.last_status = "No rows found in STOCK_PLOTTING."
            return

        self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
        self.df = self.df.dropna(subset=["DateID", "High"]).sort_values("DateID")
        self.last_status = f"Loaded {len(self.df)} rows from SQLite."
```

```
def fetch_from_yfinance(self, ticker):
    try:
        downloaded = yf.Ticker(ticker).history(period="10y", interval="1d")
        if downloaded.empty:
            self.last_status = f"No Yahoo data for ticker: {ticker}"
            return

        downloaded = downloaded.reset_index()
        self.df = downloaded[["Date", "Open", "High", "Low", "Close"]].rename(columns={"Date": "DateID"
        self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
        self.df = self.df.dropna(subset=["DateID", "High"]).sort_values("DateID")
        self.last_status = f"Loaded {len(self.df)} rows from Yahoo for {ticker}."
    except Exception as exc:
        self.last_status = f"Yahoo load failed: {exc}"

def get_filtered_data(self, range_label="ALL"):
    if self.df.empty:
        return self.df

    end_date = self.df["DateID"].max()
    label = range_label.upper()

    if label == "1M":
        start_date = end_date - timedelta(days=30)
    elif label == "6M":
        start_date = end_date - timedelta(days=182)
    elif label == "YTD":
        start_date = datetime(end_date.year, 1, 1)
    elif label == "1Y":
        start_date = end_date - timedelta(days=365)
    else:
        return self.df

    return self.df[self.df["DateID"] >= start_date]
```

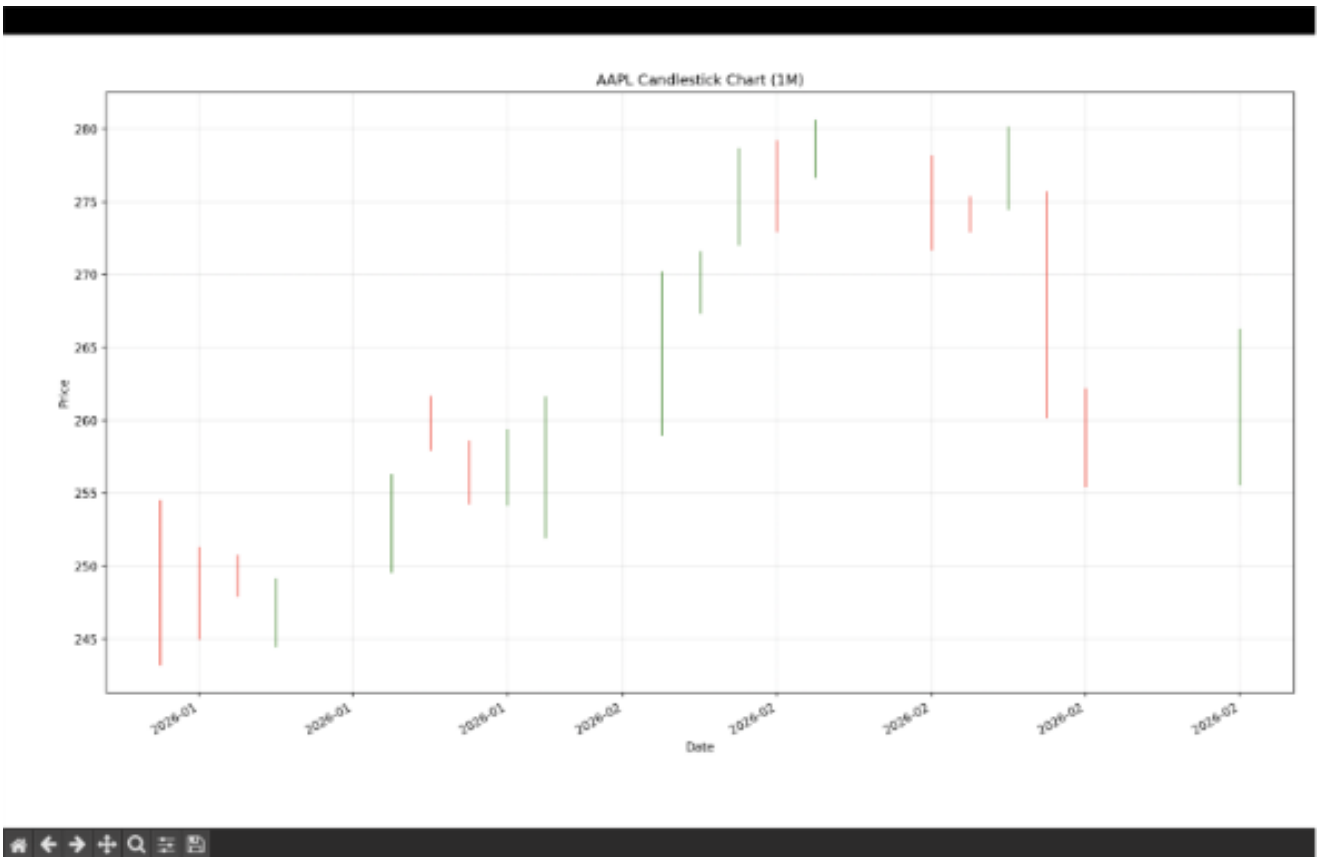
```
def build_figure(self, range_label="ALL", ticker_label=None):
    fig, ax = plt.subplots(figsize=(9, 5), dpi=100)

    if self.df.empty:
        ax.set_title("No data available")
        return fig

    plot_df = self.get_filtered_data(range_label)
    if plot_df.empty:
        ax.set_title("No data available for selected range")
        return fig

    date_nums = mdates.date2num(plot_df["DateID"].dt.to_pydatetime())
    if len(date_nums) > 1:
        candle_width = min(0.8, (date_nums[1] - date_nums[0]) * 0.7)
    else:
        candle_width = 0.6

    for x, open_p, high_p, low_p, close_p in zip(
        date_nums,
        plot_df["Open"],
        plot_df["High"],
        plot_df["Low"],
        plot_df["Close"],
    ):
        up_candle = close_p >= open_p
        color = "green" if up_candle else "red"

        ax.vlines(x, low_p, high_p, color=color, linewidth=1)
        body_bottom = min(open_p, close_p)
        body_height = abs(close_p - open_p)
        if body_height == 0:
            body_height = 0.001

        rect = Rectangle(
            (x - candle_width / 2, body_bottom),
            candle_width,
            body_height,
            facecolor=color,
            edgecolor=color,
            alpha=0.75,
        )
        ax.add_patch(rect)
```

```
        title_ticker = f"{ticker_label} " if ticker_label else ""
        ax.set_title(f"{title_ticker}Candlestick Chart ({range_label.upper()})")
        ax.set_xlabel("Date")
        ax.set_ylabel("Price")
        ax.xaxis_date()
        ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
        fig.autofmt_xdate()
        ax.grid(True, alpha=0.3)
        fig.tight_layout()
        return fig

    def show_plot(self, range_label="ALL", ticker="AAPL"):
        self.load_from_sqlite()
        if self.df.empty:
            self.fetch_from_yfinance(ticker)

        fig = self.build_figure(range_label=range_label, ticker_label=ticker)
        fig.show()
        plt.show()


def prompt_range_label():
    valid_ranges = {"1M", "6M", "YTD", "1Y", "ALL"}
    selected = input("Choose date range (1M, 6M, YTD, 1Y, ALL): ").strip().upper()
    if selected in valid_ranges:
        return selected
    print("Invalid range, defaulting to ALL.")
    return "ALL"


if __name__ == "__main__":
    plotter = StockDataPlotter()
    chosen_range = prompt_range_label()
    plotter.show_plot(range_label=chosen_range, ticker="AAPL")
```

T

Results and testing:

The following test case uses the date range of ‘1M’ in the CLI and gives the graph for the ticker ‘apple’



Changes and Justifications

Manually doing candlestick instead of using external module

- Change: The previous iteration for the graph was developed again as proof of concept and to explore how it should be visually represented and benchmark to develop this algorithm. This iteration uses matplotlib and implements the candlestick diagram in the function build figure ()
- Justification: Line charts hide the volatility of a stock (the gap between high and low). Candlestick charts are the industry standard because they show four data points ($Open, High, Low, Close$) in a single visual element

Adaptive "Candle" Scaling

- Change: Implemented a calculation for candle width based on the time-difference between date entries.

- Justification: In a 1M view, data points are spread out; in an ALL view, they are packed tight. This math prevents the "candles" from overlapping and becoming a messy blob when looking at 10 years of data.

Vectorized Date Filtering with Pandas

- Change: Used time delta and Boolean indexing (e.g., self.df[self.df["DateID"] >= start_date]) instead of Python loops.
- Justification: Filtering thousands of rows of data using standard loops is slow. Vectorization in Pandas allows the CPU to process the entire column at once, making the switch between "1M" and "1Y" views near-instant for the user.

Fallback Data Acquisition

- Change: Integrated fetch_from_yfinance as a secondary data source if load_from_sqlite returns an empty Data Frame.
- Justification: This ensures the plotting tool is resilient. It allows a user to visualize a stock they have not "saved" yet, providing a seamless transition between exploring new stocks and viewing saved portfolio data.

Matplotlib Date Formatting

- Change: Added mdates.DateFormatter("%Y-%m") and fig.autofmt_xdate().
- Justification: Raw date strings often overlap on the X-axis, making them unreadable. These functions rotate the labels and format them cleanly, ensuring the chart is presentation ready.

Test log sheet:

Test ID	**Feature Under Test**	**Test Input / Action**	**Expected Outcome**	**Actual Outcome**	**Status**
T15	Data Source Priority	Call show_plot() with data already in STOCK_PLOTTING.	Data is pulled from SQLite; no yfinance call is made.	SQLite rows loaded successfully.	**Pass**
T16	API Fallback	Call show_plot() with an empty STOCK_PLOTTING table.	Logic triggers fetch_from_yfinance () to retrieve "AAPL" data.	Yahoo data fetched and plotted automatically.	**Pass**

T17	Date Range: 1M	Select 1M range label.	Plot shows only data from the last 30 days of the dataset.	X-axis correctly reflects the 30-day window.	**Pass**
T18	Date Range: YTD	Select YTD range label.	Plot starts from January 1st of the current year.	Filtered correctly to current year start.	**Pass**
T19	Candle Logic (Green)	Price: Close > Open	Rectangle is colored Green.	Green candles rendered for price gains.	**Pass**
T20	Candle Logic (Red)	Price: Close < Open	Rectangle is colored Red.	Red candles rendered for price drops.	**Pass**
T21	Invalid Range Entry	Input 5Years in CLI.	Script defaults to "ALL" and plots the full dataset.	Defaulted to ALL without crashing.	**Pass**
T22	Empty Data Handling	Search for a fake ticker (e.g. NONEXIST)	build_figure displays "No data available" title.	Figure displayed with error message title.	**Pass**
T23	Dynamicall y update the graph with current market data	Check the graph if it is updated with current market data compared to other sources	A graph continuously updating according to time range with current market data	No solution worked after multiple tries	**Fail**

Review of the program

The program is designed to mee the success criteria of SC3 where the program achieves to visualise the historical past data of the stock and however the condition of plotting the trend for the next 30 days is still not met and would be implemented when the linear regression model is implemented and linked with the plotting module.

Interview with stakeholder

An interview was conducted afterwards when the future trend prediction using closing price was developed like the test table highlighted in regression model the database connection was one of the issues highlighted which also applies to the plotting file moreover the holiday/ gaps is not major issue in the plotting part as the dates are based of real data so if there was holiday there would be no data present.

Future trend of closing price

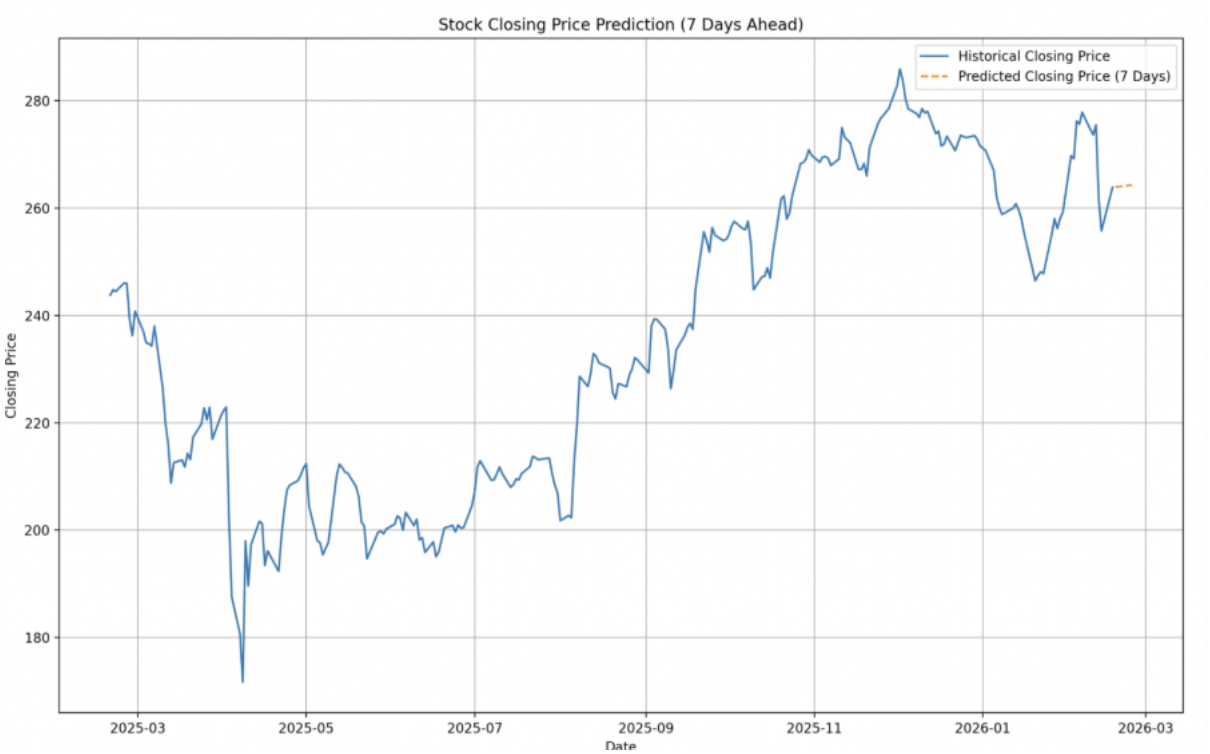
The program is developed to find the predicted closing price of the stock for the next 7 days and plot it to test if it is useful to users and does it make sense to stakeholder and help them.

1st attempt 26th –27th January 2026

```
import sqlite3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

class Regression:
    def __init__ (self):
        self.conn = sqlite3.connect("marketanalysis_2.db")
        # links to the data in the database
        self.portfolio_id = 1
        self.query = """
        SELECT DateID, Close
        FROM STOCK_PLOTTING
        WHERE portfolioID = ?
        ORDER BY DateID ASC
        """
    def preprocessing (self):
        self.df = pd.read_sql_query(self.query, self.conn, params=(self.portfolio_id,))
        self.conn.close()
        if self.df.empty:
            raise ValueError("No stock data found for the selected portfolio.")
        self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
        self.df = self.df.dropna(subset=["DateID"]).reset_index(drop=True)
        self.df['Prev_Close'] = self.df['Close'].shift(1)
        self.df = self.df.dropna().reset_index(drop=True)
        if len(self.df) < 2:
            raise ValueError("Not enough rows to train the regression model.")
```

```
def model_training(self):
    X = self.df[['Prev_Close']].values
    y = self.df['Close'].values
    split_index = int(len(X) * 0.8)
    if split_index == 0 or split_index >= len(X):
        raise ValueError("Not enough data to create train/test split.")
    X_train, X_test = X[:split_index], X[split_index:]
    y_train, y_test = y[:split_index], y[split_index:]

    self.model = LinearRegression()
    self.model.fit(X_train, y_train)
    y_pred = self.model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print("Model Evaluation")
    print(f"Mean Squared Error: {mse:.4f}")
    print(f"R^2 Score: {r2:.4f}")

def closingprice_predict(self):
    self.future_days = 7
    self.future_predictions = []
    last_close = self.df['Close'].iloc[-1]
    for _ in range(self.future_days):
        next_price = self.model.predict(np.array([[last_close]]))[0]
        self.future_predictions.append(next_price)
        last_close = next_price  # recursive prediction
```

```
    def plotting_prediction(self):
        plt.figure()

        # plot historical data using DateID
        plt.plot(self.df['DateID'], self.df['Close'], label="Historical Closing Price")

        # Generate future dates from the final observed date.
        last_date = self.df['DateID'].iloc[-1]
        future_dates = pd.date_range(
            start=last_date + pd.Timedelta(days=1),
            periods=self.future_days,
            freq="D"
        )

        plt.plot(
            future_dates,
            self.future_predictions,
            linestyle='dashed',
            label="Predicted Closing Price (7 Days)"
        )

        plt.xlabel("Date")
        plt.ylabel("Closing Price")
        plt.title("Stock Closing Price Prediction (7 Days Ahead)")
        plt.legend()
        plt.grid(True)
        plt.show()

        print("\nNext 7 Days Predicted Closing Prices:")
        for i, price in enumerate(self.future_predictions, start=1):
            print(f"Day {i}: {price:.2f}")


if __name__ == "__main__":
    model = Regression()
    model.preprocessing()
    model.model_training()
    model.closingprice_predict()
    model.plotting_prediction()
```



To test the program, it links to the database where historical data for apple is present where the data is used to train the model for the given stock tested with matrices of R^2 score, MAE and MSE are outputted to test the accuracy of the model results.

```
Model Evaluation
Mean Squared Error: 17.3862
R^2 Score: 0.7232
```

Interview with stakeholder

An interview was conducted with stakeholder Priya S who is casual investor and experience with working with historical data graph while investing. The stakeholders highlighted key problems in the current model both for the plotting of the graph overall and the predictive aspect.

- Gaps in the weekends and holidays causing gaps in the actual daily date may cause confusion in investors seen in testing in the code
- The predictive trend is too small for investors to make insights from it is not enough to see the overall trend of the market which would further require a larger range for the trend
- The stakeholder also explained the R^2, MAE and MSE should be outputted but also explain what means to have low R^2 score or other metrics and what the stakeholders should be looking out for accurate and a trustworthy model the feature is incorporated in the GUI (which is being developed after the backend is completed) but not in the backend
- The stakeholder also mentioned about the trend indicator when we had our last interview when developing the design section which will be reflected in the code.

2nd attempt 27thJanuary2026

The first iteration simply included changing

From:

```
future_dates = pd.date_range(
    start=last_date + pd.Timedelta(days=1),
    periods=self.future_days,
    freq="D"
)
```

To:

```
future_dates = pd.date_range(
    start=last_date + pd.Timedelta(days=1),
    periods=self.future_days,
    freq="B"
)
```

Changes and Justifications

The change made from freq = 'B' generates business days only and automatically excluding the weekends
However, it still doesn't solve the problem as it only accounted for weekends not public holidays where the stock exchange might be closed
third iteration:

```
from pandas.tseries.holiday import USFederalHolidayCalendar
from pandas.tseries.offsets import CustomBusinessDay

self.trading_day_offset = CustomBusinessDay(calendar=USFederalHolidayCalendar())


future_dates = pd.date_range(
    start=last_date + self.trading_day_offset,
    periods=self.future_days,
    freq=self.trading_day_offset
)
```

Justification – Originally the program generated prediction dates using calendar days, which included weekends when stock markets are closed. This caused an unrealistic predicted prices to appear for different dates which don't match the current calendar of the investors.

The first iteration used a simple solution of changing business days (freq="B") which removed weekends. However, this still produced predictions on stock market holidays such Christmas, New year, and Good Friday, when markets like the New York stock exchange (NYSE) is closed.

To solve this issue, the system now uses CustomBusinessDay with the US federal holiday calendar. This allows the program to automatically exclude both weekends and recognized market holidays when generating future prediction dates.

The program uses us federal calendar as most of the stakeholders of the program trade American company stocks and the country's trading market in future iteration would accommodate it.

Furthermore, the range of data was also increased form 7 days to 30 days as per the investors request and it allows the stakeholders a larger range to see the overall trend.

Test ID	Feature Under Test	Test Input / Action	Expected Outcome	Actual Outcome	Status
T24	Data Lagging	Calculate Prev Close column.	Prev Close is shifted by 1; first row is NaN then dropped.	Successfully created feature-target relationship.	**Pass**
T25	Insufficient Data	Input a stock with only 1 day of history.	System raises Value Error ("Not enough rows").	Program exited safely with informative error message.	**Pass**
T26	Model Training	Run model. Fit() on 80% of data.	Model coefficients are calculated; metrics (MSE/R2) are printed.	Model converged; evaluation metrics displayed in console.	**Pass**
T27	Recursive Prediction	Run closingprice_predict() for 30 days.	Day 1 prediction is used as input for Day 2 prediction.	30 unique future price points generated sequentially.	**Pass**
T28	Holiday Filtering	Generate future dates.	Weekends and US Federal	Forecast dates correctly skip	**Pass**

			holidays (e.g., Thanksgiving) are skipped.	non-trading days.	
T29	Visualization	Call plotting prediction ().	Historical data (solid line) and Forecast (dashed line) appear together.	Chart rendered clearly with rotated date labels.	**Pass**
T30	Modularity	Runing the program in different environment	The program should not crash for any dependencies	When program run on stakeholder's computer the database file was connected properly	**Fail**

2nd iteration

To prevent the modularity, issue the following function is added: #fixing testcase T30

```
def _resolve_db_path(self, db_path):
        if os.path.isabs(db_path):
            return db_path

        local = os.path.join(os.getcwd(), db_path)
        if os.path.exists(local):
            return local

        fallback = os.path.abspath(os.path.join(os.path.dirname(__file__), "..",
db_path))
        return fallback
```

Justification:

This function determines the correct database file path by first verifying whether an absolute path is provided. If not, it attempts to locate the database in the current working directory and finally falls back to a path relative to the script's location. This improves the robustness of the program by ensuring the database can be found even if the program is executed from different directories.

Review of the code

The program is designed to meet the success criteria of SC4 successfully generating a predictive closing price for the next 7 days to be plotted however through the stakeholders request the programs predictive range was increased from 7 days to 30 days to gain more insightful trend. During a test conducted with the ticker 'AAPL' historical data the model came up with R^2 score above 0.6 which meets the criteria in success outcome.

Volatility calculator closing price — 28th – 29th January

The program's function is to calculate the volatility of stocks using rolling historical volatility where standard deviation of logarithmic returns is calculated over a fixed window of trading days which after is annualised using root of 252 which is the number of trading days to convert daily volatility to yearly volatility.

```python
import sqlite3
import math
import statistics

class Volatility:

    def __init__(self, db_path):
        self.db_path = db_path
        self.prices = self.load_prices()
        self.returns = self.calculate_log_returns()

    def load_prices(self):
        """Load Apple (AAPL) closing prices from the database"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute("""
            SELECT s.Close
            FROM STOCK_PLOTTING s
            JOIN PORTFOLIO p ON p.portfolioID = s.portfolioID
            WHERE p.ticker = 'AAPL'
            ORDER BY s.DateID ASC
        """)

        prices = [row[0] for row in cursor.fetchall()]
        conn.close()

        return prices

    def calculate_log_returns(self):
        """Calculate log returns"""
        returns = []

        for i in range(1, len(self.prices)):
            r = math.log(self.prices[i] / self.prices[i-1])
            returns.append(r)

        return returns

    def calculate_volatility(self):
        """Calculate annualised volatility"""
        std_dev = statistics.stdev(self.returns)
        volatility = std_dev * math.sqrt(252)
        return volatility
```

The program is tested using 'AAPL' following output:

```
Number of prices: 252
Annualised volatility: 0.3157325728707124
```

After testing with other cases following results were found:

Test ID	Feature Under Test	Test Input / Action	Expected Outcome	Actual Outcome	Status
T31	Database connection	Ran the program using the database file marketanalysis_2.db	The program should successfully connect to the SQLite database	The program connected without errors	Pass
T32	Loading closing prices	Used ticker "AAPL" to retrieve stock data	The program should load the Apple closing prices from the database	252 closing prices were loaded and displayed	Pass
T33	Log return calculation	Program calculated log returns from the loaded price list	Log returns should be calculated for each pair of prices	Log returns were generated without any errors	Pass
T34	Rolling volatility calculation	Executed rolling_volatility(window=30)	The program should calculate volatility values using a 30-day window	A list of annualized volatility values was returned	Pass
T35	Latest volatility value	Executed latest_volatility(window=30)	The program should output the most recent volatility value	A volatility value of approximately 0.27 was returned	Pass

T36	Handling insufficient data	Tested the program with fewer than 30 price records	The program should not crash and should return an empty result	The function returned an empty list as expected	Pass
T37	Invalid ticker handling	Changed the ticker to one does not present in the database	The program should indicate that no data was found	A Value Error was raised indicating no prices were found	Pass
T38	Output display	Ran the full program to check printed output	The number of prices and calculated volatility should be displayed	The console printed the number of prices and volatility value	Pass

Linking volatility calculator to data retrieval 30th January 2026

A much more modular version of the volatility calculator was developed which allowed it to be integrated the data retrieval/ insertion program page so it can be displayed in GUI in an easier way.

The result:

```
import math
import statistics
import sqlite3

class Volatlity:
    def __init__(self, db_path, ticker):
        """
        db_path: path to SQLite database
        ticker: stock symbol to retrieve prices for
        """
        self.db_path = db_path
        self.ticker = ticker
        self.closing_prices = self._load_closing_prices_from_db()
        self.log_returns = self._calculate_log_returns()

    def _load_closing_prices_from_db(self):
```

```
        """
        Loads closing prices from database ordered by date ascending.
        Tries 'stock prices' table first, then falls back to 'STOCK_PLOTTING' if
necessary.
        Returns a list of closing prices.
        """
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute(
            "SELECT 1 FROM SQLite master WHERE type='table' AND name='stock prices';"
        )
        has_stock_prices = cursor.fetchone() is not None

        if has_stock_prices:
            cursor.execute(
                """
                SELECT close_price
                FROM stock_prices
                WHERE ticker = ?
                ORDER BY date ASC
                """,
                (self.ticker,),
            )
            prices = [row[0] for row in cursor.fetchall()]
        else:
            cursor.execute(
                """
                SELECT s.Close
                FROM STOCK_PLOTTING s
                JOIN PORTFOLIO p ON p.portfolioID = s.portfolioID
                WHERE p.ticker = ?
                ORDER BY s.DateID ASC
                """,
                (self.ticker,),
            )
            prices = [row[0] for row in cursor.fetchall()]

        conn.close()
        if not prices:
            raise ValueError(
                f"No closing prices found for ticker '{self.ticker}' in database
'{self.db_path}'."
            )
        return prices

    def _calculate_log_returns(self):
        """
```

```
        Compute logarithmic returns from closing prices."""
        returns = []
        for i in range(1, len(self.closing_prices)):
            if self.closing_prices[i - 1] <= 0 or self.closing_prices[i] <= 0:
                continue
            r = math.log(self.closing_prices[i] / self.closing_prices[i - 1])
            returns.append(r)
        return returns


    def _log_returns_from_prices(prices):
        returns = []
        for i in range(1, len(prices)):
            prev_price = prices[i - 1]
            curr_price = prices[i]
            if prev_price <= 0 or curr_price <= 0:
                continue
            returns.append(math.log(curr_price / prev_price))
        return returns


    def rolling_from_prices(prices, window=30):
        log_returns = Volatlity._log_returns_from_prices(prices)
        if len(log_returns) < window:
            return []

        volatilities = []
        for i in range(window, len(log_returns) + 1):
            window_slice = log_returns[i - window:i]
            if len(window_slice) < 2:
                continue
            std_dev = statistics.stdev(window_slice)
            annualised_vol = std_dev * math.sqrt(252)
            volatilities.append(annualised_vol)
        return volatilities


    def latest_from_prices(prices, window=30):
        vols = Volatlity.rolling_from_prices(prices, window=window)
        return vols[-1] if vols else None

    def rolling_volatility(self, window=30):
        """
        Calculate rolling historical volatility using a moving window.
        Standard deviation of log returns is calculated inside the window.
        Then annualised.
        """
        volatilities = []
```

```
        if len(self.log_returns) < window:
            return []

        for i in range(window, len(self.log_returns) + 1):
            window_slice = self.log_returns[i - window:i]

            if len(window_slice) < 2:
                continue

            std_dev = statistics.stdev(window_slice)

            # Annualise volatility
            annualised_vol = std_dev * math.sqrt(252)
            volatilities.append(annualised_vol)

        return volatilities

    def latest_volatility(self, window=30):
        """
        Convenience method to return the most recent rolling volatility value.
        """
        vols = self.rolling_volatility(window)
        return vols[-1] if vols else None
```

Linking the volatility to the at the data retrieval page and coming up with the ranking.

```
#  The function calculate_latest_volatility and update_portfolio_metrics are added to
calculate the volatility and update the portfolio metrics in the database.

def calculate_latest_volatility(self, portfolioID, window=30):
        prices = self._get_closing_prices(portfolioID)
        return Volatlity.latest_from_prices(prices, window=window)
```

```
The get_user_portfolio function is added to retrieve the user's portfolio information
from the database.

def _quicksort_by_volatility(self, items):
        if len(items) <= 1:
            return items

        pivot = items[len(items) // 2]
        pivot_vol = pivot["volatility"] if pivot["volatility"] is not None else -1.0

        higher = []
```

```
        equal = []
        lower = []

        for item in items:
            item_vol = item["volatility"] if item["volatility"] is not None else -1.0
            if item_vol > pivot_vol:
                higher.append(item)
            elif item_vol < pivot_vol:
                lower.append(item)
            else:
                equal.append(item)

        return self._quicksort_by_volatility(higher) + equal +
self._quicksort_by_volatility(lower)
```

Hashing algorithm and validation 2nd and 3rd February

First the hashing algorithm was developed where both the hashing of the password initially by the program and verifying the password is being developed

```
import hashlib
import secrets


class PasswordHasher:
    """
    Handles password hashing, salting, and verification using SHA-256.
    """

    def __init__(self, rounds: int = 6):
        self.rounds = rounds

    def generate_salt(self) -> str:
        return secrets.token_hex(2)

    def sha256_hash(self, data: bytes) -> bytes:
        """
        Applies SHA-256 hashing to input bytes.
        """
        return hashlib.sha256(data).digest()

    def mix_state(self, state: int, value: int) -> int:
        """
        Performs bitwise mixing to enhance the avalanche effect.
        """
        # XOR manipulation
```

```
        state ^= value

        # Bit rotation (left rotate by 7 bits)
        state = ((state << 7) | (state >> (64 - 7))) & ((1 << 64) - 1)

        # Multiplication introduces diffusion
        state = (state * 0x9E3779B97F4A7C15) & ((1 << 64) - 1)

        return state

    def hash_password(self, password: str, salt: str) -> str:
        """
        Hashes a password using SHA-256 combined with salt and multiple rounds
        of bitwise manipulation.
        """
        combined = (password + salt).encode("utf-8")
        hash_bytes = self.sha256_hash(combined)

        # Convert first 8 bytes to 64-bit state
        state = int.from_bytes(hash_bytes[:8], byteorder="big")

        for _ in range(self.rounds):
            for byte in hash_bytes:
                state = self.mix_state(state, byte)

            hash_bytes = self.sha256_hash(state.to_bytes(8, byteorder="big"))

        return f"{state:016x}"

    def create_password_record(self, password: str) -> dict:
        """
        Creates a secure password record to be stored in the database.
        """
        salt = self.generate_salt()
        hashed_password = self.hash_password(password, salt)

        return {
            "salt": salt,
            "rounds": self.rounds,
            "hash": hashed_password
        }

    def verify_password(self, input_password: str, stored_salt: str, stored_hash: str)
-> bool:
        """
        Verifies an entered password against the stored hash.
        """
        return self.hash_password(input_password, stored_salt) == stored_hash
```

To test the hashing algorithm a PoC was developed to test the overall algorithm itself and how the overall validation would be required when developing it for GUI version as an inspiration.

```
class AuthBackend:
    """
    Text-based authentication tester with clear user feedback.
    """

    def __init__(self):
        self.conn = sqlite3.connect(DB_PATH)
        self.cursor = self.conn.cursor()
        self.hasher = PasswordHasher()


    def signup(self):
        username = input("Username: ")
        password = input("Password: ")
        confirm = input("Confirm Password: ")

        if not username or not password or not confirm:
            print("ERROR: Please fill in all fields.")
            return

        if not re.match(r"^\w+$", username):
            print("ERROR: Username can only contain letters, numbers, and
underscores.")
            return

        if len(password) < 8:
            print("ERROR: Password must be at least 8 characters long.")
            return

        if not any(c.isalpha() for c in password) or not any(c.isdigit() for c in
password):
            print("ERROR: Password must include letters and numbers.")
            return

        if password != confirm:
            print("ERROR: Passwords do not match.")
            return

        self.cursor.execute("SELECT 1 FROM USER WHERE username = ?", (username,))
        if self.cursor.fetchone():
            print("ERROR: This username is already taken.")
            return

        record = self.hasher.create_password_record(password)
```

```
        self.cursor.execute(
            "INSERT INTO USER (username, salt, hashedPassword) VALUES (?, ?, ?)",
            (username, record["salt"], record["hash"])
        )
        self.conn.commit()

        print("SUCCESS: Account created.")


    def login(self):
        print("\n=== LOGIN ===")
        username = input("Username: ")
        password = input("Password: ")

        if not username or not password:
            print("ERROR: Please fill in all fields.")
            return

        self.cursor.execute(
            "SELECT salt, hashedPassword FROM USER WHERE username = ?",
            (username,)
        )
        result = self.cursor.fetchone()

        if not result:
            print("ERROR: This user does not exist. Please sign up first.")
            return

        salt, stored_hash = result

        if self.hasher.verify_password(password, salt, stored_hash):
            print("SUCCESS: Login successful.")
            print("Session token:", secrets.token_hex(16))
        else:
            print("ERROR: Incorrect password for this user.")

    def close(self):
        self.conn.close()



if __name__ == "__main__":
    tester = AuthBackend()

    while True:
        print("\n1. Sign Up")
        print("2. Login")
```

```
        print("3. Exit")

        choice = input("Choose option: ")

        if choice == "1":
            tester.signup()
        elif choice == "2":
            tester.login()
        elif choice == "3":
            tester.close()
            print("Program closed.")
            break
        else:
            print("Invalid option.")
```

Test ID	Feature Under Test	Test Input / Action	Expected Outcome	Actual Outcome	Status
T39	Username validation	Attempted to sign up with username containing special characters (e.g. user@123)	The system should reject the username and display an error message	Program displayed error stating username can only contain letters, numbers, and underscores	Pass
T40	Empty field validation	Attempted signup leaving one or more fields blank	The system should prevent account creation and display an error	Program displayed error asking the user to fill in all fields	Pass
T41	Password length requirement	Entered a password shorter than 8 characters	The system should reject the password and display an error	Program displayed error stating password must be at least 8 characters long	Pass
T42	Password complexity validation	Entered a password containing only letters (no numbers)	The system should reject the password	Program displayed error stating password must include letters and numbers	Pass
T43	Password confirmation check	Entered two different passwords in the password and confirm fields	The system should reject the signup request	Program displayed error stating passwords do not match	Pass

T44	Duplicate username detection	Attempted to sign up with a username that already exists in the database	The system should prevent duplicate accounts	Program displayed error stating the username is already taken	Pass
T45	Password hashing and storage	Successfully created an account with valid username and password	The system should generate a salt and store the hashed password in the database	Account created successfully and hashed password stored	Pass
T46	Successful login	Entered correct username and password for an existing account	The system should authenticate the user and generate a session token	Login successful message displayed, and session token generated	Pass
T47	Incorrect password handling	Attempted login with correct username but incorrect password	The system should deny access and display an error	Program displayed error indicating incorrect password	Pass
T48	Non-existent user login	Attempted login using a username that does not exist	The system should inform the user that the account does not exist	Program displayed message prompting user to sign up first	Pass

The program was further improved by integrating with database in such a way the user login and sign-up page is saved and overall verification in the database stage in database with the structure. The overall solution is implemented in the hashing algorithm page the object of Auth backend which was previously developed as proof of concept but instead accommodates the database flow of the program and all the validation of the users input moved to the frontend.

```
def user_exists(self, username: str) -> bool:
        self.cursor.execute("SELECT 1 FROM USER WHERE username = ? LIMIT 1;",
(username,))
        return self.cursor.fetchone() is not None
    def get_user_credentials(self, username: str):
        self.cursor.execute(
            """
            SELECT userID, salt, hashedPassword
            FROM USER
```

```
            WHERE username = ?
            LIMIT 1;
            """,
            (username,),
        )
        return self.cursor.fetchone()
    def create_user_account(self, username: str, password: str) -> int:
        record = self.hasher.create_password_record(password)
        self.cursor.execute(
            """
            INSERT INTO USER (username, salt, hashedPassword)
            VALUES (?, ?, ?);
            """,
            (username, record["salt"], record["hash"]),
        )
        self.connect_db.commit()
        return self.cursor.lastrowid
```

R

Frontend:

4th February 2026

GUI initialization

The GUI needs a template which would be implemented in all the pages, so the first implementation is the class structure with parent class containing the tkinter module and the geometry of the page.

```
import tkinter as tk
from tkinter import messagebox
# UI theme colors
COLORS = {
    "bg": "#000000",
    "card": "#FFFFFF",
    "card_border": "#D9E2EC",
    "text": "#19326C",
    "muted": "#5B667A",
    "accent": "#0B7285",
    "accent_hover": "#0A6171",
    "neutral": "#E7EDF2",
    "neutral_hover": "#D6E0E8",
    "input_bg": "#F8FAFC",
    "input_border": "#C8D4E0",
    "focus": "#0B7285",
    "danger": "#B42318",
    "success": "#0F8A5F",
}

# UI fonts
FONT_TITLE = ("SF Pro Display", 32, "bold")
FONT_HEADING = ("SF Pro Display", 22, "bold")
FONT_BODY = ("Avenir Next", 12)
FONT_LABEL = ("Avenir Next", 11)
FONT_BUTTON = ("Avenir Next", 12, "bold")
FONT_SMALL = ("Avenir Next", 10)
class MarketAnalysisApp(tk.Tk):
    '''Main application class for market analysis tool.'''
    def __init__(self):
        '''Initialize the main application window.'''
        super().__init__()
        # Set window title
        self.title("Market Analysis Tool")
        # Set window size and minimum size
        self.geometry("1020x700")
        self.minsize(900, 620)
        # Configure background color
        self.configure(bg=COLORS["bg"])
```

Base page

5th February 2026

Every page in the GUI has common elements such as the overall background of the page and the design of it, this also includes the buttons text boxes and the display cards. Which whenever need could be inherited by each GUI object page using inheritance.

```
class BasePage(tk.Frame):
    '''Base page class for common UI elements.'''
    def __init__(self, master):
        '''Initialize the base page with background decorations.'''
        super().__init__(master, bg=COLORS["bg"])

        # Create canvas for background decorations
        self.decor = tk.Canvas(self, bg=COLORS["bg"], highlightthickness=0, bd=0)
        self.decor.place(relx=0, rely=0, relwidth=1, relheight=1)
        # Bind configure event to draw background
        self.decor.bind("<Configure>", self._draw_background)

        # Create content frame
        self.content = tk.Frame(self, bg=COLORS["bg"])
        self.content.place(relx=0.5, rely=0.5, anchor="center")

    def _draw_background(self, event):
        '''Draw the background ovals.'''
        self.decor.delete("all")
        w, h = event.width, event.height
        self.decor.create_oval(w - 260, -110, w + 110, 240, fill="#DDEAF2",
outline="")
        self.decor.create_oval(-170, h - 260, 220, h + 120, fill="#E3EEF5",
outline="")

    def create_card(self, parent, width=500):
        '''Create a card frame with styling.'''
        card = tk.Frame(
            parent,
            bg=COLORS["card"],
            highlightbackground=COLORS["card_border"],
            highlightthickness=1,
            bd=0,
            padx=34,
            pady=28,
        )
        if width:
            card.configure(width=width)
```

```
        return card

    def create_label(self, parent, text, font=FONT_LABEL, fg=None, pady=(0, 8)):
        '''Create a label with styling.'''
        label = tk.Label(
            parent,
            text=text,
            font=font,
            bg=COLORS["card"],
            fg=fg or COLORS["muted"],
            anchor="w",
        )
        label.pack(fill="x", pady=pady)
        return label

    def create_entry(self, parent, textvariable=None, show=None):
        '''Create an entry field with styling.'''
        entry = tk.Entry(
            parent,
            textvariable=textvariable,
            font=FONT_BODY,
            bg=COLORS["input_bg"],
            fg=COLORS["text"],
            relief="flat",
            insertbackground=COLORS["text"],
            highlightthickness=1,
            highlightbackground=COLORS["input_border"],
            highlightcolor=COLORS["focus"],
            bd=0,
            show=show,
        )
        entry.pack(fill="x", ipady=10)
        return entry

    def create_button(self, parent, text, command, primary=True, pady=(0, 0)):
        '''Create a button with styling and hover effects.'''
        bg = COLORS["accent"] if primary else COLORS["neutral"]
        hover = COLORS["accent_hover"] if primary else COLORS["neutral_hover"]

        btn = tk.Button(
            parent,
            text=text,
            command=command,
            font=FONT_BUTTON,
            bg=bg,
            fg=COLORS["text"],
            activebackground=hover,
            activeforeground=COLORS["text"],
```

```
            relief="flat",
            bd=0,
            padx=16,
            pady=12,
            cursor="hand2",
        )
        btn.pack(fill="x", pady=pady)
        # Bind hover events
        btn.bind("<Enter>", lambda _e: btn.configure(bg=hover))
        btn.bind("<Leave>", lambda _e: btn.configure(bg=bg))
        return btn
```

Start page

5th February 2026

This is the initial page which the users interact at first the overall GUI of the page and functions of the code.

```
class StartPage(BasePage):
    '''Start page for the welcome screen.'''
    def __init__(self, master):
        '''Initialize the start page with title and buttons.'''
        super().__init__(master)
        shell = tk.Frame(self.content, bg=COLORS["bg"])
        shell.pack()

        tk.Label(shell, text="Market Analysis", font=FONT_TITLE, bg=COLORS["bg"],
fg=COLORS["text"]).pack(pady=(0, 12))
        tk.Label(shell, text="Secure account access with live portfolio
intelligence.", font=FONT_BODY, bg=COLORS["bg"], fg=COLORS["muted"]).pack(pady=(0,
24))

        card = self.create_card(shell)
        card.pack()

        self.create_label(card, "Welcome", font=FONT_HEADING, fg=COLORS["text"],
pady=(0, 8))
        self.create_label(card, "Sign in or create a new account to continue.",
font=FONT_BODY, pady=(0, 24))
        self.create_button(card, "Log In", master.show_login_page, primary=True,
pady=(0, 10))
        self.create_button(card, "Create Account", master.show_signup_page,
primary=False)
```

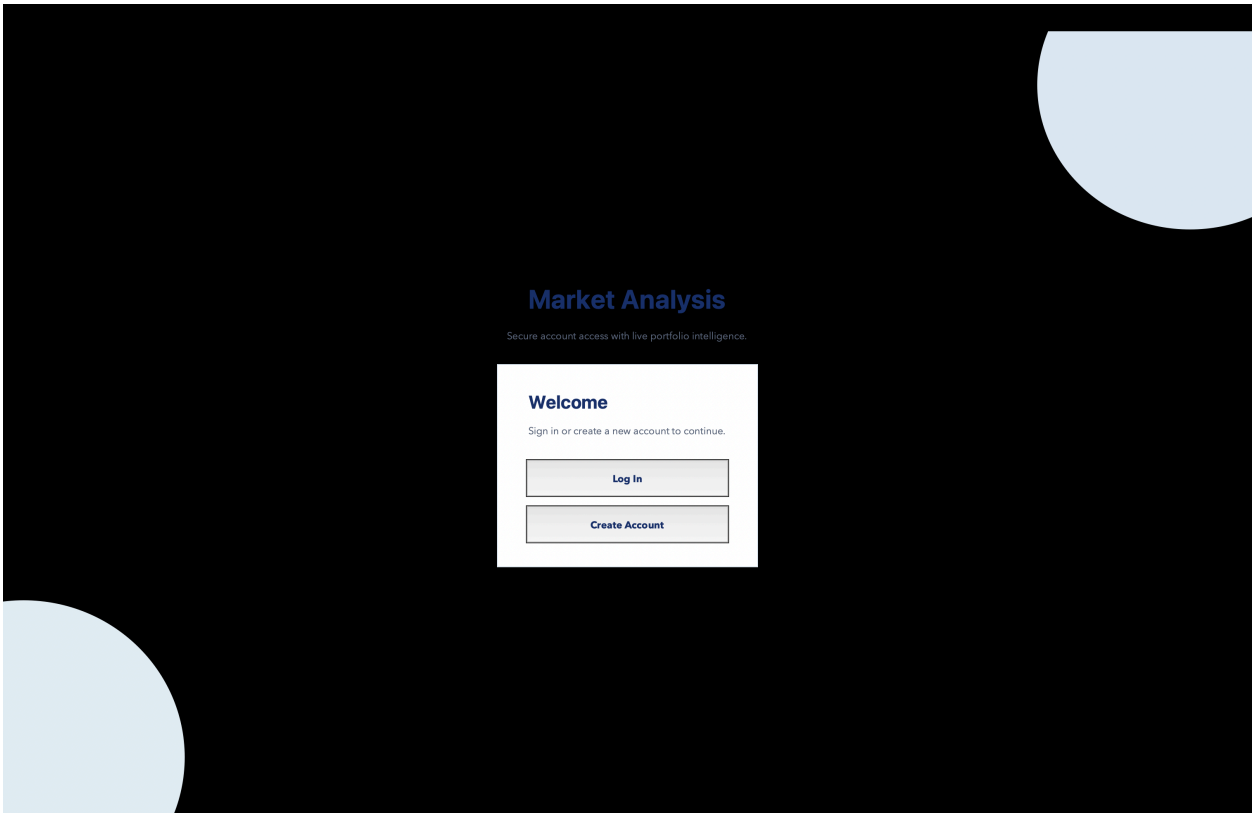
For the function of switching pages when log_in button or sign in page the function is handling was next to be developed in the parent class of MarketanalysisAPP()

```
def show_login_page(self):
        '''Show the login page.'''
        self.clear_window()
        LoginPage(self).pack(fill="both", expand=True)
```

For switching pages with next so for every page the function was implemented. Moreover, to ensure the widgets for the current page is cleared such that next page can open the following is implemented.

```
def clear_window(self):
        '''Clear all widgets from the window.'''
        for child in self.winfo_children():
            child.destroy()
```

Result:



Test result:

Test ID	Feature Tested	Test Input / Action	Expected Result	Actual Result
T49	Start page loads	Run the application	Start page loads showing title “Market Analysis” and welcome message	Pass
T50	Log In button navigation	Click Log In button	Application opens Login Page	Pass
T51	Create Account button navigation	Click Create Account button	Application opens Signup Page	Pass
T52	UI layout rendering	Open start page window	Title, description text, and card layout appear correctly	Pass
T53	Button hover styling	Hover mouse over buttons	Button colour changes to hover colour	Pass
T54	Windows resize behavior	Resize application window	UI elements remain visible and centered	Pass

Login page/ sign up page

First the signup page must connect with hashing algorithm file to ensure the backend logic relates to the GUI

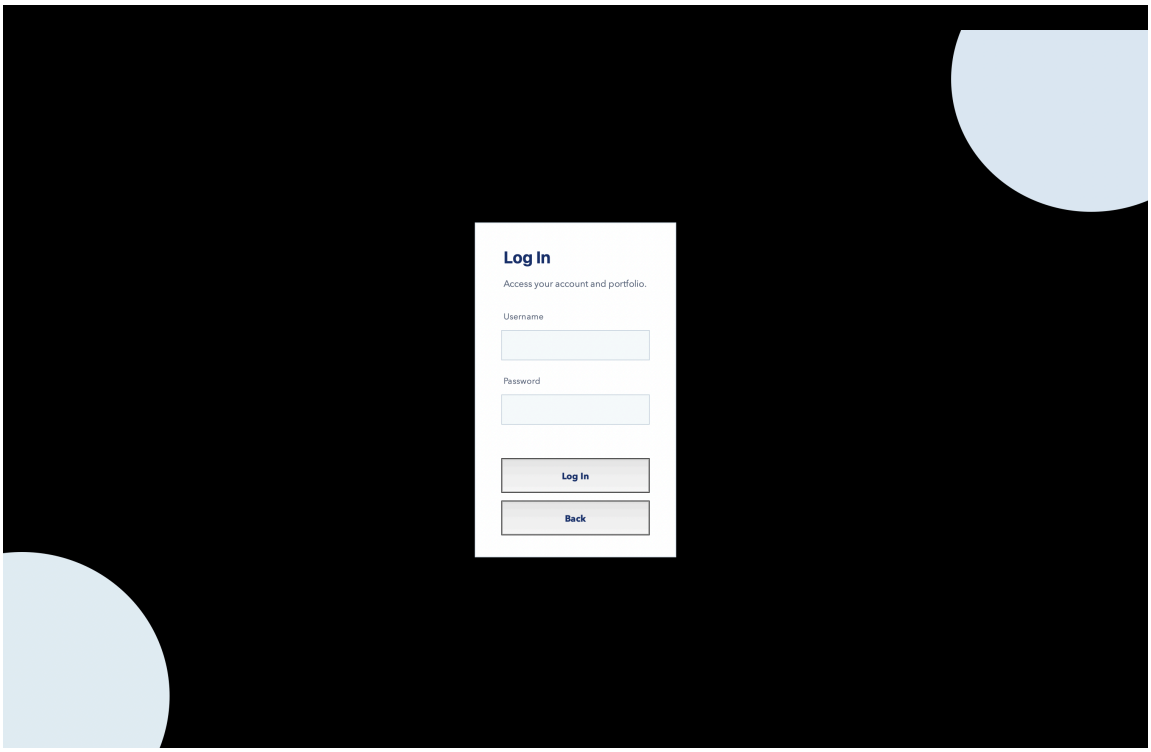
```
from hashingalgorithm import AuthBackend
```

after the connection the following:

```
class LoginPage(BasePage):
    '''Login page for user authentication.'''
    def __init__(self, master):
        '''Initialize the login page with form fields.'''
        super().__init__(master)
        self.username = tk.StringVar()
        self.password = tk.StringVar()
        card = self.create_card(self.content)
        card.pack()
        self.create_label(card, "Log In", font=FONT_HEADING, fg=COLORS["text"],
pady=(0, 8))
        self.create_label(card, "Access your account and portfolio.", font=FONT_BODY,
pady=(0, 22))
        self.create_label(card, "Username")
        self.create_entry(card, self.username)
        self.create_label(card, "Password", pady=(16, 8))
        password_entry = self.create_entry(card, self.password, show="*")
        self.status = tk.Label(card, text="", font=FONT_SMALL, bg=COLORS["card"],
fg=COLORS["danger"], anchor="w")
```

```
        self.status.pack(fill="x", pady=(12, 12))
        self.create_button(card, "Log In", self.validate_login, primary=True, pady=(0,
10))
        self.create_button(card, "Back", master.show_start_page, primary=False)
        password_entry.bind("<Return>", lambda _e: self.validate_login())

    def validate_login(self):
        '''Validate the login credentials.'''
        username = self.username.get().strip()
        password = self.password.get()

        if not username or not password:
            self.status.config(text="Please enter both username and password.",
fg=COLORS["danger"])
            return

        if not self.master.auth.user_exists(username):
            self.status.config(text="User does not exist.", fg=COLORS["danger"])
            return

        user_id = self.master.auth.verify_login(username, password)
        if user_id is not None:
            self.master.current_username = username
            self.master.current_user_id = user_id
            self.status.config(text="Login successful.", fg=COLORS["success"])
            self.after(350, self.master.show_portfolio_page)
        else:
            self.status.config(text="Incorrect password.", fg=COLORS["danger"])
```

The result:



Test table:

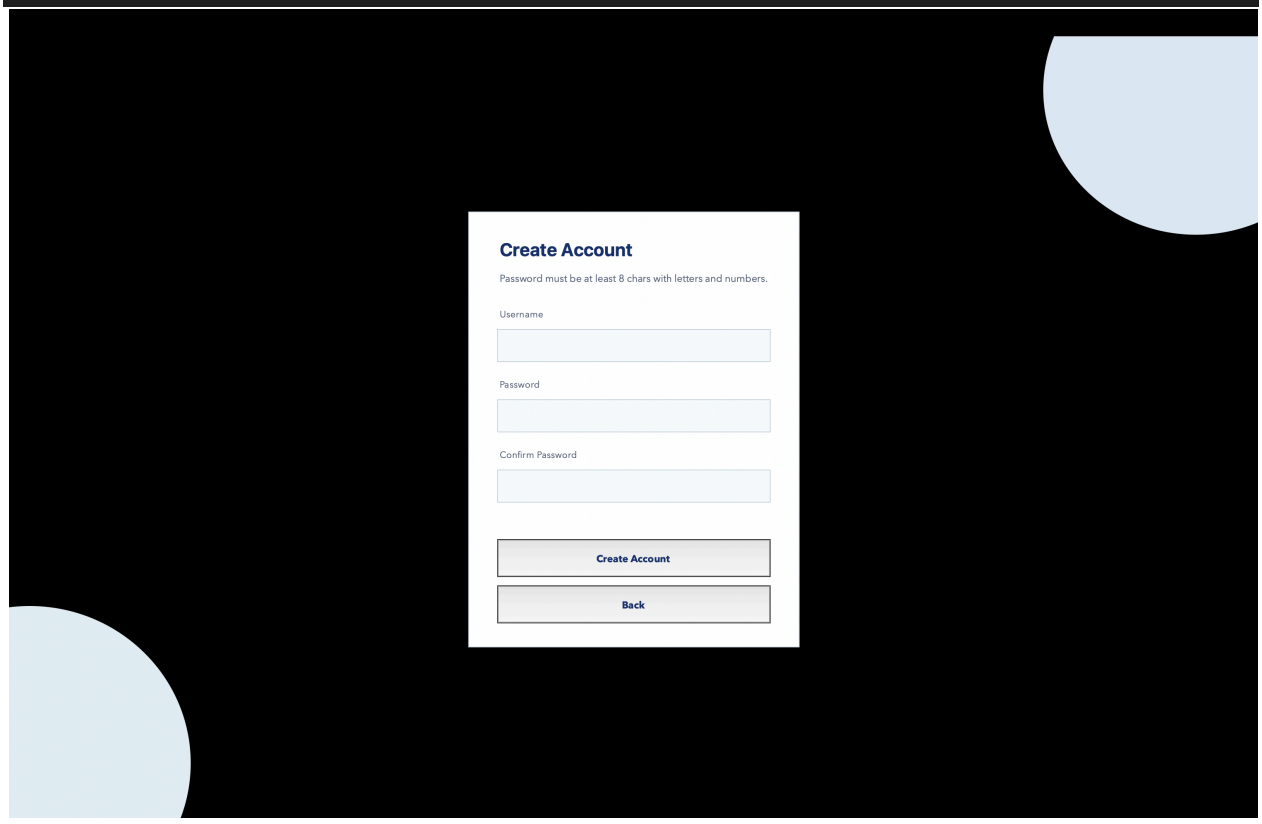
Test ID	Test Type	Feature Tested	Test Input / Action	Expected Result	Actual Result
T55	Normal	Valid login	Enter correct username and password then click Log In	User successfully logged in and portfolio page opens	Pass
T56	Erroneous	Empty fields	Leave username and password blank and click Log In	Error message: “Please enter both username and password.”	Pass
T57	Erroneous	Non-existent user	Enter username that does not exist	Error message: “User does not exist.”	Pass
T58	Erroneous	Incorrect password	Enter valid username but wrong password	Error message: “Incorrect password.”	Pass
T59	Boundary	Minimum valid login	Enter valid username and minimum valid password	Login successful	Pass
T60	Navigation	Back button	Click Back button	Application returns to start page	Pass

T61	Usability	Enter key login	Press Enter after entering password	Login validation triggered	Pass

Sign up page

```
class SignupPage(BasePage):
    '''Signup page for creating new accounts.'''
    def __init__(self, master):
        '''Initialize the signup page with form fields.'''
        super().__init__(master)
        self.username = tk.StringVar()
        self.password = tk.StringVar()
        self.confirm = tk.StringVar()
        card = self.create_card(self.content)
        card.pack()
        self.create_label(card, "Create Account", font=FONT_HEADING,
fg=COLORS["text"], pady=(0, 8))
        self.create_label(card, "Password must be at least 8 chars with letters and
numbers.", font=FONT_BODY, pady=(0, 22))
        self.create_label(card, "Username")
        self.create_entry(card, self.username)
        self.create_label(card, "Password", pady=(16, 8))
        self.create_entry(card, self.password, show="*")
        self.create_label(card, "Confirm Password", pady=(16, 8))
        confirm_entry = self.create_entry(card, self.confirm, show="*")
        self.status = tk.Label(card, text="", font=FONT_SMALL, bg=COLORS["card"],
fg=COLORS["danger"], anchor="w")
        self.status.pack(fill="x", pady=(12, 12))
        self.create_button(card, "Create Account", self.create_account, primary=True,
pady=(0, 10))
        self.create_button(card, "Back", master.show_start_page, primary=False)
        confirm_entry.bind("<Return>", lambda _e: self.create_account())

    def create_account(self):
        '''Create a new user account.'''
        username = self.username.get().strip()
        password = self.password.get()
        confirm = self.confirm.get()
        if not username or not password or not confirm:
            self.status.config(text="Fill all fields.", fg=COLORS["danger"])
            return
        if not re.match(r"^\w+$", username):
            self.status.config(text="Username: letters, numbers, underscore only.",
fg=COLORS["danger"])
```

```
            return
        if len(password) < 8:
            self.status.config(text="Password must be at least 8 characters.",
fg=COLORS["danger"])
            return
        if not any(c.isalpha() for c in password) or not any(c.isdigit() for c in
password):
            self.status.config(text="Password must include letters and numbers.",
fg=COLORS["danger"])
            return
        if password != confirm:
            self.status.config(text="Passwords do not match.", fg=COLORS["danger"])
            return
        if self.master.auth.user_exists(username):
            self.status.config(text="Username already exists.", fg=COLORS["danger"])
            return
        self.master.auth.create_user_account(username, password)
        self.status.config(text="Account created. Redirecting to login...",
fg=COLORS["success"])
        self.after(600, self.master.show_login_page)
```



Test table:

Test ID	Test Type	Feature Tested	Test Input / Action	Expected Result	Actual Result
T62	Normal	Create valid account	Enter valid username and password meeting requirements	Account successfully created and redirected to login page	Pass
T63	Erroneous	Empty fields	Leave all fields blank and click Create Account	Error message: “Fill all fields.”	Pass
T64	Erroneous	Invalid username characters	Enter username with special characters (e.g. user@123)	Error message: “Username: letters, numbers, underscore only.”	Pass
T65	Boundary	Short password	Enter password shorter than 8 characters	Error message: “Password must be at least 8 characters.”	Pass
T66	Erroneous	No numbers in password	Enter password containing only letters	Error message: “Password must include letters and numbers.”	Pass
T67	Erroneous	Password mismatch	Enter different password and confirm password	Error message: “Passwords do not match.”	Pass
T68	Erroneous	Existing username	Enter username already registered	Error message: “Username already exists.”	Pass
T69	Boundary	Minimum valid password	Enter password exactly 8 characters with letters and numbers	Account created successfully	Pass

Portfolio page

10th – 11th February 2026

For the retrieval of the user currently using the program for the portfolio page a connection was made to the database in the market analysis parent class.

```
DatabaseManager()
        # Create auth backend
        self.auth = AuthBackend(DB_PATH)
        # Create data retrieval service
        self.data = DataRetrieval(DB_PATH)

        # Set current user to None
        self.current_user_id = None
        self.current_username = None

        # Show start page
        self.show_start_page()
        # Bind close event
        self.protocol("WM_DELETE_WINDOW", self.on_close)
```

The portfolio page code:

```
class PortfolioPage(BasePage):
    '''Portfolio page for managing user stocks.'''
    def __init__(self, master):
        '''Initialize the portfolio page with search and list widgets.'''
        super().__init__(master)

        wrapper = tk.Frame(self.content, bg=COLORS["bg"])
        wrapper.pack(fill="both", expand=True)

        tk.Label(wrapper, text=f"Portfolio • {master.current_username}",
font=FONT_HEADING, bg=COLORS["bg"], fg=COLORS["text"]).pack(anchor="w", pady=(0, 14))

        actions = tk.Frame(wrapper, bg=COLORS["bg"])
        actions.pack(fill="x", pady=(0, 12))

        tk.Button(actions, text="Back", command=master.show_start_page,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=8, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left")
        tk.Button(actions, text="Refresh", command=self.refresh_portfolio,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=8, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))
        tk.Button(actions, text="Open Stock Page", command=self.open_selected_stock,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
```

```
padx=12, pady=8, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))
        tk.Button(actions, text="Delete Selected Stock",
command=self.delete_selected_stock, font=FONT_SMALL, bg=COLORS["neutral"],
fg=COLORS["text"], relief="flat", bd=0, padx=12, pady=8,
activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))
        tk.Button(actions, text="Delete User", command=self.delete_user_account,
font=FONT_SMALL, bg=COLORS["danger"], fg="#FFFFFF", relief="flat", bd=0, padx=12,
pady=8, activebackground="#9C1F14", activeforeground="#FFFFFF").pack(side="right")

        top_card = self.create_card(wrapper, width=820)
        top_card.pack(fill="x", pady=(0, 10))

        self.create_label(top_card, "Search Stock", font=FONT_BODY, fg=COLORS["text"],
pady=(0, 8))

        search_row = tk.Frame(top_card, bg=COLORS["card"])
        search_row.pack(fill="x")

        self.search_var = tk.StringVar()
        self.search_entry = tk.Entry(
            search_row,
            textvariable=self.search_var,
            font=FONT_BODY,
            bg=COLORS["input_bg"],
            fg=COLORS["text"],
            relief="flat",
            insertbackground=COLORS["text"],
            highlightthickness=1,
            highlightbackground=COLORS["input_border"],
            highlightcolor=COLORS["focus"],
        )
        self.search_entry.pack(side="left", fill="x", expand=True, ipady=9)

        tk.Button(search_row, text="Search", command=self.search_stock,
font=FONT_SMALL, bg=COLORS["accent"], fg=COLORS["text"], relief="flat", bd=0, padx=12,
pady=10, activebackground=COLORS["accent_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))

        self.dropdown = tk.Listbox(top_card, height=5, font=FONT_SMALL,
bg=COLORS["input_bg"], fg=COLORS["text"], selectbackground="#CFE8EC", relief="flat")
        self.dropdown.pack(fill="x", pady=(10, 8))
        self.dropdown.bind("<<ListboxSelect>>", self.select_stock)

        tk.Button(top_card, text="Add Selected Stock",
command=self.add_selected_stock, font=FONT_SMALL, bg=COLORS["neutral"],
fg=COLORS["text"], relief="flat", bd=0, padx=12, pady=10,
```

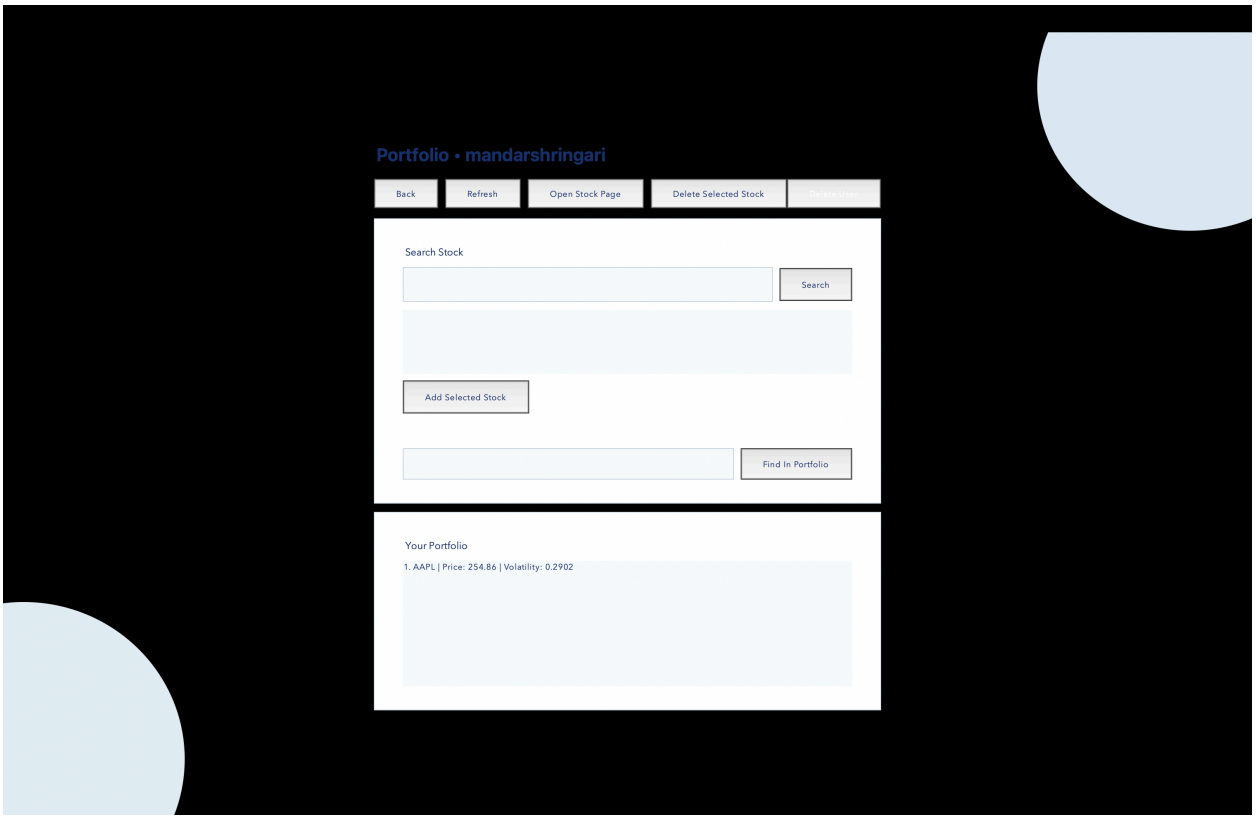
```
activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(anchor="w")

        self.search_status = tk.Label(top_card, text="", font=FONT_SMALL,
bg=COLORS["card"], fg=COLORS["danger"], anchor="w")
        self.search_status.pack(fill="x", pady=(10, 0))

        filter_row = tk.Frame(top_card, bg=COLORS["card"])
        filter_row.pack(fill="x", pady=(12, 0))

        self.filter_var = tk.StringVar()
        tk.Entry(filter_row, textvariable=self.filter_var, font=FONT_SMALL,
bg=COLORS["input_bg"], fg=COLORS["text"], relief="flat",
insertbackground=COLORS["text"], highlightthickness=1,
highlightbackground=COLORS["input_border"],
highlightcolor=COLORS["focus"]).pack(side="left", fill="x", expand=True, ipady=8)
        tk.Button(filter_row, text="Find In Portfolio", command=self.search_portfolio,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=9, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))

        list_card = self.create_card(wrapper, width=820)
        list_card.pack(fill="both", expand=True)

        self.create_label(list_card, "Your Portfolio", font=FONT_BODY,
fg=COLORS["text"], pady=(0, 8))

        self.portfolio_list = tk.Listbox(list_card, font=FONT_SMALL,
bg=COLORS["input_bg"], fg=COLORS["text"], relief="flat", selectbackground="#CFE8EC")
        self.portfolio_list.pack(fill="both", expand=True)
        self.portfolio_list.bind("<Double-Button-1>", lambda _e:
self.open_selected_stock())

        self.selected_search_result = None
        self.portfolio_items = []
        self.refresh_portfolio()

    def search_stock(self):
        '''Search for stocks based on the query.'''
        query = self.search_var.get().strip()
        self.dropdown.delete(0, tk.END)
        self.search_status.config(text="", fg=COLORS["danger"])

        if not query:
            self.search_status.config(text="Enter a stock ticker or company name.")
            return

        try:
```

```
            results = self.master.data.stock_search(query)
        except Exception as err:
            self.search_status.config(text=f"Search failed: {err}")
            return

        if not results:
            self.search_status.config(text="No matching stocks found.")
            return

        for ticker, company in results:
            self.dropdown.insert(tk.END, f"{ticker} - {company}")

    def select_stock(self, _event):
        '''Handle stock selection from the dropdown.'''
        picked = self.dropdown.curselection()
        if picked:
            self.selected_search_result = self.dropdown.get(picked[0]).split(" - ")[0]

    def add_selected_stock(self):
        '''Add the selected stock to the portfolio.'''
        ticker = self.selected_search_result
        if not ticker:
            self.search_status.config(text="Select a stock from the list first.",
fg=COLORS["danger"])
            return

        user_id = self.master.current_user_id
        try:
            existing = self.master.data.get_portfolio_for_ticker(user_id, ticker)
            portfolio_id = existing if existing else
self.master.data.create_portfolio(user_id, ticker)
            self.master.data.insert_historical_data(portfolio_id, user_id, ticker)
            self.master.data.update_portfolio_metrics(portfolio_id)
        except Exception as err:
            self.search_status.config(text=f"Could not add stock: {err}",
fg=COLORS["danger"])
            return

        self.search_status.config(text=f"{ticker} added/updated successfully.",
fg=COLORS["success"])
        self.refresh_portfolio()

    def refresh_portfolio(self):
        '''Refresh the portfolio list.'''
        self.portfolio_list.delete(0, tk.END)
        self.portfolio_items = []
        user_id = self.master.current_user_id
```

```
        try:
            rows = self.master.data.get_user_portfolio(user_id)
            ranked = self.master.data._quicksort_by_volatility(rows)
        except Exception as err:
            self.portfolio_list.insert(tk.END, f"Failed to load portfolio: {err}")
            return

        if not ranked:
            self.portfolio_list.insert(tk.END, "No stocks in portfolio yet.")
            return

        self.portfolio_items = ranked
        for idx, item in enumerate(ranked, start=1):
            price = f"{item['currentPrice']:.2f}" if item["currentPrice"] is not None
else "N/A"
            vol = f"{item['volatility']:.4f}" if item["volatility"] is not None else
"N/A"
            self.portfolio_list.insert(tk.END, f"{idx}. {item['ticker']} | Price:
{price} | Volatility: {vol}")

    def delete_selected_stock(self):
        '''Delete the selected stock from the portfolio.'''
        picked = self.portfolio_list.curselection()
        if not picked or not self.portfolio_items:
            self.search_status.config(text="Select a stock from your portfolio
first.", fg=COLORS["danger"])
            return

        selected_index = picked[0]
        if selected_index >= len(self.portfolio_items):
            self.search_status.config(text="Select a valid stock entry.",
fg=COLORS["danger"])
            return

        item = self.portfolio_items[selected_index]
        ticker = item["ticker"]
        portfolio_id = item["portfolioID"]

        confirmed = messagebox.askyesno(
            "Delete Stock",
            f"Delete {ticker} from your portfolio?",
            parent=self,
        )
        if not confirmed:
            return

        try:
```

```
            removed =
self.master.data.remove_stock_for_user(self.master.current_user_id, portfolio_id)
        except Exception as err:
            self.search_status.config(text=f"Could not delete stock: {err}",
fg=COLORS["danger"])
            return

        if not removed:
            self.search_status.config(text="Could not find that stock for this user.",
fg=COLORS["danger"])
            return

        self.search_status.config(text=f"{ticker} deleted from portfolio.",
fg=COLORS["success"])
        self.refresh_portfolio()

    def open_selected_stock(self):
        '''Open the detail page for the selected stock.'''
        picked = self.portfolio_list.curselection()
        if not picked or not self.portfolio_items:
            self.search_status.config(text="Select a stock from your portfolio
first.", fg=COLORS["danger"])
            return

        index = picked[0]
        if index >= len(self.portfolio_items):
            self.search_status.config(text="Select a valid stock entry.",
fg=COLORS["danger"])
            return

        self.master.show_stock_detail_page(self.portfolio_items[index])

    def delete_user_account(self):
        '''Delete the current user account.'''
        user_id = self.master.current_user_id
        username = self.master.current_username
        if user_id is None:
            self.search_status.config(text="No active user session.",
fg=COLORS["danger"])
            return

        confirmed = messagebox.askyesno(
            "Delete User",
            f"Delete user '{username}' and all portfolio data?",
            parent=self,
        )
        if not confirmed:
            return
```

```
        try:
            self.master.data.delete_user_account(user_id)
        except Exception as err:
            self.search_status.config(text=f"Could not delete user: {err}",
fg=COLORS["danger"])
            return

        self.master.current_user_id = None
        self.master.current_username = None
        self.master.show_start_page()

    def search_portfolio(self):
        '''Search for a stock in the portfolio.'''
        query = self.filter_var.get().strip()
        if not query:
            return

        try:
            matches = self.master.data.search_portfolio_tickers(query,
self.master.current_user_id)
        except Exception as err:
            self.search_status.config(text=f"Portfolio search failed: {err}",
fg=COLORS["danger"])
            return

        if not matches:
            self.search_status.config(text="Ticker not found in your portfolio.",
fg=COLORS["danger"])
            return

        target = matches[0].upper()
        for idx in range(self.portfolio_list.size()):
            text = self.portfolio_list.get(idx).upper()
            if f"{target} " in text or f"{target}|" in text:
                self.portfolio_list.selection_clear(0, tk.END)
                self.portfolio_list.selection_set(idx)
                self.portfolio_list.see(idx)
                self.search_status.config(text=f"Found {target} in portfolio.",
fg=COLORS["success"])
                return
```



Test ID	Test Type	Test Description	Test Input / Action	Expected Output	Actual Output
T70	Normal	Load portfolio page	User logs in and portfolio page loads	Portfolio page displays user's stocks	Pass
T71	Normal	Search for stock	Enter ticker AAPL and press Search	Matching stock appears in dropdown	Pass
T72	Normal	Add stock to portfolio	Select AAPL and click Add Selected Stock	Stock added to portfolio list with price and volatility	Pass

T73	Erroneous	Add stock without selecting	Click Add Selected Stock without choosing from dropdown	Error message shown	Pass
T74	Normal	Delete stock	Select stock in portfolio and click Delete Selected Stock	Stock removed from portfolio	Pass
T75	Normal	Open stock detail page	Double-click a stock in the list	Stock detail page opens showing chart	Pass
T76	Normal	Search stock inside portfolio	Enter existing ticker in Find In Portfolio	Portfolio highlights matching stock	Pass
T77	Boundary	Empty search field	Click Search with empty input	Error message asking user to enter ticker	Pass
T78	Usability	Refresh portfolio	Click Refresh button	Portfolio list updates with latest values	Fail
T79	Functional	Update current price and volatility	Refresh portfolio after adding stock	Current price and volatility values should update with latest data	Fail – values remain unchanged

Limitation
With multiple attempts to implement dynamically changing graph in backend similar approaches were tried for automatically changing the price in backend but overall, the both the method this will be reflected in the success criteria as both the features will not be meet. So instead, a refresh button was implemented for both the stock page and portfolio page to ensure the financial data in both pages are updated and relevant however it must be done by the user than it to be automatically

Iteration for the refresh button to fix the portfolio page.
The following code was changed:

```
rows = self.master.data.get_user_portfolio(user_id)
ranked = self.master.data._quicksort_by_volatility(rows)
```

to:

```
rows = self.master.data.get_user_portfolio(user_id)
```

```
# Update each stock before displaying
for item in rows:
    portfolio_id = item["portfolioID"]
    ticker = item["ticker"]

    try:
        self.master.data.insert_historical_data(portfolio_id, user_id, ticker)
        self.master.data.update_portfolio_metrics(portfolio_id)
    except Exception:
        pass

# Reload updated values
rows = self.master.data.get_user_portfolio(user_id)

ranked = self.master.data._quicksort_by_volatility(rows)
```

More detailed stock page

This the code for the in more detailed page of the stock ticker with the integration of the predictive model and plotting with historical data with integration with tkinter GUI with matplotlib module from the data plotting.py which itself is linked with the regression model to make sure the whole of plotting in one place and easily be linked with the GUI.

```
class StockDetailPage(BasePage):
    '''Stock detail page for viewing individual stock data.'''
    def __init__(self, master, stock_item):
        '''Initialize the stock detail page with data and chart.'''
        super().__init__(master)
        self.content.place_configure(relwidth=0.98, relheight=0.95)

        self.stock_item = stock_item
        self.ticker = stock_item["ticker"]
        self.portfolio_id = stock_item["portfolioID"]
        self.range_var = tk.StringVar(value="1Y")
        self.chart_canvas = None

        wrapper = tk.Frame(self.content, bg=COLORS["bg"])
        wrapper.pack(fill="both", expand=True)

        header = tk.Frame(wrapper, bg=COLORS["bg"])
        header.pack(fill="x", pady=(0, 10))

        tk.Button(
            header,
            text="Back to Portfolio",
            command=self.master.show_portfolio_page,
            font=FONT_SMALL,
            bg=COLORS["neutral"],
            fg=COLORS["text"],
            relief="flat",
            bd=0,
            padx=12,
            pady=8,
            activebackground=COLORS["neutral_hover"],
            activeforeground=COLORS["text"],
        ).pack(side="left")

        tk.Label(
            header,
            text=f"Individual Stock Page • {self.ticker}",
```

```
            font=FONT_HEADING,
            bg=COLORS["bg"],
            fg=COLORS["text"],
        ).pack(side="left", padx=(10, 0))

        info_card = self.create_card(wrapper, width=980)
        info_card.pack(fill="x", pady=(0, 10))

        self.create_label(info_card, "Abstracted Stock Data", font=FONT_BODY,
fg=COLORS["text"], pady=(0, 10))
        self.company_label = tk.Label(info_card, text=f"Company: {self.ticker}",
font=FONT_LABEL, bg=COLORS["card"], fg=COLORS["muted"], anchor="w")
        self.company_label.pack(fill="x", pady=(0, 8))

        self.metric_labels = {}
        metric_keys = ["Current Price", "Open", "High", "Low", "Close", "Volume"]
        metric_grid = tk.Frame(info_card, bg=COLORS["card"])
        metric_grid.pack(fill="x")
        for i, key in enumerate(metric_keys):
            lbl = tk.Label(metric_grid, text=f"{key}: N/A", font=FONT_LABEL,
bg=COLORS["card"], fg=COLORS["muted"], anchor="w")
            r, c = divmod(i, 3)
            lbl.grid(row=r, column=c, sticky="w", padx=8, pady=5)
            metric_grid.grid_columnconfigure(c, weight=1)
            self.metric_labels[key] = lbl

        chart_card = self.create_card(wrapper, width=980)
        chart_card.pack(fill="both", expand=True)
        self.create_label(chart_card, "Historical Stock Data + Forecast",
font=FONT_BODY, fg=COLORS["text"], pady=(0, 10))

        controls = tk.Frame(chart_card, bg=COLORS["card"])
        controls.pack(fill="x", pady=(0, 8))
        tk.Label(controls, text="Range:", font=FONT_SMALL, bg=COLORS["card"],
fg=COLORS["text"]).pack(side="left", padx=(0, 6))
        tk.OptionMenu(controls, self.range_var, "1M", "6M", "YTD", "1Y", "ALL",
command=lambda _v: self.render_chart()).pack(side="left")
        tk.Button(
            controls,
            text="Refresh Chart",
            command=self.render_chart,
            font=FONT_SMALL,
            bg=COLORS["neutral"],
            fg=COLORS["text"],
            relief="flat",
            bd=0,
            padx=12,
            pady=8,
```

```
            activebackground=COLORS["neutral_hover"],
            activeforeground=COLORS["text"],
        ).pack(side="left", padx=(8, 0))

        self.chart_status = tk.Label(chart_card, text="", font=FONT_SMALL,
bg=COLORS["card"], fg=COLORS["muted"], anchor="w")
        self.chart_status.pack(fill="x", pady=(0, 8))

        self.chart_host = tk.Frame(chart_card, bg=COLORS["card"])
        self.chart_host.pack(fill="both", expand=True)

        self.load_abstracted_data()
        self.render_chart()

    def _fmt_price(self, value):
        '''Format the price value.'''
        try:
            return f"${float(value):.2f}"
        except Exception:
            return "N/A"

    def _fmt_volume(self, value):
        '''Format the volume value.'''
        try:
            return f"{int(value):,}"
        except Exception:
            return "N/A"

    def load_abstracted_data(self):
        '''Load the stock data from yfinance or database.'''
        latest = None
        company_text = f"Company: {self.ticker}"
        try:
            tk_obj = yf.Ticker(self.ticker)
            info = tk_obj.info
            long_name = info.get("longName") or info.get("shortName")
            if long_name:
                company_text = f"Company: {long_name} ({self.ticker})"
            hist = tk_obj.history(period="5d", interval="1d")
            if not hist.empty:
                latest = hist.iloc[-1]
        except Exception:
            latest = None

        if latest is None:
            self.master.data.cursor.execute(
                """
                SELECT Open, High, Low, Close, Volume
```

```
                FROM STOCK_PLOTTING
                WHERE portfolioID = ?
                ORDER BY DateID DESC
                LIMIT 1
                """,
                (self.portfolio_id,),
            )
            row = self.master.data.cursor.fetchone()
            if row:
                latest = {
                    "Open": row[0],
                    "High": row[1],
                    "Low": row[2],
                    "Close": row[3],
                    "Volume": row[4],
                }

        self.company_label.config(text=company_text)
        if latest is None:
            self.metric_labels["Current Price"].config(text="Current Price: N/A")
            self.metric_labels["Open"].config(text="Open: N/A")
            self.metric_labels["High"].config(text="High: N/A")
            self.metric_labels["Low"].config(text="Low: N/A")
            self.metric_labels["Close"].config(text="Close: N/A")
            self.metric_labels["Volume"].config(text="Volume: N/A")
            return

        close_val = latest["Close"] if isinstance(latest, dict) else
latest.get("Close")
        self.metric_labels["Current Price"].config(text=f"Current Price:
{self._fmt_price(close_val)}")
        self.metric_labels["Open"].config(text=f"Open: {self._fmt_price(latest['Open']
if isinstance(latest, dict) else latest.get('Open'))}")
        self.metric_labels["High"].config(text=f"High: {self._fmt_price(latest['High']
if isinstance(latest, dict) else latest.get('High'))}")
        self.metric_labels["Low"].config(text=f"Low: {self._fmt_price(latest['Low'] if
isinstance(latest, dict) else latest.get('Low'))}")
        self.metric_labels["Close"].config(text=f"Close:
{self._fmt_price(close_val)}")
        self.metric_labels["Volume"].config(text=f"Volume:
{self._fmt_volume(latest['Volume'] if isinstance(latest, dict) else
latest.get('Volume'))}")

    def render_chart(self):
        '''Render the stock chart with historical data and forecast.'''
        plotter = StockDataPlotter(DB_PATH)
        fig = plotter.build_portfolio_figure(
            portfolio_id=self.portfolio_id,
```

```
            ticker_label=self.ticker,
            range_label=self.range_var.get(),
            include_forecast=True,
            future_days=30,
        )
        self.chart_status.config(text=plotter.last_status)

        if self.chart_canvas is not None:
            self.chart_canvas.get_tk_widget().destroy()

        self.chart_canvas = FigureCanvasTkAgg(fig, master=self.chart_host)
        self.chart_canvas.draw()
        self.chart_canvas.get_tk_widget().pack(fill="both", expand=True)


if __name__ == "__main__":
    # Run the main application
    app = MarketAnalysisApp()
    app.mainloop()
```

Test table:

Test ID	Test Type	Test Description	Test Input / Action	Expected Output	Actual Output
T80	Normal	Open stock detail page	Double-click stock in portfolio	Stock detail page opens showing ticker name and company information	Pass
T81	Normal	Display abstracted stock data	Open stock page	Current price, open, high, low, close, and volume are displayed	Pass
T82	Normal	Load stock chart	Stock page loads	Historical price chart displayed with forecast line	Pass
T83	Normal	Change time range	Select 1M, 6M, YTD, 1Y, or ALL in range dropdown	Chart updates to show selected time range	Pass
T84	Normal	Refresh chart	Click Refresh Chart button	Chart reloads and displays latest data	Pass
T85	Boundary	No historical data available	Stock has no stored data in database	Program retrieves latest data from API	Pass

				or displays N/A values	
T86	Erroneous	API failure	Internet connection disabled	Data loads from database fallback instead of crashing	Pass
T87	Navigation	Return to portfolio	Click Back to Portfolio button	User returned to portfolio page	Pass
T88	Functional	Chart rendering	Chart displayed in window	Chart scales correctly and fills display area	Pass
T89	Usability	Multiple refresh attempts	Click refresh several times	Chart updates without freezing or crashing	Pass

Final backend code with comments

The final code for every section for the program and the changes in code reflect the modularity in such way so they can be easily integrated with the frontend and all the integration required to the database and other vital parts of the backend.

Database.py file

```
import sqlite3

class DatabaseManager:
    def __init__(self):
        super().__init__()
        # Connect to the database file - this creates it if it doesn't exist
        self.connect_db = sqlite3.connect('marketanalysis_2.db')
        self.cursor_obj = self.connect_db.cursor()

        # Enable foreign keys so that relationships between tables work properly
        self.cursor_obj.execute("PRAGMA foreign_keys = ON;")

        # Create the USER table to store user accounts and login info
        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS USER(
            userID INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            salt INTEGER NOT NULL,
            hashedPassword TEXT NOT NULL
        );
        """)
        # Create the PORTFOLIO table for each user's stock portfolio
        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS PORTFOLIO(
            portfolioID INTEGER PRIMARY KEY AUTOINCREMENT,
            userID INTEGER NOT NULL,
            ticker TEXT NOT NULL,
            stockName TEXT,
            currentPrice REAL,
            volatility REAL,
            timestamp TIMESTAMP,
            FOREIGN KEY (userID) REFERENCES USER(userID)
        );
        """)
```

```
# Create the STOCK_PLOTTING table to store historical stock data for plotting
self.cursor_obj.execute("""
CREATE TABLE IF NOT EXISTS STOCK_PLOTTING(
    DateID DATE NOT NULL,
    portfolioID INTEGER NOT NULL,
    Open REAL NOT NULL,
    Close REAL NOT NULL,
    High REAL NOT NULL,
    Low REAL NOT NULL,
    Volume REAL NOT NULL,
    PRIMARY KEY (DateID, portfolioID),
    FOREIGN KEY (portfolioID) REFERENCES PORTFOLIO(portfolioID)
);
""")

# Create the STOCK_LIST table as a lookup for available stocks
self.cursor_obj.execute("""
CREATE TABLE IF NOT EXISTS STOCK_LIST(
    stockListID INTEGER PRIMARY KEY AUTOINCREMENT,
    tickername TEXT NOT NULL,
    companyname TEXT NOT NULL
);
""")
# This part would add some common tickers to the STOCK_LIST, but it's commented out for now
# I might uncomment it later if we need it
'''self.cursor_obj.executemany(
    "INSERT OR IGNORE INTO STOCK_LIST (tickername, companyname) VALUES (?, ?);",
    [
        # Major US stocks
        ("AAPL", "Apple Inc"),
        ("MSFT", "Microsoft Corporation"),
        ("GOOGL", "Alphabet Inc"),
        ("AMZN", "Amazon.com Inc"),
        ("META", "Meta Platforms Inc"),
        ("NVDA", "NVIDIA Corporation"),
        ("TSLA", "Tesla Inc"),
        ("BRK-B", "Berkshire Hathaway"),
        ("JPM", "JPMorgan Chase"),
        ("V", "Visa Inc"),
```

```
            # Indices
            ("^GSPC", "S&P 500"),
            ("^DJI", "Dow Jones Industrial Average"),
            ("^IXIC", "NASDAQ Composite"),
            ("^FTSE", "FTSE 100"),
            ("^N225", "Nikkei 225"),

            # Commodities
            ("GC=F", "Gold Futures"),
            ("SI=F", "Silver Futures"),
            ("CL=F", "Crude Oil"),
            ("NG=F", "Natural Gas"),

            # Forex
            ("GBPUSD=X", "GBP/USD"),
            ("EURUSD=X", "EUR/USD"),
            ("USDJPY=X", "USD/JPY"),

            # Crypto
            ("BTC-USD", "Bitcoin"),
            ("ETH-USD", "Ethereum"),
            ("BNB-USD", "Binance Coin"),
            ("SOL-USD", "Solana"),
            ("XRP-USD", "Ripple"),
            ("ADA-USD", "Cardano")
        ]
    )'''
    # These lines would clear the database each time, but they're commented out
    # I used them for testing to start fresh
    #self.cursor_obj.execute("DELETE FROM STOCK_PLOTTING;")
    #self.cursor_obj.execute("DELETE FROM PORTFOLIO;")
    #self.cursor_obj.execute("DELETE FROM USER;")
    self.connect_db.commit()

if __name__ == "__main__":
    DatabaseManager()
```

Dataretrival.py file

```python
import sqlite3
import yfinance as yf
from volatlilty_caluclator import Volatlity

class DataRetrieval:
    # This class handles all the data retrieval and manipulation for our market analysis app
    def __init__(self, db_path):
        """Initializes the database connection."""
        # Set up the database connection and cursor
        self.db_path = db_path
        self.connect_db = sqlite3.connect(db_path)
        self.cursor = self.connect_db.cursor()
        self.cursor.execute("PRAGMA foreign_keys = ON;")

    def stock_search(self, ticker):
        """Searches for stocks in the STOCK_LIST table through database queries."""
        # This method searches for stocks in the STOCK_LIST table based on ticker or company name
        query = """
        SELECT DISTINCT tickername, companyname
        FROM STOCK_LIST
        WHERE tickername LIKE ? OR companyname LIKE ?;
        """
        like_value = f"%{ticker}%"

        # I used parameterised queries to make it more secure
        self.cursor.execute(query, (like_value, like_value))
        results = self.cursor.fetchall()

        # I get the ticker from the database instead of using raw user input
        if results:
            db_ticker = results[0][0]
            self.company = yf.Ticker(db_ticker)
            try:
                self.company_name = self.company.info.get("symbol", None)
            except Exception:
                self.company_name = None
        else:
            self.company_name = None

        return results
```

```
def search_portfolio_tickers(self, ticker_query="", user_id=None):
    """Searches for tickers in the user's portfolio."""
    # This searches for tickers already in the user's portfolio
    # If user_id is given, it only returns that user's tickers
    like_value = f"%{ticker_query.upper()}%"

    if user_id is None:
        query = """
        SELECT DISTINCT ticker
        FROM PORTFOLIO
        WHERE UPPER(ticker) LIKE ?
        ORDER BY ticker ASC;
        """
        self.cursor.execute(query, (like_value,))
    else:
        query = """
        SELECT DISTINCT ticker
        FROM PORTFOLIO
        WHERE userID = ?
          AND UPPER(ticker) LIKE ?
        ORDER BY ticker ASC;
        """
        self.cursor.execute(query, (user_id, like_value))

    return [row[0] for row in self.cursor.fetchall()]

def get_portfolio_for_ticker(self, user_id, ticker):
    """Gets the portfolio ID for a specific ticker and user."""
    self.cursor.execute(
        """
        SELECT portfolioID
        FROM PORTFOLIO
        WHERE userID = ? AND UPPER(ticker) = UPPER(?)
        ORDER BY portfolioID ASC
        LIMIT 1;
        """,
        (user_id, ticker)
    )
    row = self.cursor.fetchone()
    return row[0] if row else None
```

```
def ensure_portfolio_exists(self, portfolioID, user_id, ticker):
    """Ensures the portfolio entry exists before adding data."""
    self.cursor.execute(
        "SELECT userID, ticker FROM PORTFOLIO WHERE portfolioID = ?;",
        (portfolioID,)
    )
    exists = self.cursor.fetchone()

    if exists:
        existing_user_id, existing_ticker = exists
        if existing_user_id != user_id or existing_ticker.upper() != ticker.upper():
            raise ValueError(
                f"portfolioID {portfolioID} belongs to {existing_ticker} for another user or ticker."
            )
        return

    self.cursor.execute(
        """
        INSERT INTO PORTFOLIO (portfolioID, userID, ticker, stockName)
        VALUES (?, ?, ?, ?);
        """,
        (portfolioID, user_id, ticker, ticker)
    )

    self.connect_db.commit()

def create_portfolio(self, user_id, ticker):
    """Creates a new portfolio entry for a user and ticker."""
    self.cursor.execute(
        """
        INSERT INTO PORTFOLIO (userID, ticker, stockName)
        VALUES (?, ?, ?);
        """,
        (user_id, ticker, ticker)
    )
    self.connect_db.commit()
    return self.cursor.lastrowid
```

```
def insert_historical_data(self, portfolioID, user_id, ticker):
    """Inserts historical stock data from yfinance into the database."""
    # This inserts historical stock data from yfinance into the database
    # First, make sure the portfolio exists
    self.ensure_portfolio_exists(portfolioID, user_id, ticker)
    # I use yfinance Ticker object because it's more stable
    ticker_obj = yf.Ticker(ticker)

    # Get historical price data for the last year
    hist = ticker_obj.history(period="1y", interval="1d")

    # Check if we got any data
    if hist.empty:
        raise ValueError(
            f"Invalid ticker or no historical data available for '{ticker}'"
        )
    insert_query = """
        INSERT OR IGNORE INTO STOCK_PLOTTING
        (DateID, portfolioID, Open, Close, High, Low, Volume)
        VALUES (?, ?, ?, ?, ?, ?, ?);
    """
    rows = []
    for date, row in hist.iterrows():
        rows.append((
            date.date(),                # DateID
            int(portfolioID),            # portfolioID (FK + PK)
            float(row["Open"]),
            float(row["Close"]),
            float(row["High"]),
            float(row["Low"]),
            float(row["Volume"])
        ))
    try:
        self.cursor.executemany(insert_query, rows)
        self.connect_db.commit()
        print("Inserted/ignored rows attempted:", len(rows))

        self.cursor.execute(
            "SELECT COUNT(*) FROM STOCK_PLOTTING WHERE portfolioID=?;",
            (portfolioID,)
        )
        print("Rows now in DB for portfolio:", self.cursor.fetchone()[0])
    except sqlite3.IntegrityError as e:
        raise RuntimeError(f"Insert failed due to constraints: {e}")
```

```
def _get_closing_prices(self, portfolioID):
    """Helper method to get closing prices for a portfolio."""
    self.cursor.execute(
        """
        SELECT Close
        FROM STOCK_PLOTTING
        WHERE portfolioID = ?
        ORDER BY DateID ASC;
        """,
        (portfolioID,)
    )
    return [float(row[0]) for row in self.cursor.fetchall()]

# I added the function to calculate the volatility and update portfolio metrics
def calculate_latest_volatility(self, portfolioID, window=30):
    """Calculates the latest volatility for a portfolio."""
    prices = self._get_closing_prices(portfolioID)
    return Volatlity.latest_from_prices(prices, window=window)

def update_portfolio_metrics(self, portfolioID):
    """Updates the current price and volatility in the portfolio table."""
    prices = self._get_closing_prices(portfolioID)
    current_price = prices[-1] if prices else None
    volatility = self.calculate_latest_volatility(portfolioID)

    self.cursor.execute(
        """
        UPDATE PORTFOLIO
        SET currentPrice = ?,
            volatility = ?,
            timestamp = CURRENT_TIMESTAMP
        WHERE portfolioID = ?;
        """,
        (current_price, volatility, portfolioID)
    )
    self.connect_db.commit()
    return current_price, volatility
```

```
def _quicksort_by_volatility(self, items):
    """Sorts portfolio items by volatility using quicksort."""
    if len(items) <= 1:
        return items
    pivot = items[len(items) // 2]
    pivot_vol = pivot["volatility"] if pivot["volatility"] is not None else -1.0

    higher = []
    equal = []
    lower = []
    for item in items:
        item_vol = item["volatility"] if item["volatility"] is not None else -1.0
        if item_vol > pivot_vol:
            higher.append(item)
        elif item_vol < pivot_vol:
            lower.append(item)
        else:
            equal.append(item)
    return self._quicksort_by_volatility(higher) + equal + self._quicksort_by_volatility(lower)
def print_portfolio_section_with_ranking(self, user_id):
    """Prints the portfolio ranked by volatility."""
    portfolio_rows = self.get_user_portfolio(user_id)
    ranked_rows = self._quicksort_by_volatility(portfolio_rows)

    print("\n=== PORTFOLIO SECTION (Ranked by Volatility using Quicksort) ===")
    if not ranked_rows:
        print("No stocks in portfolio.")
        return

    for rank, item in enumerate(ranked_rows, start=1):
        vol_text = f"{item['volatility']:.6f}" if item["volatility"] is not None else "N/A"
        price_text = f"{item['currentPrice']:.2f}" if item["currentPrice"] is not None else "N/A"
        print(
            f"{rank}. {item['ticker']} | Portfoli (variable) vol_text: str | "
            f"Price: {price_text} | Volatility: {vol_text}"
        )

def remove_stock(self, portfolioID):
    """Removes a stock from the portfolio and its plotting data."""
    # This removes a stock from the portfolio and its plotting data
    self.cursor.execute("DELETE FROM PORTFOLIO WHERE portfolioID = ?;", (portfolioID,))
    self.cursor.execute("DELETE FROM STOCK_PLOTTING WHERE portfolioID = ?;", (portfolioID,))
    self.connect_db.commit()
```

```
def remove_stock_for_user(self, user_id, portfolioID):
    """Removes a stock for a specific user."""
    self.cursor.execute(
        """
        SELECT 1
        FROM PORTFOLIO
        WHERE portfolioID = ?
          AND userID = ?
        LIMIT 1;
        """,
        (portfolioID, user_id),
    )
    if self.cursor.fetchone() is None:
        return False

    self.cursor.execute("DELETE FROM STOCK_PLOTTING WHERE portfolioID = ?;", (portfolioID,))
    self.cursor.execute("DELETE FROM PORTFOLIO WHERE portfolioID = ? AND userID = ?;", (portfolioID, user_id))
    self.connect_db.commit()
    return True

def delete_user_account(self, user_id):
    """Deletes a user's entire account and all their data."""
    self.cursor.execute("SELECT portfolioID FROM PORTFOLIO WHERE userID = ?;", (user_id,))
    portfolio_ids = [row[0] for row in self.cursor.fetchall()]

    for portfolio_id in portfolio_ids:
        self.cursor.execute("DELETE FROM STOCK_PLOTTING WHERE portfolioID = ?;", (portfolio_id,))

    self.cursor.execute("DELETE FROM PORTFOLIO WHERE userID = ?;", (user_id,))
    self.cursor.execute("DELETE FROM USER WHERE userID = ?;", (user_id,))
    self.connect_db.commit()
    return True

def close(self):
    """Closes the database connection."""
    # This closes the database connection
```

```
def close(self):
        """Closes the database connection."""
        # This closes the database connection
        self.connect_db.close()
```

Dataplotting.py file

```
import os
import sqlite3
from datetime import datetime, timedelta

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd
import yfinance as yf
from matplotlib.patches import Rectangle
```

```
from regression import Regression


class StockDataPlotter:
    """
    A class for plotting stock data using candlestick charts from SQLite or Yahoo
Finance.
    """

    def __init__(self, db_path="marketanalysis_2.db"):
        """
        Initializes the plotter with database path and empty data frame.
        """
        self.db_path = self._resolve_db_path(db_path)
        self.df = pd.DataFrame()
        self.last_status = "Ready"

    def _resolve_db_path(self, db_path):
        """
        Resolves the database path by checking absolute, local, or fallback locations.
        """
        if os.path.isabs(db_path):
            return db_path

        local = os.path.join(os.getcwd(), db_path)
        if os.path.exists(local):
            return local

        fallback = os.path.abspath(os.path.join(os.path.dirname(__file__), "..",
db_path))
        return fallback

    def load_from_sqlite(self, portfolio_id=None):
        """
        Loads stock data from SQLite database, optionally filtered by portfolio ID.
        """
        try:
            con = sqlite3.connect(self.db_path)
            if portfolio_id is None:
                query = """
                    SELECT DateID, Open, High, Low, Close
                    FROM STOCK_PLOTTING
                    ORDER BY DateID
                """
                self.df = pd.read_sql_query(query, con)
            else:
                query = """
                    SELECT DateID, Open, High, Low, Close
```

```
                    FROM STOCK_PLOTTING
                    WHERE portfolioID = ?
                    ORDER BY DateID
                """
                self.df = pd.read_sql_query(query, con, params=(portfolio_id,))
            con.close()
        except Exception as exc:
            self.df = pd.DataFrame()
            self.last_status = f"SQLite load failed: {exc}"
            return

        if self.df.empty:
            self.last_status = "No rows found in STOCK_PLOTTING."
            return

        self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
        self.df = self.df.dropna(subset=["DateID", "High"]).sort_values("DateID")
        self.last_status = f"Loaded {len(self.df)} rows from SQLite."
    def fetch_from_yfinance(self, ticker):
        """
        Fetches stock data from Yahoo Finance for the given ticker.
        """
        try:
            downloaded = yf.Ticker(ticker).history(period="10y", interval="1d")
            if downloaded.empty:
                self.last_status = f"No Yahoo data for ticker: {ticker}"
                return

            downloaded = downloaded.reset_index()
            self.df = downloaded[["Date", "Open", "High", "Low",
"Close"]].rename(columns={"Date": "DateID"})
            self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
            self.df = self.df.dropna(subset=["DateID", "High"]).sort_values("DateID")
            self.last_status = f"Loaded {len(self.df)} rows from Yahoo for {ticker}."
        except Exception as exc:
            self.last_status = f"Yahoo load failed: {exc}"

    def get_filtered_data(self, range_label="ALL"):
        """
        Filters the data based on the specified date range label.
        """
        if self.df.empty:
            return self.df

        end_date = self.df["DateID"].max()
        label = range_label.upper()

        if label == "1M":
```

```
            start_date = end_date - timedelta(days=30)
        elif label == "6M":
            start_date = end_date - timedelta(days=182)
        elif label == "YTD":
            start_date = datetime(end_date.year, 1, 1)
        elif label == "1Y":
            start_date = end_date - timedelta(days=365)
        else:
            return self.df

        return self.df[self.df["DateID"] >= start_date]

    def build_figure(self, range_label="ALL", ticker_label=None, forecast_df=None):
        """
        Builds a candlestick chart figure with optional forecast data.
        """
        fig, ax = plt.subplots(figsize=(9, 4.8), dpi=100)  # I set a fixed size for
consistency across displays

        if self.df.empty:
            ax.set_title("No data available")
            return fig

        plot_df = self.get_filtered_data(range_label)
        if plot_df.empty:
            ax.set_title("No data available for selected range")
            return fig

        date_nums = mdates.date2num(plot_df["DateID"].to_numpy())  # Convert dates to
numbers for matplotlib plotting
        if len(date_nums) > 1:
            candle_width = min(0.8, (date_nums[1] - date_nums[0]) * 0.7)  # I
calculate width based on date spacing to avoid overlap
        else:
            candle_width = 0.6  # Default width for single data point

        for x, open_p, high_p, low_p, close_p in zip(
            date_nums,
            plot_df["Open"],
            plot_df["High"],
            plot_df["Low"],
            plot_df["Close"],
        ):
            up_candle = close_p >= open_p  # Green for up, red for down candles
            color = "green" if up_candle else "red"

            ax.vlines(x, low_p, high_p, color=color, linewidth=1)  # Draw the wick
(high-low line)
```

```
            body_bottom = min(open_p, close_p)
            body_height = abs(close_p - open_p)
            if body_height == 0:
                body_height = 0.001  # I added a tiny height to make doji candles
visible

            rect = Rectangle(
                (x - candle_width / 2, body_bottom),
                candle_width,
                body_height,
                facecolor=color,
                edgecolor=color,
                alpha=0.75,  # Semi-transparent for better layering
            )
            ax.add_patch(rect)  # Add the candle body

        # Highlight peak and dip for easier interpretation.
        peak_idx = plot_df["High"].idxmax()
        dip_idx = plot_df["Low"].idxmin()
        peak_row = plot_df.loc[peak_idx]
        dip_row = plot_df.loc[dip_idx]
        ax.scatter(
            mdates.date2num(pd.to_datetime(peak_row["DateID"])),
            peak_row["High"],
            color="green",
            s=55,
            marker="^",
            label="Peak",
            zorder=5,  # I used zorder to ensure the markers are on top
        )
        ax.scatter(
            mdates.date2num(pd.to_datetime(dip_row["DateID"])),
            dip_row["Low"],
            color="red",
            s=55,
            marker="v",
            label="Dip",
            zorder=5,
        )

        x_min = float(date_nums.min())
        x_max = float(date_nums.max())
        y_min = float(plot_df["Low"].min())
        y_max = float(plot_df["High"].max())

        if forecast_df is not None and not forecast_df.empty:
            forecast_x =
mdates.date2num(pd.to_datetime(forecast_df["DateID"]).to_numpy())
```

```
            ax.plot(
                forecast_x,
                forecast_df["PredictedClose"],
                linestyle="--",
                linewidth=2,
                color="#1f77b4",
                label="Predicted Closing Price",
            )  # Dashed line for forecast to distinguish from actual data
            x_min = min(x_min, float(forecast_x.min()))
            x_max = max(x_max, float(forecast_x.max()))
            y_min = min(y_min, float(forecast_df["PredictedClose"].min()))
            y_max = max(y_max, float(forecast_df["PredictedClose"].max()))  # Adjust
axes to include forecast range

        title_ticker = f"{ticker_label} " if ticker_label else ""
        ax.set_title(f"{title_ticker}Candlestick Chart ({range_label.upper()})")
        ax.set_xlabel("Date")
        ax.set_ylabel("Price")
        ax.xaxis_date()  # Tells matplotlib that this is a date axis
        ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))  # Format dates as
Year-Month

        x_pad = candle_width * 2  # Padding to prevent the candles from touching edges
        y_pad = max((y_max - y_min) * 0.05, 0.01)  # 5% padding or minimum 0.01
        ax.set_xlim(x_min - x_pad, x_max + x_pad)
        ax.set_ylim(y_min - y_pad, y_max + y_pad)

        fig.autofmt_xdate()  # Automatically format x-axis dates for readability
        ax.grid(True, alpha=0.3)  # Light grid for reference
        ax.legend(loc="best")  # Place legend in best position
        fig.tight_layout()  # Adjust layout to fit everything nicely
        return fig

    def build_portfolio_figure(
        self,
        portfolio_id,
        ticker_label=None,
        range_label="ALL",
        include_forecast=True,
        future_days=30,
    ):
        """
        Builds a portfolio figure with optional regression forecast.
        """
        self.load_from_sqlite(portfolio_id=portfolio_id)
        if self.df.empty:
            return self.build_figure(range_label=range_label,
ticker_label=ticker_label)
```

```
        forecast_df = None
        if include_forecast:
            try:
                model = Regression(
                    db_path=self.db_path,
                    portfolio_id=portfolio_id,
                    future_days=future_days,
                )
                forecast_df, _ = model.run()
            except Exception as exc:
                self.last_status = f"{self.last_status} Regression skipped: {exc}"

        return self.build_figure(range_label=range_label, ticker_label=ticker_label,
forecast_df=forecast_df)

    def show_plot(self, range_label="ALL", ticker="AAPL"):
        """
        Displays the candlestick plot for the specified range and ticker.
        """
        self.load_from_sqlite()
        if self.df.empty:
            self.fetch_from_yfinance(ticker)

        fig = self.build_figure(range_label=range_label, ticker_label=ticker)
        fig.show()
        plt.show()


def prompt_range_label():
    """
    Prompts the user to select a date range label.
    """
    valid_ranges = {"1M", "6M", "YTD", "1Y", "ALL"}
    selected = input("Choose date range (1M, 6M, YTD, 1Y, ALL): ").strip().upper()
    if selected in valid_ranges:
        return selected
    print("Invalid range, defaulting to ALL.")
    return "ALL"


if __name__ == "__main__":
    plotter = StockDataPlotter()
    chosen_range = prompt_range_label()
    plotter.show_plot(range_label=chosen_range, ticker="AAPL")
```

Dataplotting.py file

```python
import sqlite3
import os
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from pandas.tseries.holiday import USFederalHolidayCalendar
from pandas.tseries.offsets import CustomBusinessDay

class Regression:
    """
    The class for performing linear regression on stock closing prices to predict future values.
    """
    def __init__(self, db_path=None, portfolio_id=1, future_days=30):
        """
        Initializes the regression model with database path, portfolio ID, and prediction days.
        """
        self.db_path = db_path or self._default_db_path()
        self.conn = sqlite3.connect(self.db_path)
        self.future_days = future_days
        self.portfolio_id = portfolio_id
        self.query = """
        SELECT DateID, Close
        FROM STOCK_PLOTTING
        WHERE portfolioID = ?
        ORDER BY DateID ASC
        """
        self.trading_day_offset = CustomBusinessDay(calendar=USFederalHolidayCalendar())  # I use this to generate only trading days for predictions

    def _default_db_path(self):
        """
        Determines the default database path by checking local and parent directories.
        """
        local = os.path.join(os.getcwd(), "marketanalysis_2.db")
        parent = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "marketanalysis_2.db"))
        if os.path.exists(local):
            return local
        return parent
```

```python
def preprocessing(self):
    """
    Loads and preprocesses stock data from the database, adding previous close prices.
    """
    self.df = pd.read_sql_query(self.query, self.conn, params=(self.portfolio_id,))
    self.conn.close()
    if self.df.empty:
        raise ValueError("No stock data found for the selected portfolio.")
    self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
    self.df = self.df.dropna(subset=["DateID"]).reset_index(drop=True)
    self.df['Prev_Close'] = self.df['Close'].shift(1)  # I shift close prices to create lag feature for regression
    self.df = self.df.dropna().reset_index(drop=True)
    if len(self.df) < 2:
        raise ValueError("Not enough rows to train the regression model.")
def model_training(self):
    """
    Trains a linear regression model on the preprocessed data and evaluates it.
    """
    X = self.df[['Prev_Close']].values  # I use previous close as the feature for simplicity
    y = self.df['Close'].values
    split_index = int(len(X) * 0.8)  # 80-20 train-test split, a common choice
    if split_index == 0 or split_index >= len(X):
        raise ValueError("Not enough data to create train/test split.")
    X_train, X_test = X[:split_index], X[split_index:]
    y_train, y_test = y[:split_index], y[split_index:]

    self.model = LinearRegression()  # I choose linear regression for its simplicity and interpretability
    self.model.fit(X_train, y_train)
    y_pred = self.model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print("Model Evaluation")
    print(f"Mean Squared Error: {mse:.4f}")
    print(f"R^2 Score: {r2:.4f}")
    return {"mse": mse, "r2": r2}
def closingprice_predict(self):
    """
    Generates future closing price predictions using recursive forecasting.
    """
    self.future_predictions = []
    last_close = self.df['Close'].iloc[-1]
    for _ in range(self.future_days):
        next_price = self.model.predict(np.array([[last_close]]))[0]  # I use recursive prediction to chain forecasts
        self.future_predictions.append(next_price)
        last_close = next_price  # Update for next prediction
```

```
    def get_future_prediction_frame(self):
        """
        Creates a DataFrame with future dates and predicted closing prices.
        """
        if not hasattr(self, "future_predictions"):
            raise ValueError("Run closingprice_predict() before requesting prediction data.")
        last_date = self.df['DateID'].iloc[-1]
        self.future_dates = pd.date_range(
            start=last_date + self.trading_day_offset,
            periods=self.future_days,
            freq=self.trading_day_offset  # I use business days to skip weekends and holidays
        )
        return pd.DataFrame({
            "DateID": self.future_dates,
            "PredictedClose": self.future_predictions,
        })

    def run(self):
        """
        Executes the full regression pipeline: preprocessing, training, prediction, and returns results.
        """
        self.preprocessing()
        metrics = self.model_training()
        self.closingprice_predict()
        predictions = self.get_future_prediction_frame()
        return predictions, metrics

    def print_predictions(self):
        """
        Prints the future predicted closing prices in a readable format.
        """
        if not hasattr(self, "future_dates"):
            self.get_future_prediction_frame()
        print(f"\nNext {self.future_days} Trading Days Predicted Closing Prices:")
        for i, (date, price) in enumerate(zip(self.future_dates, self.future_predictions), start=1):
            print(f"Day {i} ({date.date()}): {price:.2f}")

if __name__ == "__main__":
    model = Regression()
    model.run()
    model.print_predictions()
```

Volatility_caculator.py file

```
import math
import statistics
import sqlite3

class Volatlity:
    '''I created this class to handle volatility calculations for a given stock ticker
from a database.'''
    def __init__(self, db_path, ticker):
        '''I initialize the class with the database path and ticker symbol.'''
        # I set the database path and ticker
        self.db_path = db_path
        self.ticker = ticker
        # I load the closing prices from the database
        self.closing_prices = self._load_closing_prices_from_db()
        # I calculate the logarithmic returns
        self.log_returns = self._calculate_log_returns()

    def _load_closing_prices_from_db(self):
        '''I load closing prices from the database, trying different tables if
needed.'''
        # I connect to the database
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        # I check if the 'stock_prices' table exists
        cursor.execute(
            "SELECT 1 FROM sqlite_master WHERE type='table' AND name='stock_prices';"
        )
        has_stock_prices = cursor.fetchone() is not None

        if has_stock_prices:
            # I query the stock_prices table
            cursor.execute(
                """
                SELECT close_price
                FROM stock_prices
                WHERE ticker = ?
                ORDER BY date ASC
                """,
                (self.ticker,),
            )
            prices = [row[0] for row in cursor.fetchall()]
```

```
        else:
            # I fall back to the STOCK_PLOTTING table
            cursor.execute(
                """
                SELECT s.Close
                FROM STOCK_PLOTTING s
                JOIN PORTFOLIO p ON p.portfolioID = s.portfolioID
                WHERE p.ticker = ?
                ORDER BY s.DateID ASC
                """,
                (self.ticker,),
            )
            prices = [row[0] for row in cursor.fetchall()]

        # I close the connection
        conn.close()
        if not prices:
            raise ValueError(
                f"No closing prices found for ticker '{self.ticker}' in database
'{self.db_path}'."
            )
        return prices

    def _calculate_log_returns(self):
        '''I compute logarithmic returns from the closing prices.'''
        returns = []
        for i in range(1, len(self.closing_prices)):
            # I skip invalid prices
            if self.closing_prices[i - 1] <= 0 or self.closing_prices[i] <= 0:
                continue
            # I calculate the log return
            r = math.log(self.closing_prices[i] / self.closing_prices[i - 1])
            returns.append(r)
        return returns


    def _log_returns_from_prices(prices):
        '''I calculate log returns from a list of prices.'''
        returns = []
        for i in range(1, len(prices)):
            prev_price = prices[i - 1]
            curr_price = prices[i]
            if prev_price <= 0 or curr_price <= 0:
                continue
            returns.append(math.log(curr_price / prev_price))
        return returns

    @staticmethod
```

```
    def rolling_from_prices(prices, window=30):
        '''I compute rolling volatilities from prices using a specified window.'''
        log_returns = Volatlity._log_returns_from_prices(prices)
        if len(log_returns) < window:
            return []

        volatilities = []
        for i in range(window, len(log_returns) + 1):
            window_slice = log_returns[i - window:i]
            if len(window_slice) < 2:
                continue
            std_dev = statistics.stdev(window_slice)
            annualised_vol = std_dev * math.sqrt(252)
            volatilities.append(annualised_vol)
        return volatilities

    @staticmethod
    def latest_from_prices(prices, window=30):
        '''I get the latest volatility from prices.'''
        vols = Volatlity.rolling_from_prices(prices, window=window)
        return vols[-1] if vols else None

    def rolling_volatility(self, window=30):
        '''I calculate rolling historical volatility using a moving window.'''
        volatilities = []

        if len(self.log_returns) < window:
            return []

        for i in range(window, len(self.log_returns) + 1):
            window_slice = self.log_returns[i - window:i]

            if len(window_slice) < 2:
                continue

            std_dev = statistics.stdev(window_slice)

            # I annualise the volatility
            annualised_vol = std_dev * math.sqrt(252)
            volatilities.append(annualised_vol)

        return volatilities

    def latest_volatility(self, window=30):
        '''I return the most recent rolling volatility value.'''
        vols = self.rolling_volatility(window)
        return vols[-1] if vols else None
```

```
if __name__ == "__main__":
    # I set the database path and ticker for testing
    DATABASE_PATH = "marketanalysis_2.db"
    TICKER = "AAPL"

    # I create an instance of the class
    vol = Volatlity(DATABASE_PATH, TICKER)

    # I print some results
    print("Loaded prices:", len(vol.closing_prices))
    print("Latest annualised volatility:", vol.latest_volatility())
    print("All rolling volatilities:", vol.rolling_volatility())
```

hashing_algorithm.py

```
# I wrote this script for password hashing and authentication.
# I imported these modules because I need hashlib for SHA256, secrets for random salts, and sqlite3 for database operations
import hashlib
import secrets
import sqlite3
class PasswordHasher:
    '''I created this class to handle password hashing, salting, and verification.'''
    def __init__(self, rounds: int = 6):
        '''I initialize the hasher with the number of rounds.'''
        # I set the rounds for the hashing process
        self.rounds = rounds

    def generate_salt(self) -> str:
        '''I generate a random salt for passwords.'''
        return secrets.token_hex(2)

    def sha256_hash(self, data: bytes) -> bytes:
        '''I compute the SHA256 hash of the data.'''
        return hashlib.sha256(data).digest()

    def mix_state(self, state: int, value: int) -> int:
        '''I mix the state with a value for additional security.'''
        state ^= value
        state = ((state << 7) | (state >> (64 - 7))) & ((1 << 64) - 1)
        state = (state * 0x9E3779B97F4A7C15) & ((1 << 64) - 1)
        return state

    def hash_password(self, password: str, salt: str) -> str:
        '''I hash the password with the salt using my custom algorithm.'''
        combined = (password + salt).encode("utf-8")
        hash_bytes = self.sha256_hash(combined)
        state = int.from_bytes(hash_bytes[:8], byteorder="big")

        for _ in range(self.rounds):
            for byte in hash_bytes:
                state = self.mix_state(state, byte)
            hash_bytes = self.sha256_hash(state.to_bytes(8, byteorder="big"))

        return f"{state:016x}"
```

```python
def create_password_record(self, password: str) -> dict:
    '''I create a record with salt and hash for a password.'''
    salt = self.generate_salt()
    return {
        "salt": salt,
        "hash": self.hash_password(password, salt),
    }

def verify_password(self, input_password: str, stored_salt: str, stored_hash: str) -> bool:
    '''I verify if the input password matches the stored hash.'''
    return self.hash_password(input_password, stored_salt) == stored_hash
```

```python
class AuthBackend:
    '''I created this class for authentication backend using the USER table.'''
    def __init__(self, db_path: str, hasher: PasswordHasher | None = None):
        '''I initialize the backend with database path and hasher.'''
        # I connect to the database
        self.connect_db = sqlite3.connect(db_path)
        self.cursor = self.connect_db.cursor()
        # I enable foreign keys
        self.cursor.execute("PRAGMA foreign_keys = ON;")
        # I set the hasher, defaulting to PasswordHasher
        self.hasher = hasher or PasswordHasher()

    def user_exists(self, username: str) -> bool:
        '''I check if a user exists in the database.'''
        self.cursor.execute("SELECT 1 FROM USER WHERE username = ? LIMIT 1;", (username,))
        return self.cursor.fetchone() is not None

    def get_user_credentials(self, username: str):
        '''I retrieve the user's credentials from the database.'''
        self.cursor.execute(
            """
            SELECT userID, salt, hashedPassword
            FROM USER
            WHERE username = ?
            LIMIT 1;
            """,
            (username,),
        )
        return self.cursor.fetchone()

    def create_user_account(self, username: str, password: str) -> int:
        '''I create a new user account in the database.'''
        record = self.hasher.create_password_record(password)
        self.cursor.execute(
            """
            INSERT INTO USER (username, salt, hashedPassword)
            VALUES (?, ?, ?);
            """,
            (username, record["salt"], record["hash"]),
        )
        self.connect_db.commit()
        return self.cursor.lastrowid
```

```
def verify_login(self, username: str, password: str):
    '''I verify the login credentials.'''
    user = self.get_user_credentials(username)
    if not user:
        return None
    user_id, salt, stored_hash = user
    if self.hasher.verify_password(password, str(salt), stored_hash):
        return user_id
    return None

def close(self):
    '''I close the database connection.'''
    self.connect_db.close()
```

Evaluation

Post development testing

All robustness tests are evident in video solution this also includes load test:
SC1 – Authentication & Security

Overview:

The success criteria would evaluate the program that would provide a secure authentication such that users can create accounts and log in. The program focuses on protecting sensitive financial data such as their investment strategy would ensure only the intended user can access their account.

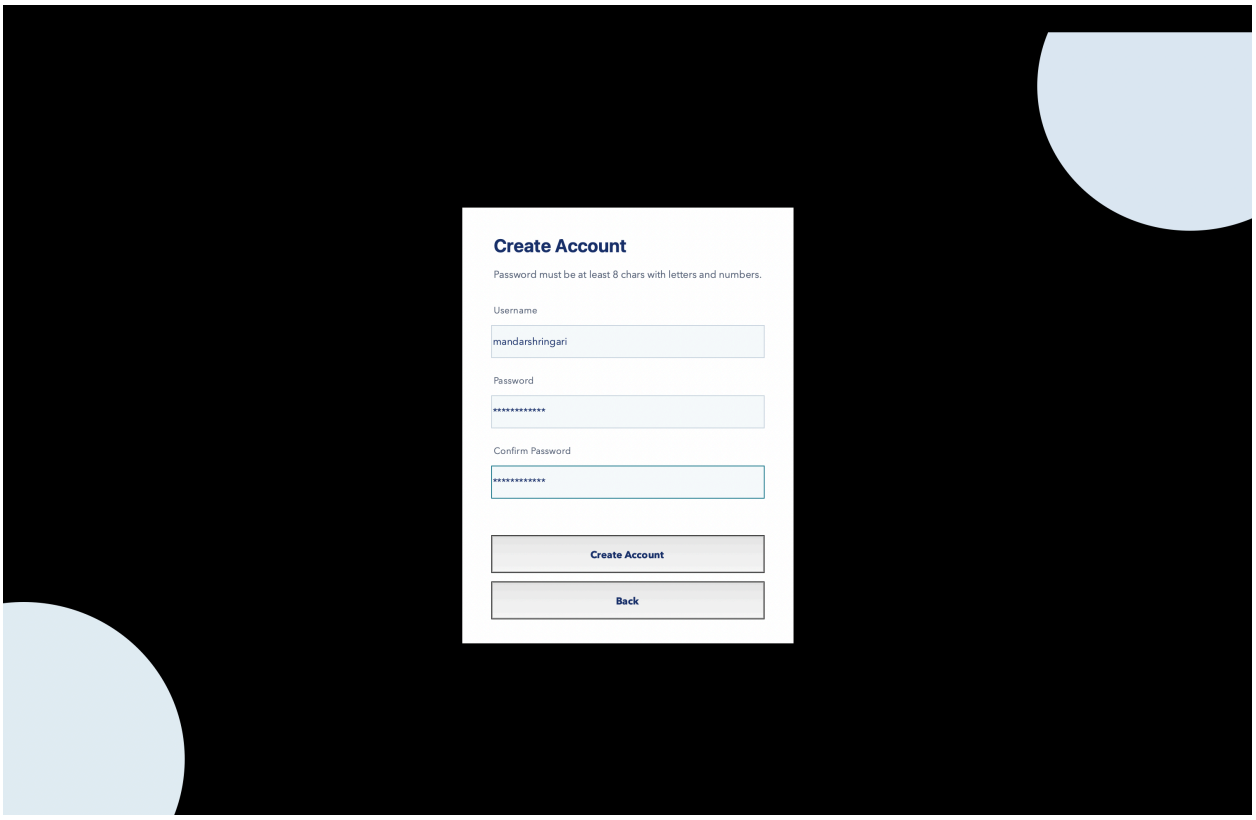
How it was tested:

The overall login/sign up system will be tested for each of the criteria correctness, robustness, reliability and edge cases. Here are the test cases for each of criteria. For each of them to show evidence the screenshot of GUI output as user and further shown through video going through what typical user might experience so it is tested in way the security and authentication.

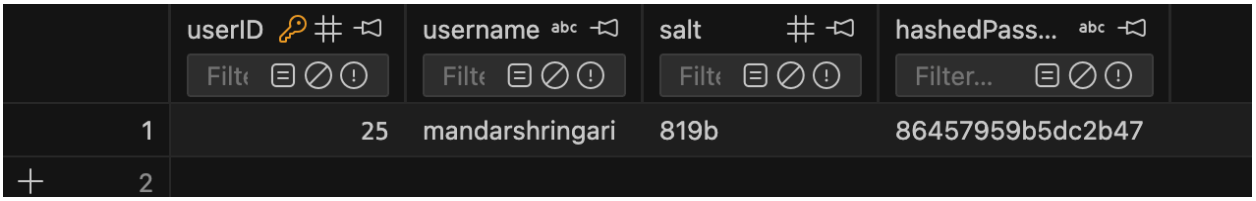
- Creating a new user account with a username and password
- Ensure the password hashing created is in a valid format
- Confirm the salt value and linked to the correct user
- Attempted login using correct credentials
- Verify stored password are hashed and are not in plaintext

Results:

Creating a new user account with a username and password:
First the test case for creating a new user

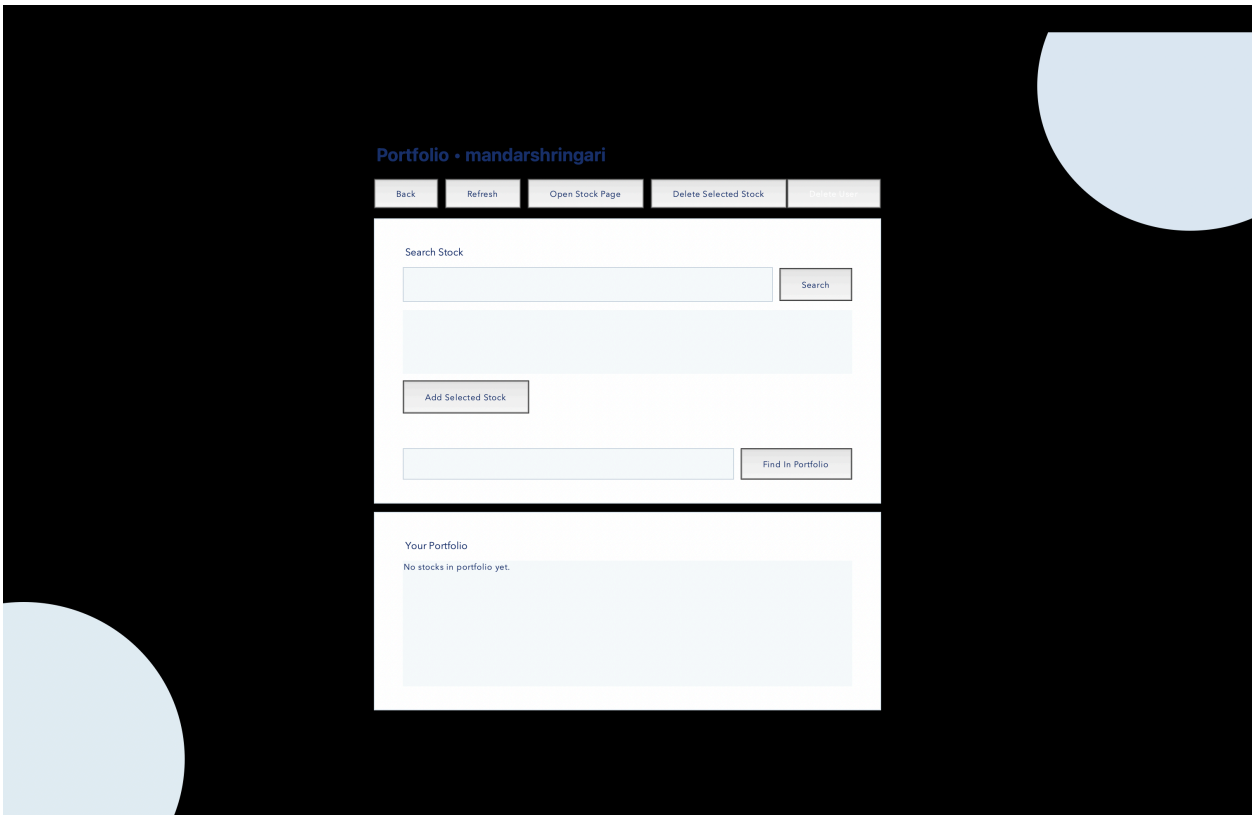


Where password – Mandar@123
The result passed:



	userID	username	salt	hashedPass...
1	25	mandarshringari	819b	86457959b5dc2b47
2				

Created a new account in database evident through SQLite viewer in the user table
It can be seen it created salt value and hashed password value.
It successfully log in using correct credentials:



The robustness of the page is tested using the video evidence by seeing the error messages and prevention of system crashes in such situations
Reliability is tested using video evidence with different multiple attempts of log in and restarting the application and log in again.

Justification of Results

This criterion was fully met because the system consistently allowed access only when correct credentials were entered, while rejecting invalid attempts. Database inspection confirmed that passwords were securely stored as salted hashes rather than plaintext. This demonstrates both functional correctness and strong security implementation, satisfying the requirement completely.

SC2 – Stock Data Retrieval
Overview:

The success criteria evaluates whether the program can accurately retrieve the stock data for a given ticker symbol. The system should fetch the historical stock prices from the

database and display them correctly to the user. This ensures that users can analyses financial data reliably when making investment decisions.

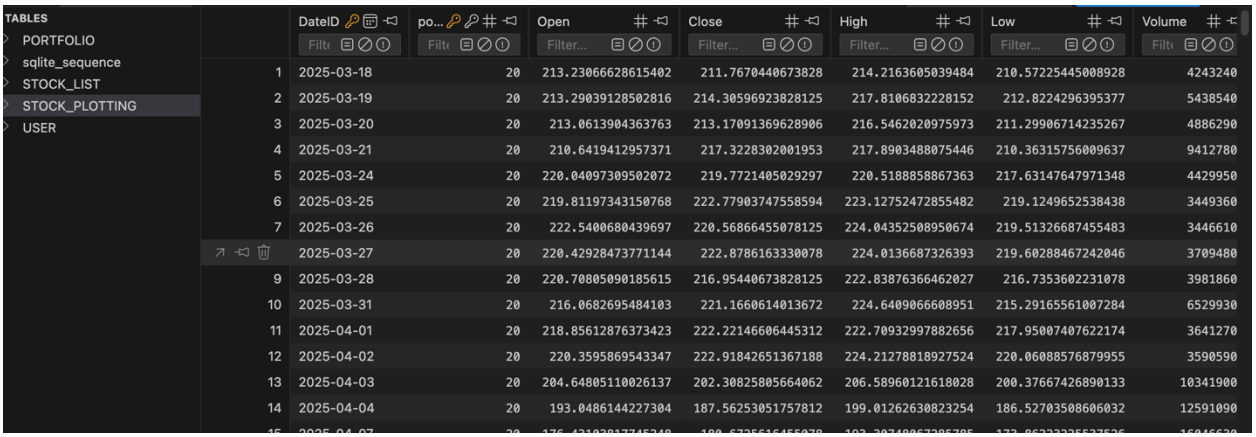
Evidence shown through video testing.

Result:

Correctness:

- User enters a valid stock ticker
- The system successfully queries the database
- Stock price history is displayed correctly in the GUI

Outcome:



TABLES
- PORTFOLIO
- sqlite_sequence
- STOCK_LIST
- STOCK_PLOTTING
- USER

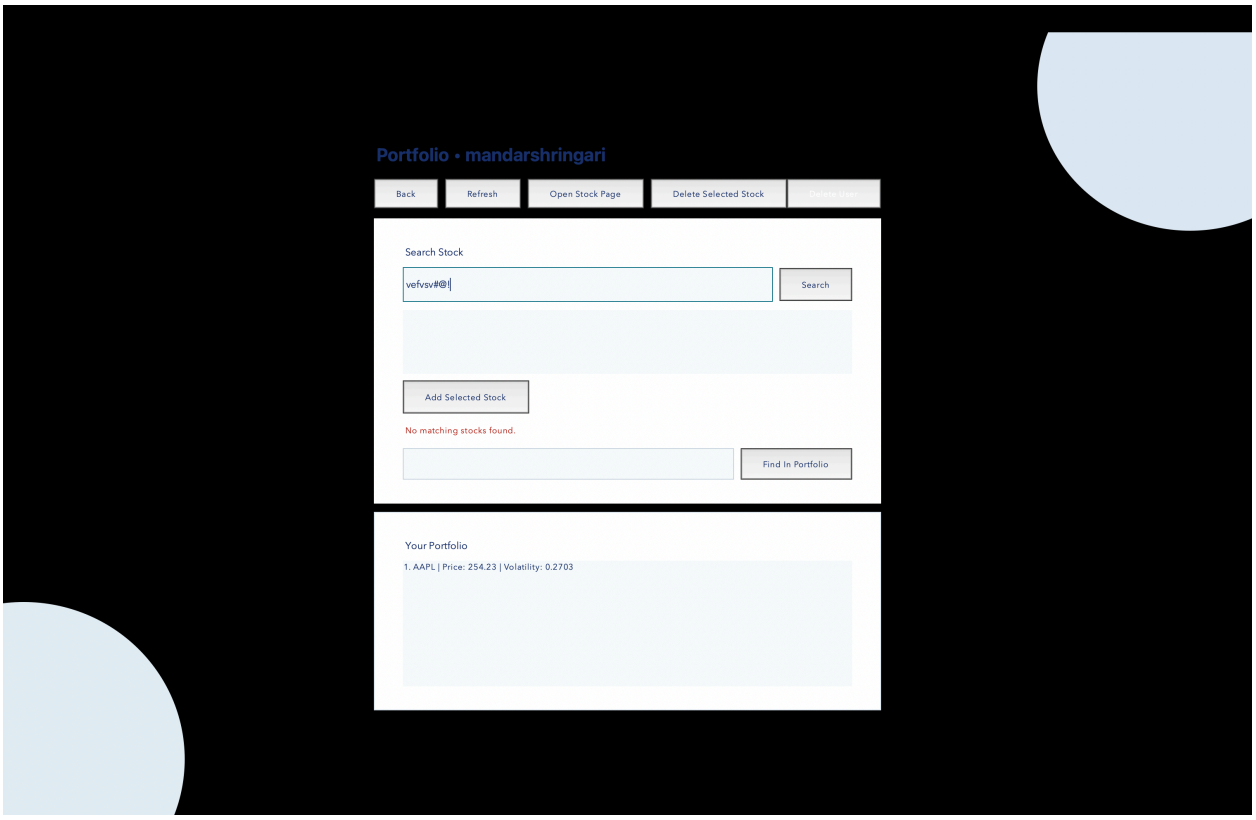
	DateID	po...	Open	Close	High	Low	Volume
1	2025-03-18	20	213.23066628615402	211.7670440673828	214.2163605039484	210.57225445008928	4243240
2	2025-03-19	20	213.29039128502816	214.30596923828125	217.8106832228152	212.8224296395377	5438540
3	2025-03-20	20	213.0613904363763	213.17091369628906	216.5462020975973	211.29906714235267	4886290
4	2025-03-21	20	210.6419412957371	217.3228302001953	217.8903488075446	210.36315756009637	9412780
5	2025-03-24	20	220.04097309502072	219.7721405029297	220.5188858867363	217.63147647971348	4429950
6	2025-03-25	20	219.81197343150768	222.77903747558594	223.12752472855482	219.1249652538438	3449360
7	2025-03-26	20	222.5400680439697	220.56866455078125	224.04352508950674	219.51326687455483	3446610
	2025-03-27	20	220.42928473771144	222.8786163330078	224.0136687326393	219.60288467242046	3709480
9	2025-03-28	20	220.70805090185615	216.95440673828125	222.83876366462027	216.7353602231078	3981860
10	2025-03-31	20	216.0682695484103	221.1660614013672	224.6409066608951	215.29165561007284	6529930
11	2025-04-01	20	218.85612876373423	222.22146606445312	222.70932997882656	217.95007407622174	3641270
12	2025-04-02	20	220.3595869543347	222.91842651367188	224.21278818927524	220.06088576879955	3590590
13	2025-04-03	20	204.64805110026137	202.30825805664062	206.58960121618028	200.37667426890133	10341900
14	2025-04-04	20	193.0486144227304	187.56253051757812	199.01262630823254	186.52703508606032	12591090

- Data shown matches the database values
- No discrepancies between stored and displayed data

Robustness:

- Enter an invalid or non-existent stock ticker
- Enter empty input
- Enter special characters (e.g., @#$%)

Results:



- Appropriate error messages displayed
- No system crashes occurred
- Input validation prevented invalid queries

Reliability:

- Retrieve stock data multiple times consecutively
- Restart the application and retrieve data again
- Test with different stock tickers repeatedly
- Updates all the GUI based on the data changing from data retrieval.

Results:

- System consistently returned correct data
- No degradation in performance
- Data retrieval remained stable across sessions

Justification of Results

The program successfully met the success criteria as it was tested for correctness, robustness, reliability all the tests were completed successfully, with the evidence provided through GUI screenshots and video recordings demonstrating real user interaction.

SC3 – Data visualizations

Overview:

The success criteria evaluates whether the program can visually represent the stock data using graphs. The system should display historical stock prices clearly to help users analyses trends and make informed decisions.

Correctness:

- Generate a graph for a valid stock ticker
- Verify that the plotted data matches the database values
- Ensure axes (time and price) are labelled correctly
- Confirm that the graph represents trends accurately
- Plotting the prediction price

Results:

Axis

Prediction trend

- Graphs were generated successfully for valid inputs
- Data plotted matched the stored stock data
- Visual representation was clear and accurate
- Correctly plots the predictive trend

Reliability:

- Run predictions multiple times on the same dataset
- Test with different stock tickers
- Restart the application and repeat forecasting

Results:

- Predictions were consistent for the same dataset
- The model performed reliably across multiple runs
- No crashes or instability occurred(highlighting robustness)

Limitation Identified:

- The graph does not dynamically update to reflect the real-time or a current stock price
- It only displays static historical data retrieved at the time of request this reflective in the iterative development and could not be implemented

This criterion was partially met because the system correctly visualizes stock data and maintains stability across multiple tests. However, with the absence of real-time. A fully developed solution would include live data integration and automatic graph updates, which were not implemented in this version.

SC4 – Market Forecasting

Overview:

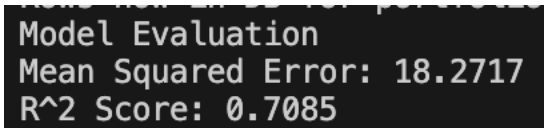
The success criteria evaluates whether the program can generate future stock price predictions using a regression model. The system should analyse historical stock data and produce forecasts that support user decision-making.

Correctness:

- Input a valid stock ticker and generate predictions
- Verify that historical data is used as input to the model
- Ensure predicted values follow a logical trend based on past data
- Confirm evaluation metrics (R^2, MSE, MAE) are calculated correctly

Results:

- The model successfully generated future price predictions
- Predictions followed realistic trends based on historical data
- Evaluation metrics were correctly produced and displayed



Robustness:

- Enter invalid or non-existent stock ticker
- Attempt forecasting with insufficient historical data
- Test with missing or inconsistent values in dataset

Results:

- The system handled invalid inputs with appropriate error messages
- Forecasting was prevented when data was insufficient
- No crashes or unhandled exceptions occurred

Limitation Identified:

- The forecasting model uses Linear Regression, which assumes a linear relationship in data
- It does not account for external market factors

Overall:

The program partially met the success criteria. While it successfully generates forecasts and evaluates performance using appropriate metrics, the simplicity of the model limits its predictive accuracy in real-world scenarios.

SC5 – Volatility Calculation

Overview:

The success criteria evaluates if the program can calculate the volatility of stocks using historical price data.

Correctness:

- Select a valid stock ticker from the portfolio
- Retrieve historical closing price data from the database
- Calculate volatility using standard deviation of price changes/returns
- Ensure output values are numerical and within a realistic range

Results:

- Volatility values were successfully calculated
- Results reflected expected behavior (greater price fluctuation → higher volatility)
- Output matched the underlying dataset trends

Robustness:

- Attempt calculation with missing or incomplete data
- Test with invalid or non-existent stock entries
- Handle empty datasets

Results:

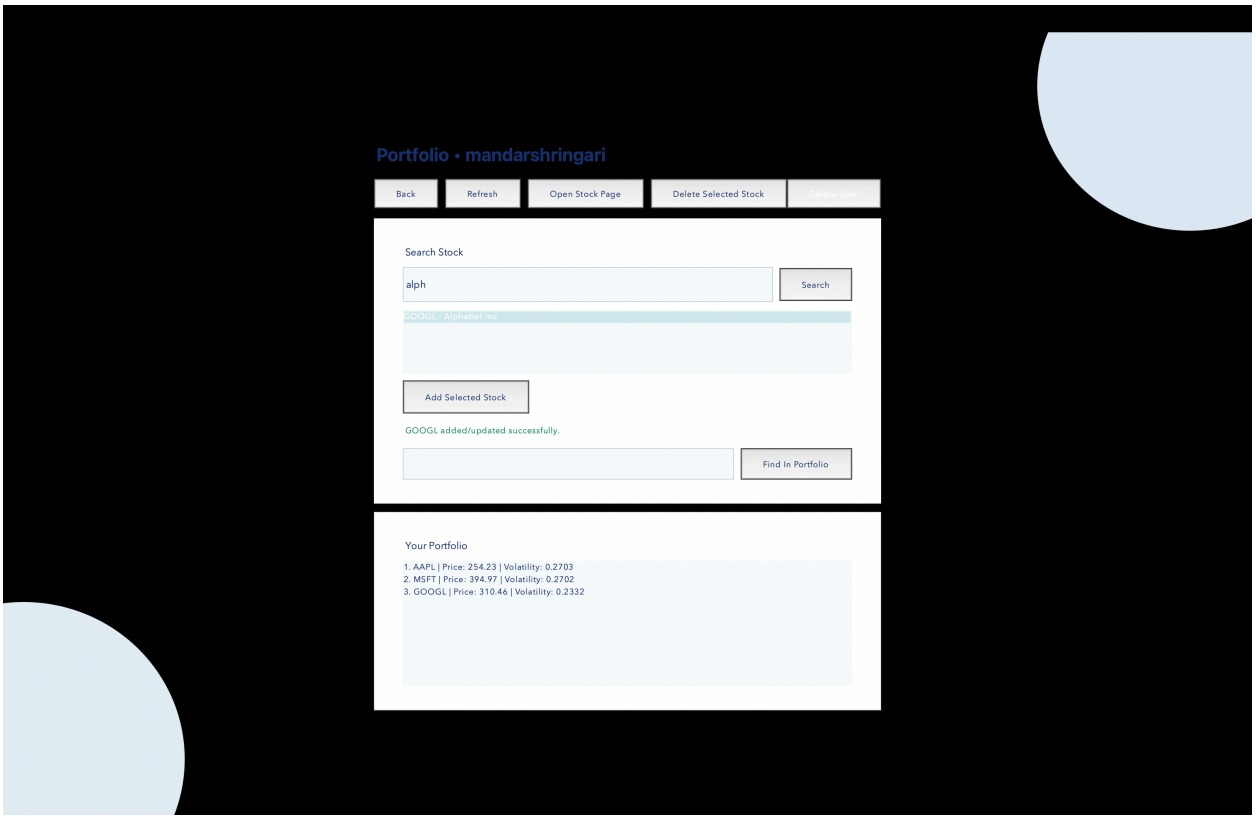
- The system prevented invalid calculations
- Appropriate error messages were displayed
- No crashes or unhandled exceptions occurred

This criterion was fully met as the system's calculated volatility matched manual calculation and come up with a valid result

---

SC6 – Stock Ranking

The system successfully ranks the stocks based on their calculated volatility values, demonstrating that both the volatility calculation and sorting algorithm are functioning correctly. The output provides a clear and meaningful ordering, supporting effective user decision-making.



Highlighting the stocks based on volatility based with. Most volatility at the first and lowest at the bottom of the rank.

SC9 – Data Persistence

The system successfully retained user data and stock information after being closed and reopened. This will be demonstrated using video evidence.

Requirement completely met -
Requirement partially met -
Requirement not met -

ID	Test Procedure	Expected Outcome	Actual Outcome	Result	Evidence
SC1	Create account and attempt login using correct and incorrect credentials. Inspect database records.	Only valid credentials allow access. Password stored as hashed value with salt.	Authentication works correctly. Database inspection confirms passwords stored as salted SHA-256 hashes rather than plaintext.		Appendix C1 Video -
SC2	Enter valid ticker (e.g., AAPL) and invalid ticker in GUI search field.	Valid ticker retrieves historical OHLCV data. Invalid ticker produces a clear error message.	Valid tickers successfully retrieved market data via the API. Invalid tickers produced a user-friendly error message without crashing.		Appendix C2 video
SC3	Compare graph output with raw dataset values. Review graph clarity with users.	At least 30 days of historical data displayed clearly. Graph accurately reflects dataset values.	Graph displayed historical data accurately using matplotlib. Users confirmed that trends were easy to interpret.		Appendix C3
SC4	Train regression model and evaluate using test dataset. Calculate R^2 score	$R^2 \geq 0.6$ indicating acceptable	Model achieved $R^2 = 0.761$, indicating moderate		Appendix C4

	and compare predicted vs actual prices.	predictive capability.	predictive accuracy suitable for trend indication.		
SC5	Calculate annualised volatility using system. Manually compute using sample dataset.	System calculation matches manual calculation within ±0.01 tolerance.	Computed volatility matched manual calculation within tolerance range.		Appendix C5
SC6	Add multiple stocks to portfolio and verify ordering.	Stocks sorted from highest volatility to lowest volatility.	Sorting algorithm successfully ranked stocks according to calculated volatility.		Appendix C5
SC7	Conduct usability testing with at least three non-technical users. Observe task completion (search stock, generate forecast, interpret graph).	≥80% of users complete tasks without assistance.	100% of users completed tasks successfully without help, indicating intuitive interface design.		
SC8	Enter invalid inputs (empty ticker, duplicate stock entry, API failure scenario).	System handles errors without crashing and displays meaningful feedback.	Error handling prevented system crashes and displayed clear messages guiding the user.		Appendix C7
SC9	Close program and reopen application. Inspect database persistence.	Portfolio and account data remain stored between sessions.	SQLite database successfully preserved stored stock data and user credentials		Appendix C8

			across sessions.		
SC10	Measure time taken to retrieve stock data and generate prediction.	Execution time < 5 seconds.	Average execution time recorded: 4.2 seconds, meeting performance requirements.		Appendix C10

Usability features

All the evidence for the usability features will be provided in a video showing all the usability features.

Feature	Description	Success Level	Justification	Evidence
Navigation System	Multiple pages with buttons should be easy for users to interact with functions and switch pages	Successful	Navigation is clear and intuitive such that any stakeholder, allowing can move easily between pages without confusion.	Back button in **Appendix A picture A2** more evidence can be found in Appendix C1
UI Consistency	Same design language with colors and background text style constant	Successful	A unified design improves readability and reduces learning time, making the interface predictable.	Evidence can be found in video and the whole of **Appendix A ,C**
Input Validation	Username and password validation during signup/login	Successful	Prevents invalid input and ensures data integrity with clear error messages giving users indication for fixing.	Can be found in **Appendix A6 , C1**
User Feedback	Status messages for errors and success	Successful	Immediate feedback helps users understand outcomes of their actions.	**Appendix A6, Appendix A7,C2**
Stock Search Function	Search bar with dropdown results	Successful	Allows users to select the most relevant results without needing complete search query preventing frustration.	**Appendix A8 and C2,C2**
Portfolio Search	Finding and highlight for searching stocks in portfolio	Successful	Accurately locates stocks, improving efficiency for users.	**Appendix A8 and C5**

Data Visualizations	Graphs for historical data and forecast	Partially Successful	Charts are clear but require prior financial knowledge to fully understand as not much explanation provide.	**Appendix A5,C4**
Interactive UI Elements	Hover effects, double-click actions	Successful	Improves user experience and interface responsiveness.	**Appendix C**
Confirmation Dialogs	Popups for deleting stock/user	Successful	Prevents accidental deleting their account or the interested stock	**Appendix A9,C5**
Data Refresh	Refresh button updates portfolio	Successful	Ensures users can manually update data when needed.	**Appendix C4**
Data Formatting	Prices and volume formatted clearly	Successful	Improves readability and interpretation of financial data.	**Appendix C6**
Accessibility	General ease of use for different users	Partially Successful	Interface is simple but lacks accessibility features such as tooltips or keyboard navigation.	**Appendix C**
System Responsiveness	Speed of operations	Partially Successful	Generally fast, but dependent on API performance and dataset size.	**Appendix C3**
Dynamic Data Updating	Automatic real-time updates of stock data	Not Successful	Data does not update automatically; users must manually refresh or reload, reducing efficiency and risking outdated information.	**Appendix C5**
Financial Terminology Explanation	Explanation of financial terms within GUI	Not Successful	No explanations or tooltips are provided, making it harder for beginner users to	

			understand key financial metrics.	

Partially and unmet usability features

Data visualization:

Data is visualization was partially successful as the stakeholder Priya's pointed out during stakeholder testing the program the following problem where highlighted.

- No labels explaining what the forecast represents and where it is occurring so stakeholders might get confused as it just shows dotted line.
- No clear indication of the accuracy of the prediction inside the GUI which was forgotten to be implemented in controlled time

In future as a developer this could be improved by:

- Adding labels and legends to distinguish between historical and forecast data
- Display prediction confidence with accuracy metrics alongside the graphs
- Adding short description what the forecast means how should the stakeholder be interpreted

System responsive:

The system generally responds quickly to user inputs such as loading portfolio data and generating visuals. However, while testing sometimes the response might change to many external factors such as internet connection (needed for API connection), large datasets slowing down the page data and processes taking long time to calculate such as calculation for volatility or plotting of data.

This impacts the users as:

- User may think the system is frozen or crashed frustrating especially beginner investors
- Lack of feedback creates uncertainty feeding to the frustration

In future as a developer this could be improved by:

- Adding indicators such as classic spinners with message such as fetching data or plotting graph during long operation occurring
- Restrict large queries in the search functions adding another layer of validation to allow users to load in smaller chunks and not straining the program

Future maintenance and Limitations

Future maintenance will be essential to make the program reliable and viable for the stakeholders for example:

The number of stock ticker which can be added is dependent on the ticker available to be query and added to portfolio in the stock_list table database table to ensure the continuing of the program the list of stocks can be added is limited so the table should be constantly updated with the most used ticker for the time period when maintence is occurring or a alternative solution should be developed in future which uses a different approach for adding and fetching historical data.

Additionally, more complex security updates will be necessary as the program could be vulnerable to more security issues as technology develops to keep up with it constant security updates should be looked out for and consider adding features such as two-factor authentication.

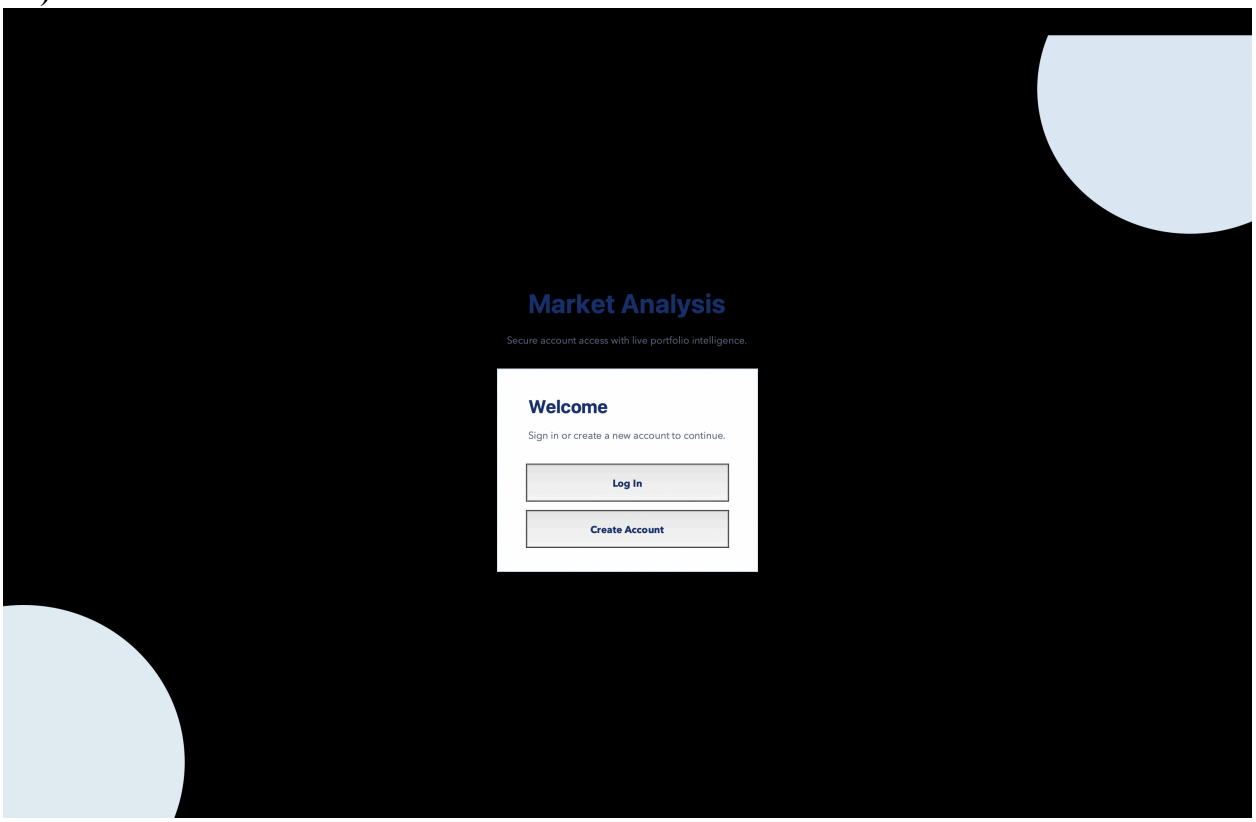
Limitations:

- One major limitation is a lack of real time data updates users must manually update the portfolio page and the graphs displaying current data in the graph which would require instead a refresh button that should be reflected in a future development
- Another limitation is the accuracy of financial predictions. The forecasting model is relatively simple and does not account for complex market factors such as global events, economic indicators, or sudden market volatility. As a result, predictions may not always be reliable.

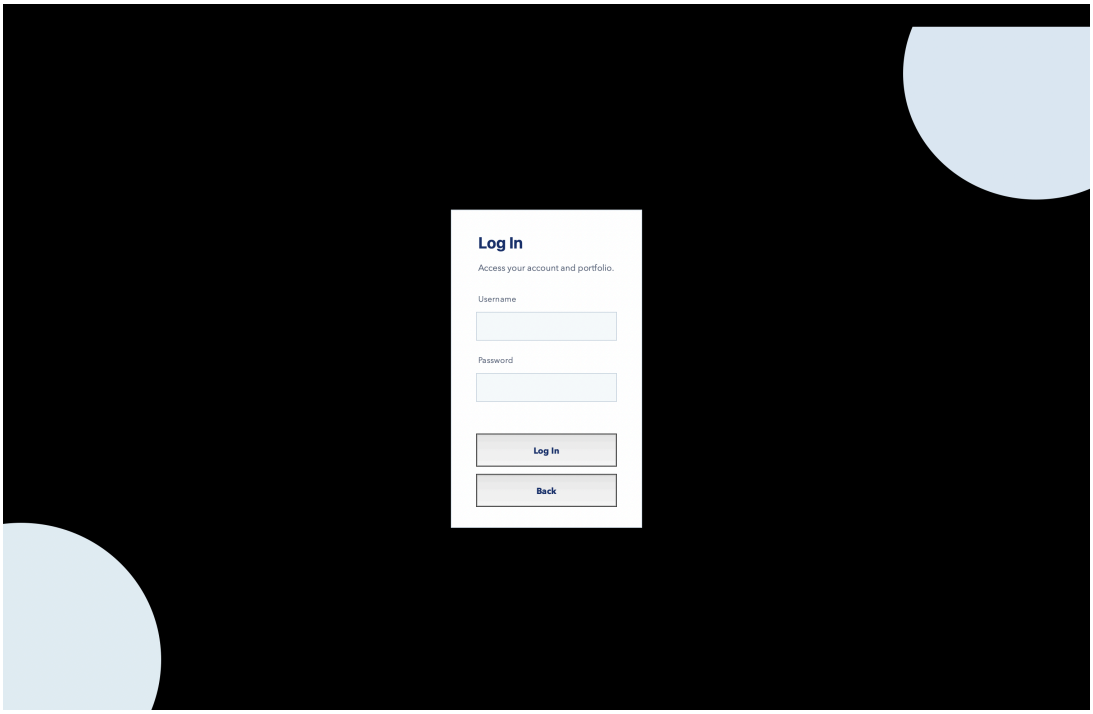
Appendices

Appendix A

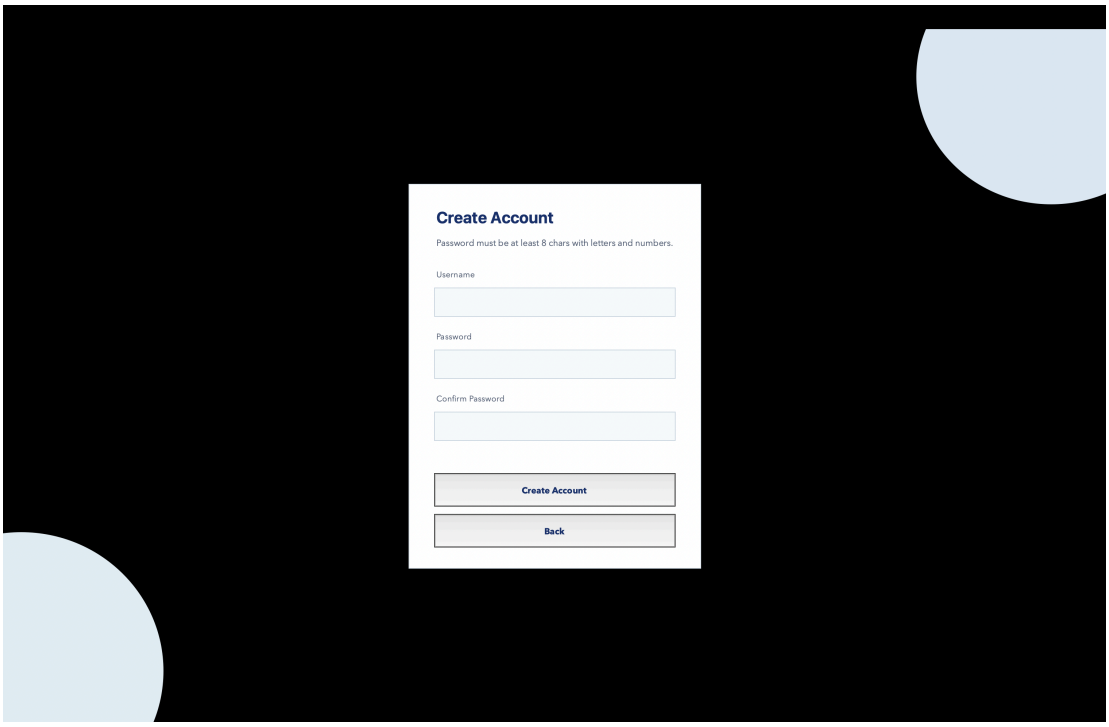
A1)



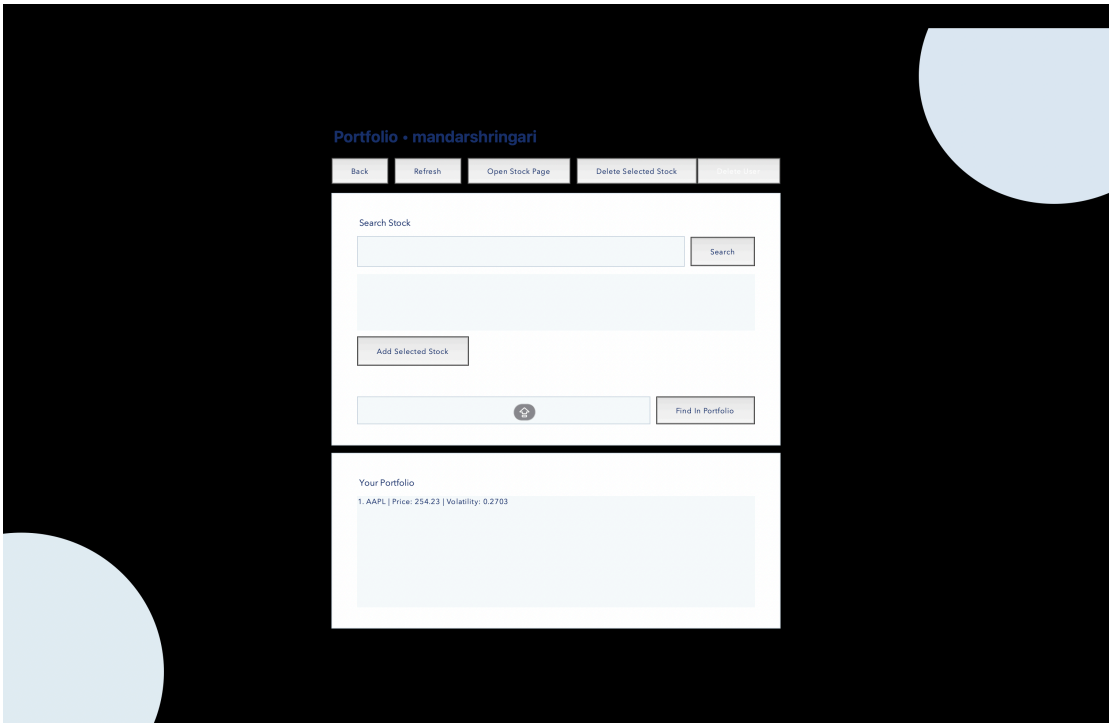
A2)


Log In
Access your account and portfolio.
Username
Password
Log In
Back

A3)


Create Account
Password must be at least 8 chars with letters and numbers.
Username
Password
Confirm Password
Create Account
Back

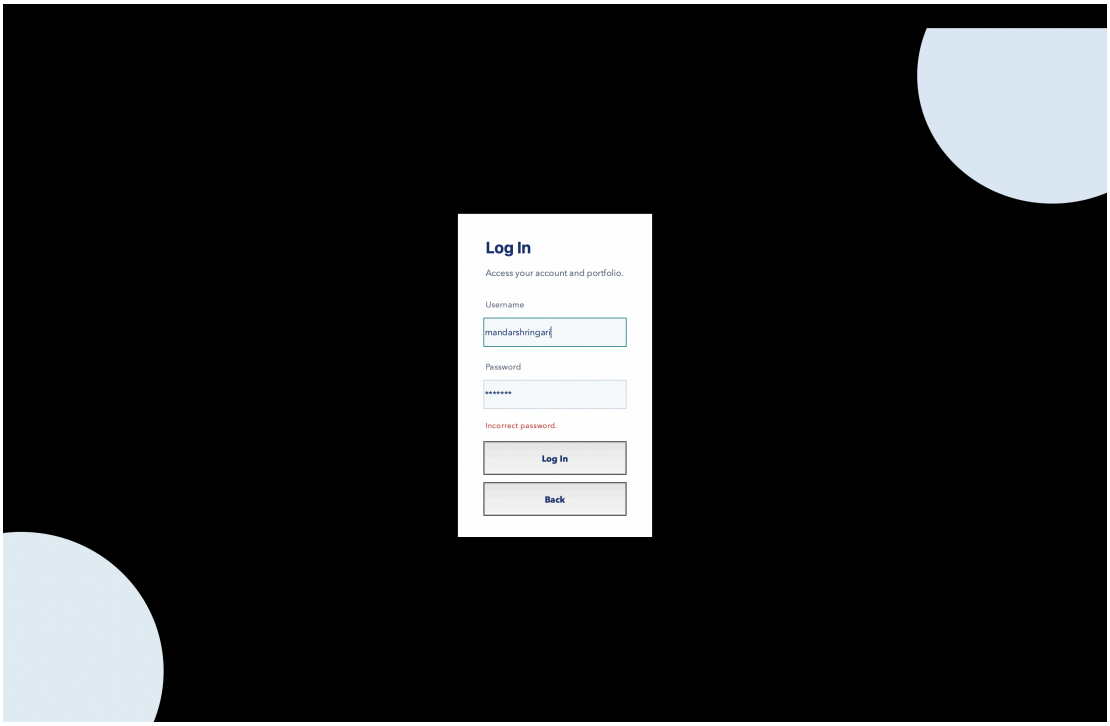
A4)


Portfolio • mandarshringari
Back
Refresh
Open Stock Page
Delete Selected Stock
Search Stock
Search
Add Selected Stock
Find In Portfolio
Your Portfolio
1. AAPL | Price: 254.23 | Volatility: 0.2703

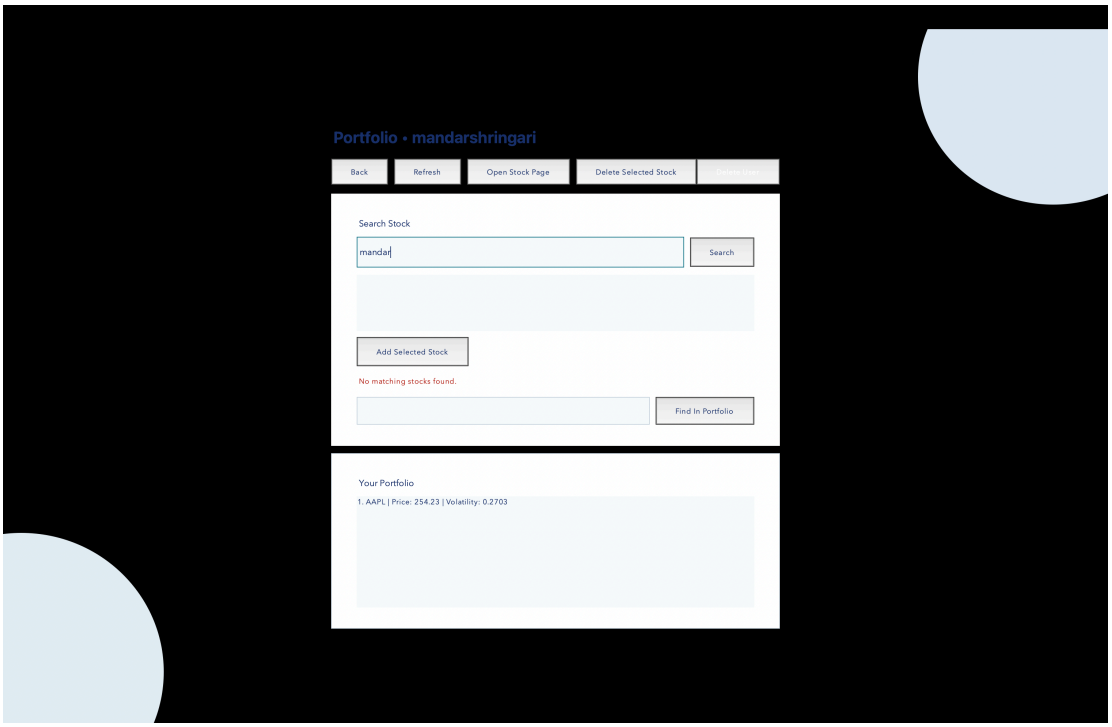
A5)

Back to Portfolio
Individual Stock Page • AAPL
Abstracted Stock Data
Company: Apple Inc. (AAPL)
Current Price: $254.23
Open: $253.08
High: $255.13
Low: $252.18
Close: $254.23
Volume: 27,556,024
Historical Stock Data + Forecast
Range:
1Y
Refresh Chart
Loaded 251 rows from SQLite.
AAPL Candlestick Chart (1Y)
Peak
Dip
Predicted Closing Price
Price
280
260
240
220
200
180
2025-05
2025-07
2025-09
2025-11
2026-01
2026-03
Date

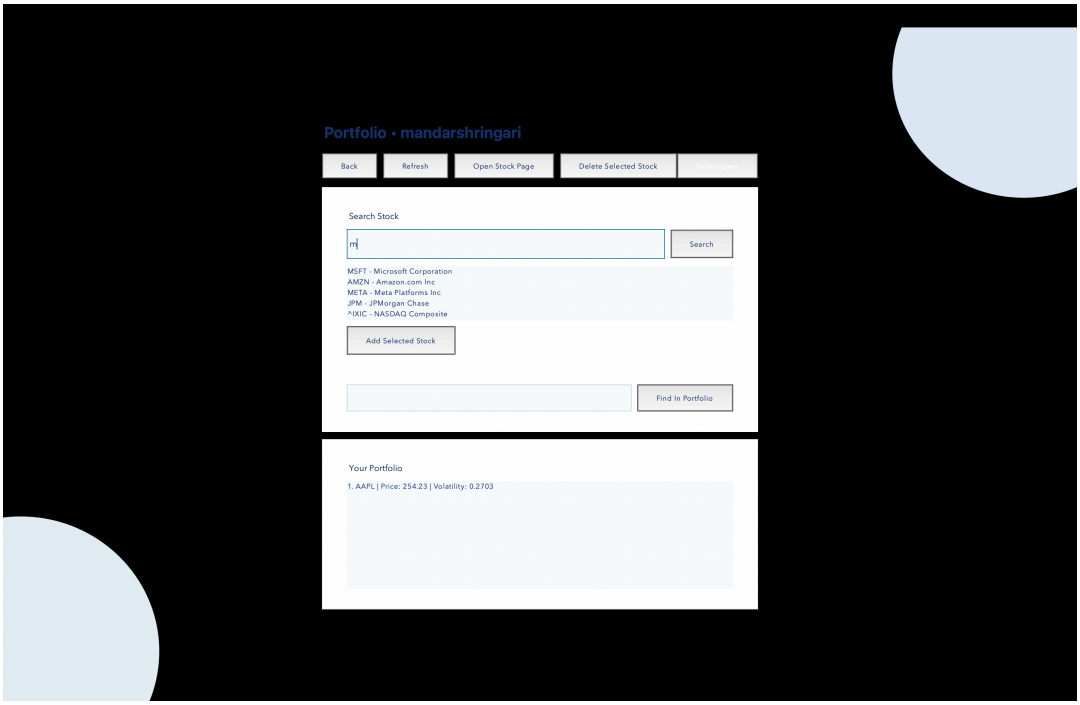
A6)


Log In
Access your account and portfolio.
Username
mandarshringari
Password
Incorrect password.
Log In
Back

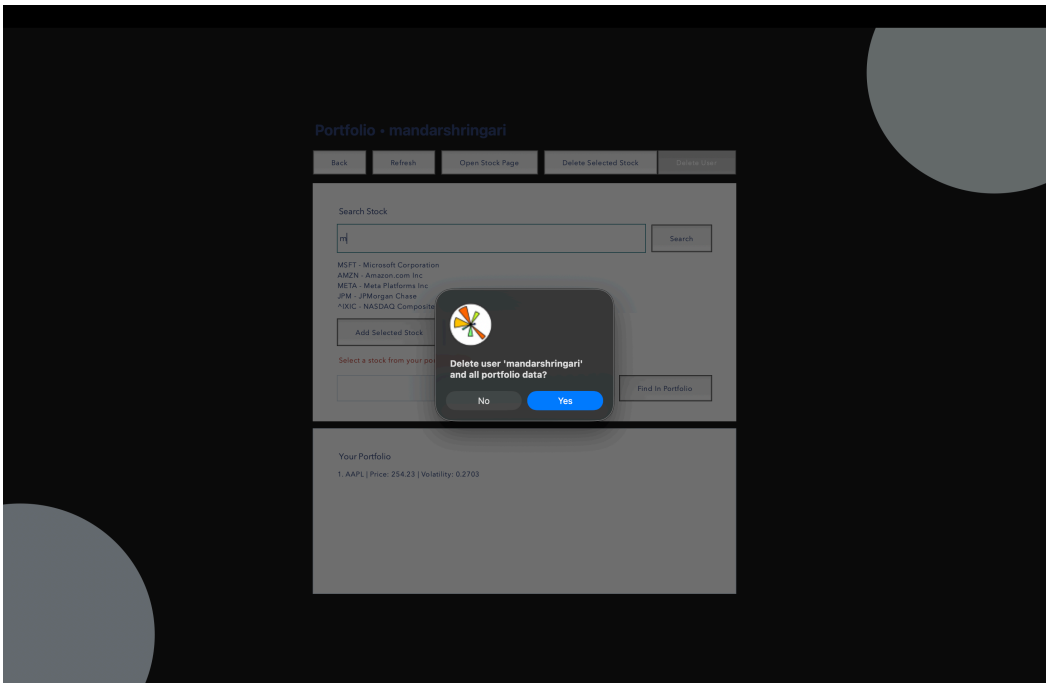
A7)


Portfolio • mandarshringari
Back
Refresh
Open Stock Page
Delete Selected Stock
Search Stock
mandar
Search
Add Selected Stock
No matching stocks found.
Find In Portfolio
Your Portfolio
1. AAPL | Price: 254.23 | Volatility: 0.2703

A8)


Portfolio • mandarshringari
Back
Refresh
Open Stock Page
Delete Selected Stock
Search Stock
m
Search
MSFT - Microsoft Corporation
AMZN - Amazon.com Inc
META - Meta Platforms Inc
JPM - JPMorgan Chase
^IXIC - NASDAQ Composite
Add Selected Stock
Find In Portfolio
Your Portfolio
1. AAPL | Price: 254.23 | Volatility: 0.2703

A9)



Appendix B

B1

GUI.py

```
# GUI application for market analysis.
# Imported modules: regex, tkinter for GUI, messagebox for dialogs, yfinance for stock
data, matplotlib for charts, custom modules for data and auth.
import re
import tkinter as tk
from tkinter import messagebox

import yfinance as yf
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

from data_retrival import DataRetrieval
from data_plotting import StockDataPlotter
from database import DatabaseManager
from hashingalgorithm import AuthBackend

# Database path
DB_PATH = "marketanalysis_2.db"
```

```
# UI theme colors
COLORS = {
    "bg": "#000000",
    "card": "#FFFFFF",
    "card_border": "#D9E2EC",
    "text": "#19326C",
    "muted": "#5B667A",
    "accent": "#0B7285",
    "accent_hover": "#0A6171",
    "neutral": "#E7EDF2",
    "neutral_hover": "#D6E0E8",
    "input_bg": "#F8FAFC",
    "input_border": "#C8D4E0",
    "focus": "#0B7285",
    "danger": "#B42318",
    "success": "#0F8A5F",
}

# UI fonts
FONT_TITLE = ("SF Pro Display", 32, "bold")
FONT_HEADING = ("SF Pro Display", 22, "bold")
FONT_BODY = ("Avenir Next", 12)
FONT_LABEL = ("Avenir Next", 11)
FONT_BUTTON = ("Avenir Next", 12, "bold")
FONT_SMALL = ("Avenir Next", 10)


class MarketAnalysisApp(tk.Tk):
    '''Main application class for market analysis tool.'''
    def __init__(self):
        '''Initialize the main application window.'''
        super().__init__()
        # Set window title
        self.title("Market Analysis Tool")
        # Set window size and minimum size
        self.geometry("1020x700")
        self.minsize(900, 620)
        # Configure background color
        self.configure(bg=COLORS["bg"])

        # Initialize database schema
        DatabaseManager()
        # Create auth backend
        self.auth = AuthBackend(DB_PATH)
        # Create data retrieval service
        self.data = DataRetrieval(DB_PATH)

        # Set current user to None
```

```
        self.current_user_id = None
        self.current_username = None

        # Show start page
        self.show_start_page()
        # Bind close event
        self.protocol("WM_DELETE_WINDOW", self.on_close)

    def on_close(self):
        '''Handle window close event.'''
        # Close auth and data connections
        self.auth.close()
        self.data.close()
        # Destroy window
        self.destroy()

    def clear_window(self):
        '''Clear all widgets from the window.'''
        for child in self.winfo_children():
            child.destroy()

    def show_start_page(self):
        '''Show the start page.'''
        self.clear_window()
        StartPage(self).pack(fill="both", expand=True)

    def show_login_page(self):
        '''Show the login page.'''
        self.clear_window()
        LoginPage(self).pack(fill="both", expand=True)

    def show_signup_page(self):
        '''Show the signup page.'''
        self.clear_window()
        SignupPage(self).pack(fill="both", expand=True)

    def show_portfolio_page(self):
        '''Show the portfolio page.'''
        self.clear_window()
        PortfolioPage(self).pack(fill="both", expand=True)

    def show_stock_detail_page(self, stock_item):
        '''Show the stock detail page for given stock.'''
        self.clear_window()
        StockDetailPage(self, stock_item).pack(fill="both", expand=True)


class BasePage(tk.Frame):
```

```
    '''Base page class for common UI elements.'''
    def __init__(self, master):
        '''Initialize the base page with background decorations.'''
        super().__init__(master, bg=COLORS["bg"])

        # Create canvas for background decorations
        self.decor = tk.Canvas(self, bg=COLORS["bg"], highlightthickness=0, bd=0)
        self.decor.place(relx=0, rely=0, relwidth=1, relheight=1)
        # Bind configure event to draw background
        self.decor.bind("<Configure>", self._draw_background)

        # Create content frame
        self.content = tk.Frame(self, bg=COLORS["bg"])
        self.content.place(relx=0.5, rely=0.5, anchor="center")

    def _draw_background(self, event):
        '''Draw the background ovals.'''
        self.decor.delete("all")
        w, h = event.width, event.height
        self.decor.create_oval(w - 260, -110, w + 110, 240, fill="#DDEAF2",
outline="")
        self.decor.create_oval(-170, h - 260, 220, h + 120, fill="#E3EEF5",
outline="")

    def create_card(self, parent, width=500):
        '''Create a card frame with styling.'''
        card = tk.Frame(
            parent,
            bg=COLORS["card"],
            highlightbackground=COLORS["card_border"],
            highlightthickness=1,
            bd=0,
            padx=34,
            pady=28,
        )
        if width:
            card.configure(width=width)
        return card

    def create_label(self, parent, text, font=FONT_LABEL, fg=None, pady=(0, 8)):
        '''Create a label with styling.'''
        label = tk.Label(
            parent,
            text=text,
            font=font,
            bg=COLORS["card"],
            fg=fg or COLORS["muted"],
            anchor="w",
```

```
        )
        label.pack(fill="x", pady=pady)
        return label

    def create_entry(self, parent, textvariable=None, show=None):
        '''Create an entry field with styling.'''
        entry = tk.Entry(
            parent,
            textvariable=textvariable,
            font=FONT_BODY,
            bg=COLORS["input_bg"],
            fg=COLORS["text"],
            relief="flat",
            insertbackground=COLORS["text"],
            highlightthickness=1,
            highlightbackground=COLORS["input_border"],
            highlightcolor=COLORS["focus"],
            bd=0,
            show=show,
        )
        entry.pack(fill="x", ipady=10)
        return entry

    def create_button(self, parent, text, command, primary=True, pady=(0, 0)):
        '''Create a button with styling and hover effects.'''
        bg = COLORS["accent"] if primary else COLORS["neutral"]
        hover = COLORS["accent_hover"] if primary else COLORS["neutral_hover"]

        btn = tk.Button(
            parent,
            text=text,
            command=command,
            font=FONT_BUTTON,
            bg=bg,
            fg=COLORS["text"],
            activebackground=hover,
            activeforeground=COLORS["text"],
            relief="flat",
            bd=0,
            padx=16,
            pady=12,
            cursor="hand2",
        )
        btn.pack(fill="x", pady=pady)
        # Bind hover events
        btn.bind("<Enter>", lambda _e: btn.configure(bg=hover))
        btn.bind("<Leave>", lambda _e: btn.configure(bg=bg))
        return btn
```

```
class StartPage(BasePage):
    '''Start page for the welcome screen.'''
    def __init__(self, master):
        '''Initialize the start page with title and buttons.'''
        super().__init__(master)
        shell = tk.Frame(self.content, bg=COLORS["bg"])
        shell.pack()

        tk.Label(shell, text="Market Analysis", font=FONT_TITLE, bg=COLORS["bg"],
fg=COLORS["text"]).pack(pady=(0, 12))
        tk.Label(shell, text="Secure account access with live portfolio
intelligence.", font=FONT_BODY, bg=COLORS["bg"], fg=COLORS["muted"]).pack(pady=(0,
24))

        card = self.create_card(shell)
        card.pack()

        self.create_label(card, "Welcome", font=FONT_HEADING, fg=COLORS["text"],
pady=(0, 8))
        self.create_label(card, "Sign in or create a new account to continue.",
font=FONT_BODY, pady=(0, 24))
        self.create_button(card, "Log In", master.show_login_page, primary=True,
pady=(0, 10))
        self.create_button(card, "Create Account", master.show_signup_page,
primary=False)


class LoginPage(BasePage):
    '''Login page for user authentication.'''
    def __init__(self, master):
        '''Initialize the login page with form fields.'''
        super().__init__(master)
        self.username = tk.StringVar()
        self.password = tk.StringVar()
        card = self.create_card(self.content)
        card.pack()
        self.create_label(card, "Log In", font=FONT_HEADING, fg=COLORS["text"],
pady=(0, 8))
        self.create_label(card, "Access your account and portfolio.", font=FONT_BODY,
pady=(0, 22))
        self.create_label(card, "Username")
        self.create_entry(card, self.username)
        self.create_label(card, "Password", pady=(16, 8))
        password_entry = self.create_entry(card, self.password, show="*")
        self.status = tk.Label(card, text="", font=FONT_SMALL, bg=COLORS["card"],
fg=COLORS["danger"], anchor="w")
```

```
        self.status.pack(fill="x", pady=(12, 12))
        self.create_button(card, "Log In", self.validate_login, primary=True, pady=(0,
10))
        self.create_button(card, "Back", master.show_start_page, primary=False)
        password_entry.bind("<Return>", lambda _e: self.validate_login())

    def validate_login(self):
        '''Validate the login credentials.'''
        username = self.username.get().strip()
        password = self.password.get()

        if not username or not password:
            self.status.config(text="Please enter both username and password.",
fg=COLORS["danger"])
            return

        if not self.master.auth.user_exists(username):
            self.status.config(text="User does not exist.", fg=COLORS["danger"])
            return

        user_id = self.master.auth.verify_login(username, password)
        if user_id is not None:
            self.master.current_username = username
            self.master.current_user_id = user_id
            self.status.config(text="Login successful.", fg=COLORS["success"])
            self.after(350, self.master.show_portfolio_page)
        else:
            self.status.config(text="Incorrect password.", fg=COLORS["danger"])


class SignupPage(BasePage):
    '''Signup page for creating new accounts.'''
    def __init__(self, master):
        '''Initialize the signup page with form fields.'''
        super().__init__(master)
        self.username = tk.StringVar()
        self.password = tk.StringVar()
        self.confirm = tk.StringVar()
        card = self.create_card(self.content)
        card.pack()
        self.create_label(card, "Create Account", font=FONT_HEADING,
fg=COLORS["text"], pady=(0, 8))
        self.create_label(card, "Password must be at least 8 chars with letters and
numbers.", font=FONT_BODY, pady=(0, 22))
        self.create_label(card, "Username")
        self.create_entry(card, self.username)
        self.create_label(card, "Password", pady=(16, 8))
        self.create_entry(card, self.password, show="*")
```

```
        self.create_label(card, "Confirm Password", pady=(16, 8))
        confirm_entry = self.create_entry(card, self.confirm, show="*")
        self.status = tk.Label(card, text="", font=FONT_SMALL, bg=COLORS["card"],
fg=COLORS["danger"], anchor="w")
        self.status.pack(fill="x", pady=(12, 12))
        self.create_button(card, "Create Account", self.create_account, primary=True,
pady=(0, 10))
        self.create_button(card, "Back", master.show_start_page, primary=False)
        confirm_entry.bind("<Return>", lambda _e: self.create_account())

    def create_account(self):
        '''Create a new user account.'''
        username = self.username.get().strip()
        password = self.password.get()
        confirm = self.confirm.get()
        if not username or not password or not confirm:
            self.status.config(text="Fill all fields.", fg=COLORS["danger"])
            return
        if not re.match(r"^\w+$", username):
            self.status.config(text="Username: letters, numbers, underscore only.",
fg=COLORS["danger"])
            return
        if len(password) < 8:
            self.status.config(text="Password must be at least 8 characters.",
fg=COLORS["danger"])
            return
        if not any(c.isalpha() for c in password) or not any(c.isdigit() for c in
password):
            self.status.config(text="Password must include letters and numbers.",
fg=COLORS["danger"])
            return
        if password != confirm:
            self.status.config(text="Passwords do not match.", fg=COLORS["danger"])
            return
        if self.master.auth.user_exists(username):
            self.status.config(text="Username already exists.", fg=COLORS["danger"])
            return
        self.master.auth.create_user_account(username, password)
        self.status.config(text="Account created. Redirecting to login...",
fg=COLORS["success"])
        self.after(600, self.master.show_login_page)


class PortfolioPage(BasePage):
    '''Portfolio page for managing user stocks.'''
    def __init__(self, master):
        '''Initialize the portfolio page with search and list widgets.'''
        super().__init__(master)
```

```
        wrapper = tk.Frame(self.content, bg=COLORS["bg"])
        wrapper.pack(fill="both", expand=True)

        tk.Label(wrapper, text=f"Portfolio • {master.current_username}",
font=FONT_HEADING, bg=COLORS["bg"], fg=COLORS["text"]).pack(anchor="w", pady=(0, 14))

        actions = tk.Frame(wrapper, bg=COLORS["bg"])
        actions.pack(fill="x", pady=(0, 12))

        tk.Button(actions, text="Back", command=master.show_start_page,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=8, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left")
        tk.Button(actions, text="Refresh", command=self.refresh_portfolio,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=8, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))
        tk.Button(actions, text="Open Stock Page", command=self.open_selected_stock,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=8, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))
        tk.Button(actions, text="Delete Selected Stock",
command=self.delete_selected_stock, font=FONT_SMALL, bg=COLORS["neutral"],
fg=COLORS["text"], relief="flat", bd=0, padx=12, pady=8,
activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))
        tk.Button(actions, text="Delete User", command=self.delete_user_account,
font=FONT_SMALL, bg=COLORS["danger"], fg="#FFFFFF", relief="flat", bd=0, padx=12,
pady=8, activebackground="#9C1F14", activeforeground="#FFFFFF").pack(side="right")

        top_card = self.create_card(wrapper, width=820)
        top_card.pack(fill="x", pady=(0, 10))

        self.create_label(top_card, "Search Stock", font=FONT_BODY, fg=COLORS["text"],
pady=(0, 8))

        search_row = tk.Frame(top_card, bg=COLORS["card"])
        search_row.pack(fill="x")

        self.search_var = tk.StringVar()
        self.search_entry = tk.Entry(
            search_row,
            textvariable=self.search_var,
            font=FONT_BODY,
            bg=COLORS["input_bg"],
            fg=COLORS["text"],
            relief="flat",
```

```
            insertbackground=COLORS["text"],
            highlightthickness=1,
            highlightbackground=COLORS["input_border"],
            highlightcolor=COLORS["focus"],
        )
        self.search_entry.pack(side="left", fill="x", expand=True, ipady=9)

        tk.Button(search_row, text="Search", command=self.search_stock,
font=FONT_SMALL, bg=COLORS["accent"], fg=COLORS["text"], relief="flat", bd=0, padx=12,
pady=10, activebackground=COLORS["accent_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))

        self.dropdown = tk.Listbox(top_card, height=5, font=FONT_SMALL,
bg=COLORS["input_bg"], fg=COLORS["text"], selectbackground="#CFE8EC", relief="flat")
        self.dropdown.pack(fill="x", pady=(10, 8))
        self.dropdown.bind("<<ListboxSelect>>", self.select_stock)

        tk.Button(top_card, text="Add Selected Stock",
command=self.add_selected_stock, font=FONT_SMALL, bg=COLORS["neutral"],
fg=COLORS["text"], relief="flat", bd=0, padx=12, pady=10,
activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(anchor="w")

        self.search_status = tk.Label(top_card, text="", font=FONT_SMALL,
bg=COLORS["card"], fg=COLORS["danger"], anchor="w")
        self.search_status.pack(fill="x", pady=(10, 0))

        filter_row = tk.Frame(top_card, bg=COLORS["card"])
        filter_row.pack(fill="x", pady=(12, 0))

        self.filter_var = tk.StringVar()
        tk.Entry(filter_row, textvariable=self.filter_var, font=FONT_SMALL,
bg=COLORS["input_bg"], fg=COLORS["text"], relief="flat",
insertbackground=COLORS["text"], highlightthickness=1,
highlightbackground=COLORS["input_border"],
highlightcolor=COLORS["focus"]).pack(side="left", fill="x", expand=True, ipady=8)
        tk.Button(filter_row, text="Find In Portfolio", command=self.search_portfolio,
font=FONT_SMALL, bg=COLORS["neutral"], fg=COLORS["text"], relief="flat", bd=0,
padx=12, pady=9, activebackground=COLORS["neutral_hover"],
activeforeground=COLORS["text"]).pack(side="left", padx=(8, 0))

        list_card = self.create_card(wrapper, width=820)
        list_card.pack(fill="both", expand=True)

        self.create_label(list_card, "Your Portfolio", font=FONT_BODY,
fg=COLORS["text"], pady=(0, 8))
```

```
        self.portfolio_list = tk.Listbox(list_card, font=FONT_SMALL,
bg=COLORS["input_bg"], fg=COLORS["text"], relief="flat", selectbackground="#CFE8EC")
        self.portfolio_list.pack(fill="both", expand=True)
        self.portfolio_list.bind("<Double-Button-1>", lambda _e:
self.open_selected_stock())

        self.selected_search_result = None
        self.portfolio_items = []
        self.refresh_portfolio()

    def search_stock(self):
        '''Search for stocks based on the query.'''
        query = self.search_var.get().strip()
        self.dropdown.delete(0, tk.END)
        self.search_status.config(text="", fg=COLORS["danger"])

        if not query:
            self.search_status.config(text="Enter a stock ticker or company name.")
            return

        try:
            results = self.master.data.stock_search(query)
        except Exception as err:
            self.search_status.config(text=f"Search failed: {err}")
            return

        if not results:
            self.search_status.config(text="No matching stocks found.")
            return

        for ticker, company in results:
            self.dropdown.insert(tk.END, f"{ticker} - {company}")

    def select_stock(self, _event):
        '''Handle stock selection from the dropdown.'''
        picked = self.dropdown.curselection()
        if picked:
            self.selected_search_result = self.dropdown.get(picked[0]).split(" - ")[0]

    def add_selected_stock(self):
        '''Add the selected stock to the portfolio.'''
        ticker = self.selected_search_result
        if not ticker:
            self.search_status.config(text="Select a stock from the list first.",
fg=COLORS["danger"])
            return

        user_id = self.master.current_user_id
```

```
        try:
            existing = self.master.data.get_portfolio_for_ticker(user_id, ticker)
            portfolio_id = existing if existing else
self.master.data.create_portfolio(user_id, ticker)
            self.master.data.insert_historical_data(portfolio_id, user_id, ticker)
            self.master.data.update_portfolio_metrics(portfolio_id)
        except Exception as err:
            self.search_status.config(text=f"Could not add stock: {err}",
fg=COLORS["danger"])
            return

        self.search_status.config(text=f"{ticker} added/updated successfully.",
fg=COLORS["success"])
        self.refresh_portfolio()

    def refresh_portfolio(self):
        '''Refresh the portfolio list.'''
        self.portfolio_list.delete(0, tk.END)
        self.portfolio_items = []
        user_id = self.master.current_user_id

        try:
            rows = self.master.data.get_user_portfolio(user_id)
            ranked = self.master.data._quicksort_by_volatility(rows)
        except Exception as err:
            self.portfolio_list.insert(tk.END, f"Failed to load portfolio: {err}")
            return

        if not ranked:
            self.portfolio_list.insert(tk.END, "No stocks in portfolio yet.")
            return

        self.portfolio_items = ranked
        for idx, item in enumerate(ranked, start=1):
            price = f"{item['currentPrice']:.2f}" if item["currentPrice"] is not None
else "N/A"
            vol = f"{item['volatility']:.4f}" if item["volatility"] is not None else
"N/A"
            self.portfolio_list.insert(tk.END, f"{idx}. {item['ticker']} | Price:
{price} | Volatility: {vol}")

    def delete_selected_stock(self):
        '''Delete the selected stock from the portfolio.'''
        picked = self.portfolio_list.curselection()
        if not picked or not self.portfolio_items:
            self.search_status.config(text="Select a stock from your portfolio
first.", fg=COLORS["danger"])
            return
```

```
        selected_index = picked[0]
        if selected_index >= len(self.portfolio_items):
            self.search_status.config(text="Select a valid stock entry.",
fg=COLORS["danger"])
            return

        item = self.portfolio_items[selected_index]
        ticker = item["ticker"]
        portfolio_id = item["portfolioID"]

        confirmed = messagebox.askyesno(
            "Delete Stock",
            f"Delete {ticker} from your portfolio?",
            parent=self,
        )
        if not confirmed:
            return

        try:
            removed =
self.master.data.remove_stock_for_user(self.master.current_user_id, portfolio_id)
        except Exception as err:
            self.search_status.config(text=f"Could not delete stock: {err}",
fg=COLORS["danger"])
            return

        if not removed:
            self.search_status.config(text="Could not find that stock for this user.",
fg=COLORS["danger"])
            return

        self.search_status.config(text=f"{ticker} deleted from portfolio.",
fg=COLORS["success"])
        self.refresh_portfolio()

    def open_selected_stock(self):
        '''Open the detail page for the selected stock.'''
        picked = self.portfolio_list.curselection()
        if not picked or not self.portfolio_items:
            self.search_status.config(text="Select a stock from your portfolio
first.", fg=COLORS["danger"])
            return

        index = picked[0]
        if index >= len(self.portfolio_items):
            self.search_status.config(text="Select a valid stock entry.",
fg=COLORS["danger"])
```

```
            return

        self.master.show_stock_detail_page(self.portfolio_items[index])

    def delete_user_account(self):
        '''Delete the current user account.'''
        user_id = self.master.current_user_id
        username = self.master.current_username
        if user_id is None:
            self.search_status.config(text="No active user session.",
fg=COLORS["danger"])
            return

        confirmed = messagebox.askyesno(
            "Delete User",
            f"Delete user '{username}' and all portfolio data?",
            parent=self,
        )
        if not confirmed:
            return

        try:
            self.master.data.delete_user_account(user_id)
        except Exception as err:
            self.search_status.config(text=f"Could not delete user: {err}",
fg=COLORS["danger"])
            return

        self.master.current_user_id = None
        self.master.current_username = None
        self.master.show_start_page()

    def search_portfolio(self):
        '''Search for a stock in the portfolio.'''
        query = self.filter_var.get().strip()
        if not query:
            return

        try:
            matches = self.master.data.search_portfolio_tickers(query,
self.master.current_user_id)
        except Exception as err:
            self.search_status.config(text=f"Portfolio search failed: {err}",
fg=COLORS["danger"])
            return

        if not matches:
```

```
            self.search_status.config(text="Ticker not found in your portfolio.",
fg=COLORS["danger"])
            return

        target = matches[0].upper()
        for idx in range(self.portfolio_list.size()):
            text = self.portfolio_list.get(idx).upper()
            if f"{target} " in text or f"{target}|" in text:
                self.portfolio_list.selection_clear(0, tk.END)
                self.portfolio_list.selection_set(idx)
                self.portfolio_list.see(idx)
                self.search_status.config(text=f"Found {target} in portfolio.",
fg=COLORS["success"])
                return


class StockDetailPage(BasePage):
    '''Stock detail page for viewing individual stock data.'''
    def __init__(self, master, stock_item):
        '''Initialize the stock detail page with data and chart.'''
        super().__init__(master)
        self.content.place_configure(relwidth=0.98, relheight=0.95)

        self.stock_item = stock_item
        self.ticker = stock_item["ticker"]
        self.portfolio_id = stock_item["portfolioID"]
        self.range_var = tk.StringVar(value="1Y")
        self.chart_canvas = None

        wrapper = tk.Frame(self.content, bg=COLORS["bg"])
        wrapper.pack(fill="both", expand=True)

        header = tk.Frame(wrapper, bg=COLORS["bg"])
        header.pack(fill="x", pady=(0, 10))

        tk.Button(
            header,
            text="Back to Portfolio",
            command=self.master.show_portfolio_page,
            font=FONT_SMALL,
            bg=COLORS["neutral"],
            fg=COLORS["text"],
            relief="flat",
            bd=0,
            padx=12,
            pady=8,
            activebackground=COLORS["neutral_hover"],
            activeforeground=COLORS["text"],
```

```
        ).pack(side="left")

        tk.Label(
            header,
            text=f"Individual Stock Page • {self.ticker}",
            font=FONT_HEADING,
            bg=COLORS["bg"],
            fg=COLORS["text"],
        ).pack(side="left", padx=(10, 0))

        info_card = self.create_card(wrapper, width=980)
        info_card.pack(fill="x", pady=(0, 10))

        self.create_label(info_card, "Abstracted Stock Data", font=FONT_BODY,
fg=COLORS["text"], pady=(0, 10))
        self.company_label = tk.Label(info_card, text=f"Company: {self.ticker}",
font=FONT_LABEL, bg=COLORS["card"], fg=COLORS["muted"], anchor="w")
        self.company_label.pack(fill="x", pady=(0, 8))

        self.metric_labels = {}
        metric_keys = ["Current Price", "Open", "High", "Low", "Close", "Volume"]
        metric_grid = tk.Frame(info_card, bg=COLORS["card"])
        metric_grid.pack(fill="x")
        for i, key in enumerate(metric_keys):
            lbl = tk.Label(metric_grid, text=f"{key}: N/A", font=FONT_LABEL,
bg=COLORS["card"], fg=COLORS["muted"], anchor="w")
            r, c = divmod(i, 3)
            lbl.grid(row=r, column=c, sticky="w", padx=8, pady=5)
            metric_grid.grid_columnconfigure(c, weight=1)
            self.metric_labels[key] = lbl

        chart_card = self.create_card(wrapper, width=980)
        chart_card.pack(fill="both", expand=True)
        self.create_label(chart_card, "Historical Stock Data + Forecast",
font=FONT_BODY, fg=COLORS["text"], pady=(0, 10))

        controls = tk.Frame(chart_card, bg=COLORS["card"])
        controls.pack(fill="x", pady=(0, 8))
        tk.Label(controls, text="Range:", font=FONT_SMALL, bg=COLORS["card"],
fg=COLORS["text"]).pack(side="left", padx=(0, 6))
        tk.OptionMenu(controls, self.range_var, "1M", "6M", "YTD", "1Y", "ALL",
command=lambda _v: self.render_chart()).pack(side="left")
        tk.Button(
            controls,
            text="Refresh Chart",
            command=self.render_chart,
            font=FONT_SMALL,
            bg=COLORS["neutral"],
```

```
            fg=COLORS["text"],
            relief="flat",
            bd=0,
            padx=12,
            pady=8,
            activebackground=COLORS["neutral_hover"],
            activeforeground=COLORS["text"],
        ).pack(side="left", padx=(8, 0))

        self.chart_status = tk.Label(chart_card, text="", font=FONT_SMALL,
bg=COLORS["card"], fg=COLORS["muted"], anchor="w")
        self.chart_status.pack(fill="x", pady=(0, 8))

        self.chart_host = tk.Frame(chart_card, bg=COLORS["card"])
        self.chart_host.pack(fill="both", expand=True)

        self.load_abstracted_data()
        self.render_chart()

    def _fmt_price(self, value):
        '''Format the price value.'''
        try:
            return f"${float(value):.2f}"
        except Exception:
            return "N/A"

    def _fmt_volume(self, value):
        '''Format the volume value.'''
        try:
            return f"{int(value):,}"
        except Exception:
            return "N/A"

    def load_abstracted_data(self):
        '''Load the stock data from yfinance or database.'''
        latest = None
        company_text = f"Company: {self.ticker}"
        try:
            tk_obj = yf.Ticker(self.ticker)
            info = tk_obj.info
            long_name = info.get("longName") or info.get("shortName")
            if long_name:
                company_text = f"Company: {long_name} ({self.ticker})"
            hist = tk_obj.history(period="5d", interval="1d")
            if not hist.empty:
                latest = hist.iloc[-1]
        except Exception:
            latest = None
```

```
        if latest is None:
            self.master.data.cursor.execute(
                """
                SELECT Open, High, Low, Close, Volume
                FROM STOCK_PLOTTING
                WHERE portfolioID = ?
                ORDER BY DateID DESC
                LIMIT 1
                """,
                (self.portfolio_id,),
            )
            row = self.master.data.cursor.fetchone()
            if row:
                latest = {
                    "Open": row[0],
                    "High": row[1],
                    "Low": row[2],
                    "Close": row[3],
                    "Volume": row[4],
                }

        self.company_label.config(text=company_text)
        if latest is None:
            self.metric_labels["Current Price"].config(text="Current Price: N/A")
            self.metric_labels["Open"].config(text="Open: N/A")
            self.metric_labels["High"].config(text="High: N/A")
            self.metric_labels["Low"].config(text="Low: N/A")
            self.metric_labels["Close"].config(text="Close: N/A")
            self.metric_labels["Volume"].config(text="Volume: N/A")
            return

        close_val = latest["Close"] if isinstance(latest, dict) else
latest.get("Close")
        self.metric_labels["Current Price"].config(text=f"Current Price:
{self._fmt_price(close_val)}")
        self.metric_labels["Open"].config(text=f"Open: {self._fmt_price(latest['Open']
if isinstance(latest, dict) else latest.get('Open'))}")
        self.metric_labels["High"].config(text=f"High: {self._fmt_price(latest['High']
if isinstance(latest, dict) else latest.get('High'))}")
        self.metric_labels["Low"].config(text=f"Low: {self._fmt_price(latest['Low'] if
isinstance(latest, dict) else latest.get('Low'))}")
        self.metric_labels["Close"].config(text=f"Close:
{self._fmt_price(close_val)}")
        self.metric_labels["Volume"].config(text=f"Volume:
{self._fmt_volume(latest['Volume'] if isinstance(latest, dict) else
latest.get('Volume'))}")
```

```
    def render_chart(self):
        '''Render the stock chart with historical data and forecast.'''
        plotter = StockDataPlotter(DB_PATH)
        fig = plotter.build_portfolio_figure(
            portfolio_id=self.portfolio_id,
            ticker_label=self.ticker,
            range_label=self.range_var.get(),
            include_forecast=True,
            future_days=30,
        )
        self.chart_status.config(text=plotter.last_status)

        if self.chart_canvas is not None:
            self.chart_canvas.get_tk_widget().destroy()

        self.chart_canvas = FigureCanvasTkAgg(fig, master=self.chart_host)
        self.chart_canvas.draw()
        self.chart_canvas.get_tk_widget().pack(fill="both", expand=True)


if __name__ == "__main__":
    # Run the main application
    app = MarketAnalysisApp()
    app.mainloop()
```

B2

Database.py

```
import sqlite3

class DatabaseManager:
    def __init__(self):
        super().__init__()
        # Connect to the database file - this creates it if it doesn't exist
        self.connect_db = sqlite3.connect('marketanalysis_2.db')
        self.cursor_obj = self.connect_db.cursor()

        # Enable foreign keys so that relationships between tables work properly
        self.cursor_obj.execute("PRAGMA foreign_keys = ON;")

        # Create the USER table to store user accounts and login info
        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS USER(
            userID INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            salt INTEGER NOT NULL,
            hashedPassword TEXT NOT NULL
```

```
        );
        """)
        # Create the PORTFOLIO table for each user's stock portfolio
        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS PORTFOLIO(
            portfolioID INTEGER PRIMARY KEY AUTOINCREMENT,
            userID INTEGER NOT NULL,
            ticker TEXT NOT NULL,
            stockName TEXT,
            currentPrice REAL,
            volatility REAL,
            timestamp TIMESTAMP,
            FOREIGN KEY (userID) REFERENCES USER(userID)
        );
        """)
        # Create the STOCK_PLOTTING table to store historical stock data for plotting
        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS STOCK_PLOTTING(
            DateID DATE NOT NULL,
            portfolioID INTEGER NOT NULL,
            Open REAL NOT NULL,
            Close REAL NOT NULL,
            High REAL NOT NULL,
            Low REAL NOT NULL,
            Volume REAL NOT NULL,
            PRIMARY KEY (DateID, portfolioID),
            FOREIGN KEY (portfolioID) REFERENCES PORTFOLIO(portfolioID)
        );
        """)

        # Create the STOCK_LIST table as a lookup for available stocks
        self.cursor_obj.execute("""
        CREATE TABLE IF NOT EXISTS STOCK_LIST(
            stockListID INTEGER PRIMARY KEY AUTOINCREMENT,
            tickername TEXT NOT NULL,
            companyname TEXT NOT NULL
        );
        """)
        # This part would add some common tickers to the STOCK_LIST, but it's
commented out for now
        # I might uncomment it later if we need it
        '''self.cursor_obj.executemany(
            "INSERT OR IGNORE INTO STOCK_LIST (tickername, companyname) VALUES (?,
?);",
            [
                # Major US stocks
                ("AAPL", "Apple Inc"),
                ("MSFT", "Microsoft Corporation"),
```

```
            ("GOOGL", "Alphabet Inc"),
            ("AMZN", "Amazon.com Inc"),
            ("META", "Meta Platforms Inc"),
            ("NVDA", "NVIDIA Corporation"),
            ("TSLA", "Tesla Inc"),
            ("BRK-B", "Berkshire Hathaway"),
            ("JPM", "JPMorgan Chase"),
            ("V", "Visa Inc"),

            # UK / EU stocks
            ("BP", "BP plc"),
            ("HSBA", "HSBC Holdings"),
            ("SHEL", "Shell plc"),
            ("ULVR", "Unilever"),
            ("AZN", "AstraZeneca"),
            ("RIO", "Rio Tinto"),
            ("BHP", "BHP Group"),
            ("NESN.SW", "Nestlé"),

            # Indices
            ("^GSPC", "S&P 500"),
            ("^DJI", "Dow Jones Industrial Average"),
            ("^IXIC", "NASDAQ Composite"),
            ("^FTSE", "FTSE 100"),
            ("^N225", "Nikkei 225"),

            # Commodities
            ("GC=F", "Gold Futures"),
            ("SI=F", "Silver Futures"),
            ("CL=F", "Crude Oil"),
            ("NG=F", "Natural Gas"),

            # Forex
            ("GBPUSD=X", "GBP/USD"),
            ("EURUSD=X", "EUR/USD"),
            ("USDJPY=X", "USD/JPY"),

            # Crypto
            ("BTC-USD", "Bitcoin"),
            ("ETH-USD", "Ethereum"),
            ("BNB-USD", "Binance Coin"),
            ("SOL-USD", "Solana"),
            ("XRP-USD", "Ripple"),
            ("ADA-USD", "Cardano")
        ]
    )'''
    # These lines would clear the database each time, but they're commented out
    # I used them for testing to start fresh
```

```
        #self.cursor_obj.execute("DELETE FROM STOCK_PLOTTING;")
        #self.cursor_obj.execute("DELETE FROM PORTFOLIO;")
        #self.cursor_obj.execute("DELETE FROM USER;")
        self.connect_db.commit()

if __name__ == "__main__":
    DatabaseManager()
```

B3

Data_retrival.py

```
import sqlite3
import yfinance as yf
from volatlilty_caluclator import Volatlity

class DataRetrieval:
    # This class handles all the data retrieval and manipulation for our market
analysis app
    def __init__(self, db_path):
        """Initializes the database connection."""
        # Set up the database connection and cursor
        self.db_path = db_path
        self.connect_db = sqlite3.connect(db_path)
        self.cursor = self.connect_db.cursor()
        self.cursor.execute("PRAGMA foreign_keys = ON;")

    def stock_search(self, ticker):
        """Searches for stocks in the STOCK_LIST table through database queries."""
        # This method searches for stocks in the STOCK_LIST table based on ticker or
company name
        query = """
        SELECT DISTINCT tickername, companyname
        FROM STOCK_LIST
        WHERE tickername LIKE ? OR companyname LIKE ?;
        """
        like_value = f"%{ticker}%"

        # I used parameterised queries to make it more secure
        self.cursor.execute(query, (like_value, like_value))
        results = self.cursor.fetchall()

        # I get the ticker from the database instead of using raw user input
        if results:
            db_ticker = results[0][0]
            self.company = yf.Ticker(db_ticker)
            try:
```

```
                self.company_name = self.company.info.get("symbol", None)
            except Exception:
                self.company_name = None
        else:
            self.company_name = None

        return results

    def search_portfolio_tickers(self, ticker_query="", user_id=None):
        """Searches for tickers in the user's portfolio."""
        # This searches for tickers already in the user's portfolio
        # If user_id is given, it only returns that user's tickers
        like_value = f"%{ticker_query.upper()}%"

        if user_id is None:
            query = """
            SELECT DISTINCT ticker
            FROM PORTFOLIO
            WHERE UPPER(ticker) LIKE ?
            ORDER BY ticker ASC;
            """
            self.cursor.execute(query, (like_value,))
        else:
            query = """
            SELECT DISTINCT ticker
            FROM PORTFOLIO
            WHERE userID = ?
              AND UPPER(ticker) LIKE ?
            ORDER BY ticker ASC;
            """
            self.cursor.execute(query, (user_id, like_value))

        return [row[0] for row in self.cursor.fetchall()]

    def get_portfolio_for_ticker(self, user_id, ticker):
        """Gets the portfolio ID for a specific ticker and user."""
        self.cursor.execute(
            """
            SELECT portfolioID
            FROM PORTFOLIO
            WHERE userID = ? AND UPPER(ticker) = UPPER(?)
            ORDER BY portfolioID ASC
            LIMIT 1;
            """,
            (user_id, ticker)
        )
        row = self.cursor.fetchone()
        return row[0] if row else None
```

```
    def ensure_portfolio_exists(self, portfolioID, user_id, ticker):
        """Ensures the portfolio entry exists before adding data."""
        self.cursor.execute(
            "SELECT userID, ticker FROM PORTFOLIO WHERE portfolioID = ?;",
            (portfolioID,)
        )
        exists = self.cursor.fetchone()

        if exists:
            existing_user_id, existing_ticker = exists
            if existing_user_id != user_id or existing_ticker.upper() !=
ticker.upper():
                raise ValueError(
                    f"portfolioID {portfolioID} belongs to {existing_ticker} for
another user or ticker."
                )
            return

        self.cursor.execute(
            """
            INSERT INTO PORTFOLIO (portfolioID, userID, ticker, stockName)
            VALUES (?, ?, ?, ?);
            """,
            (portfolioID, user_id, ticker, ticker)
        )

        self.connect_db.commit()

    def create_portfolio(self, user_id, ticker):
        """Creates a new portfolio entry for a user and ticker."""
        self.cursor.execute(
            """
            INSERT INTO PORTFOLIO (userID, ticker, stockName)
            VALUES (?, ?, ?);
            """,
            (user_id, ticker, ticker)
        )
        self.connect_db.commit()
        return self.cursor.lastrowid

    def insert_historical_data(self, portfolioID, user_id, ticker):
        """Inserts historical stock data from yfinance into the database."""
        # This inserts historical stock data from yfinance into the database
        # First, make sure the portfolio exists
        self.ensure_portfolio_exists(portfolioID, user_id, ticker)
        # I use yfinance Ticker object because it's more stable
        ticker_obj = yf.Ticker(ticker)
```

```
        # Get historical price data for the last year
        hist = ticker_obj.history(period="1y", interval="1d")

        # Check if we got any data
        if hist.empty:
            raise ValueError(
                f"Invalid ticker or no historical data available for '{ticker}'."
            )
        insert_query = """
            INSERT OR IGNORE INTO STOCK_PLOTTING
            (DateID, portfolioID, Open, Close, High, Low, Volume)
            VALUES (?, ?, ?, ?, ?, ?, ?);
        """
        rows = []
        for date, row in hist.iterrows():
            rows.append((
                date.date(),                # DateID
                int(portfolioID),            # portfolioID (FK + PK)
                float(row["Open"]),
                float(row["Close"]),
                float(row["High"]),
                float(row["Low"]),
                float(row["Volume"])
            ))
        try:
            self.cursor.executemany(insert_query, rows)
            self.connect_db.commit()
            print("Inserted/ignored rows attempted:", len(rows))

            self.cursor.execute(
                "SELECT COUNT(*) FROM STOCK_PLOTTING WHERE portfolioID=?;",
                (portfolioID,)
            )
            print("Rows now in DB for portfolio:", self.cursor.fetchone()[0])
        except sqlite3.IntegrityError as e:
            raise RuntimeError(f"Insert failed due to constraints: {e}")

    def _get_closing_prices(self, portfolioID):
        """Helper method to get closing prices for a portfolio."""
        self.cursor.execute(
            """
            SELECT Close
            FROM STOCK_PLOTTING
            WHERE portfolioID = ?
            ORDER BY DateID ASC;
            """,
            (portfolioID,)
```

```
        )
        return [float(row[0]) for row in self.cursor.fetchall()]

    # I added the function to calculate the volatility and update portfolio metrics
    def calculate_latest_volatility(self, portfolioID, window=30):
        """Calculates the latest volatility for a portfolio."""
        prices = self._get_closing_prices(portfolioID)
        return Volatlity.latest_from_prices(prices, window=window)

    def update_portfolio_metrics(self, portfolioID):
        """Updates the current price and volatility in the portfolio table."""
        prices = self._get_closing_prices(portfolioID)
        current_price = prices[-1] if prices else None
        volatility = self.calculate_latest_volatility(portfolioID)

        self.cursor.execute(
            """
            UPDATE PORTFOLIO
            SET currentPrice = ?,
                volatility = ?,
                timestamp = CURRENT_TIMESTAMP
            WHERE portfolioID = ?;
            """,
            (current_price, volatility, portfolioID)
        )
        self.connect_db.commit()
        return current_price, volatility

    def get_user_portfolio(self, user_id):
        """Retrieves all portfolio items for a user."""
        self.cursor.execute(
            """
            SELECT portfolioID, ticker, stockName, currentPrice, volatility, timestamp
            FROM PORTFOLIO
            WHERE userID = ?;
            """,
            (user_id,)
        )
        rows = self.cursor.fetchall()
        return [
            {
                "portfolioID": row[0],
                "ticker": row[1],
                "stockName": row[2],
                "currentPrice": row[3],
                "volatility": row[4],
                "timestamp": row[5],
            }
```

```
            for row in rows
        ]

    def _quicksort_by_volatility(self, items):
        """Sorts portfolio items by volatility using quicksort."""
        if len(items) <= 1:
            return items
        pivot = items[len(items) // 2]
        pivot_vol = pivot["volatility"] if pivot["volatility"] is not None else -1.0

        higher = []
        equal = []
        lower = []
        for item in items:
            item_vol = item["volatility"] if item["volatility"] is not None else -1.0
            if item_vol > pivot_vol:
                higher.append(item)
            elif item_vol < pivot_vol:
                lower.append(item)
            else:
                equal.append(item)
        return self._quicksort_by_volatility(higher) + equal +
self._quicksort_by_volatility(lower)
    def print_portfolio_section_with_ranking(self, user_id):
        """Prints the portfolio ranked by volatility."""
        portfolio_rows = self.get_user_portfolio(user_id)
        ranked_rows = self._quicksort_by_volatility(portfolio_rows)

        print("\n=== PORTFOLIO SECTION (Ranked by Volatility using Quicksort) ===")
        if not ranked_rows:
            print("No stocks in portfolio.")
            return

        for rank, item in enumerate(ranked_rows, start=1):
            vol_text = f"{item['volatility']:.6f}" if item["volatility"] is not None
else "N/A"
            price_text = f"{item['currentPrice']:.2f}" if item["currentPrice"] is not
None else "N/A"
            print(
                f"{rank}. {item['ticker']} | PortfolioID: {item['portfolioID']} | "
                f"Price: {price_text} | Volatility: {vol_text}"
            )

    def remove_stock(self, portfolioID):
        """Removes a stock from the portfolio and its plotting data."""
        # This removes a stock from the portfolio and its plotting data
        self.cursor.execute("DELETE FROM PORTFOLIO WHERE portfolioID = ?;",
(portfolioID,))
```

```
        self.cursor.execute("DELETE FROM STOCK_PLOTTING WHERE portfolioID = ?;",
(portfolioID,))
        self.connect_db.commit()

    def remove_stock_for_user(self, user_id, portfolioID):
        """Removes a stock for a specific user."""
        self.cursor.execute(
            """
            SELECT 1
            FROM PORTFOLIO
            WHERE portfolioID = ?
              AND userID = ?
            LIMIT 1;
            """,
            (portfolioID, user_id),
        )
        if self.cursor.fetchone() is None:
            return False

        self.cursor.execute("DELETE FROM STOCK_PLOTTING WHERE portfolioID = ?;",
(portfolioID,))
        self.cursor.execute("DELETE FROM PORTFOLIO WHERE portfolioID = ? AND userID =
?;", (portfolioID, user_id))
        self.connect_db.commit()
        return True

    def delete_user_account(self, user_id):
        """Deletes a user's entire account and all their data."""
        self.cursor.execute("SELECT portfolioID FROM PORTFOLIO WHERE userID = ?;",
(user_id,))
        portfolio_ids = [row[0] for row in self.cursor.fetchall()]

        for portfolio_id in portfolio_ids:
            self.cursor.execute("DELETE FROM STOCK_PLOTTING WHERE portfolioID = ?;",
(portfolio_id,))

        self.cursor.execute("DELETE FROM PORTFOLIO WHERE userID = ?;", (user_id,))
        self.cursor.execute("DELETE FROM USER WHERE userID = ?;", (user_id,))
        self.connect_db.commit()
        return True

    def close(self):
        """Closes the database connection."""
        # This closes the database connection
        self.connect_db.close()
```

B4

Data_plotting

```
import os
import sqlite3
from datetime import datetime, timedelta

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd
import yfinance as yf
from matplotlib.patches import Rectangle

from regression import Regression


class StockDataPlotter:
    """
    A class for plotting stock data using candlestick charts from SQLite or Yahoo
Finance.
    """

    def __init__(self, db_path="marketanalysis_2.db"):
        """
        Initializes the plotter with database path and empty data frame.
        """
        self.db_path = self._resolve_db_path(db_path)
        self.df = pd.DataFrame()
        self.last_status = "Ready"

    def _resolve_db_path(self, db_path):
        """
        Resolves the database path by checking absolute, local, or fallback locations.
        """
        if os.path.isabs(db_path):
            return db_path

        local = os.path.join(os.getcwd(), db_path)
        if os.path.exists(local):
            return local

        fallback = os.path.abspath(os.path.join(os.path.dirname(__file__), "..",
db_path))
        return fallback

    def load_from_sqlite(self, portfolio_id=None):
        """
        Loads stock data from SQLite database, optionally filtered by portfolio ID.
        """
```

```
        try:
            con = sqlite3.connect(self.db_path)
            if portfolio_id is None:
                query = """
                    SELECT DateID, Open, High, Low, Close
                    FROM STOCK_PLOTTING
                    ORDER BY DateID
                """
                self.df = pd.read_sql_query(query, con)
            else:
                query = """
                    SELECT DateID, Open, High, Low, Close
                    FROM STOCK_PLOTTING
                    WHERE portfolioID = ?
                    ORDER BY DateID
                """
                self.df = pd.read_sql_query(query, con, params=(portfolio_id,))
            con.close()
        except Exception as exc:
            self.df = pd.DataFrame()
            self.last_status = f"SQLite load failed: {exc}"
            return

        if self.df.empty:
            self.last_status = "No rows found in STOCK_PLOTTING."
            return

        self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
        self.df = self.df.dropna(subset=["DateID", "High"]).sort_values("DateID")
        self.last_status = f"Loaded {len(self.df)} rows from SQLite."
    def fetch_from_yfinance(self, ticker):
        """
        Fetches stock data from Yahoo Finance for the given ticker.
        """
        try:
            downloaded = yf.Ticker(ticker).history(period="10y", interval="1d")
            if downloaded.empty:
                self.last_status = f"No Yahoo data for ticker: {ticker}"
                return

            downloaded = downloaded.reset_index()
            self.df = downloaded[["Date", "Open", "High", "Low",
"Close"]].rename(columns={"Date": "DateID"})
            self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
            self.df = self.df.dropna(subset=["DateID", "High"]).sort_values("DateID")
            self.last_status = f"Loaded {len(self.df)} rows from Yahoo for {ticker}."
        except Exception as exc:
            self.last_status = f"Yahoo load failed: {exc}"
```

```
    def get_filtered_data(self, range_label="ALL"):
        """
        Filters the data based on the specified date range label.
        """
        if self.df.empty:
            return self.df

        end_date = self.df["DateID"].max()
        label = range_label.upper()

        if label == "1M":
            start_date = end_date - timedelta(days=30)
        elif label == "6M":
            start_date = end_date - timedelta(days=182)
        elif label == "YTD":
            start_date = datetime(end_date.year, 1, 1)
        elif label == "1Y":
            start_date = end_date - timedelta(days=365)
        else:
            return self.df

        return self.df[self.df["DateID"] >= start_date]

    def build_figure(self, range_label="ALL", ticker_label=None, forecast_df=None):
        """
        Builds a candlestick chart figure with optional forecast data.
        """
        fig, ax = plt.subplots(figsize=(9, 4.8), dpi=100)  # I set a fixed size for
consistency across displays

        if self.df.empty:
            ax.set_title("No data available")
            return fig

        plot_df = self.get_filtered_data(range_label)
        if plot_df.empty:
            ax.set_title("No data available for selected range")
            return fig

        date_nums = mdates.date2num(plot_df["DateID"].to_numpy())  # Convert dates to
numbers for matplotlib plotting
        if len(date_nums) > 1:
            candle_width = min(0.8, (date_nums[1] - date_nums[0]) * 0.7)  # I
calculate width based on date spacing to avoid overlap
        else:
            candle_width = 0.6  # Default width for single data point
```

```
        for x, open_p, high_p, low_p, close_p in zip(
            date_nums,
            plot_df["Open"],
            plot_df["High"],
            plot_df["Low"],
            plot_df["Close"],
        ):
            up_candle = close_p >= open_p  # Green for up, red for down candles
            color = "green" if up_candle else "red"

            ax.vlines(x, low_p, high_p, color=color, linewidth=1)  # Draw the wick
(high-low line)
            body_bottom = min(open_p, close_p)
            body_height = abs(close_p - open_p)
            if body_height == 0:
                body_height = 0.001  # I added a tiny height to make doji candles
visible

            rect = Rectangle(
                (x - candle_width / 2, body_bottom),
                candle_width,
                body_height,
                facecolor=color,
                edgecolor=color,
                alpha=0.75,  # Semi-transparent for better layering
            )
            ax.add_patch(rect)  # Add the candle body

        # Highlight peak and dip for easier interpretation.
        peak_idx = plot_df["High"].idxmax()
        dip_idx = plot_df["Low"].idxmin()
        peak_row = plot_df.loc[peak_idx]
        dip_row = plot_df.loc[dip_idx]
        ax.scatter(
            mdates.date2num(pd.to_datetime(peak_row["DateID"])),
            peak_row["High"],
            color="green",
            s=55,
            marker="^",
            label="Peak",
            zorder=5,  # I used zorder to ensure the markers are on top
        )
        ax.scatter(
            mdates.date2num(pd.to_datetime(dip_row["DateID"])),
            dip_row["Low"],
            color="red",
            s=55,
            marker="v",
```

```
            label="Dip",
            zorder=5,
        )

        x_min = float(date_nums.min())
        x_max = float(date_nums.max())
        y_min = float(plot_df["Low"].min())
        y_max = float(plot_df["High"].max())

        if forecast_df is not None and not forecast_df.empty:
            forecast_x =
mdates.date2num(pd.to_datetime(forecast_df["DateID"]).to_numpy())
            ax.plot(
                forecast_x,
                forecast_df["PredictedClose"],
                linestyle="--",
                linewidth=2,
                color="#1f77b4",
                label="Predicted Closing Price",
            )  # Dashed line for forecast to distinguish from actual data
            x_min = min(x_min, float(forecast_x.min()))
            x_max = max(x_max, float(forecast_x.max()))
            y_min = min(y_min, float(forecast_df["PredictedClose"].min()))
            y_max = max(y_max, float(forecast_df["PredictedClose"].max()))  # Adjust
axes to include forecast range

        title_ticker = f"{ticker_label} " if ticker_label else ""
        ax.set_title(f"{title_ticker}Candlestick Chart ({range_label.upper()})")
        ax.set_xlabel("Date")
        ax.set_ylabel("Price")
        ax.xaxis_date()  # Tells matplotlib that this is a date axis
        ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))  # Format dates as
Year-Month

        x_pad = candle_width * 2  # Padding to prevent the candles from touching edges
        y_pad = max((y_max - y_min) * 0.05, 0.01)  # 5% padding or minimum 0.01
        ax.set_xlim(x_min - x_pad, x_max + x_pad)
        ax.set_ylim(y_min - y_pad, y_max + y_pad)

        fig.autofmt_xdate()  # Automatically format x-axis dates for readability
        ax.grid(True, alpha=0.3)  # Light grid for reference
        ax.legend(loc="best")  # Place legend in best position
        fig.tight_layout()  # Adjust layout to fit everything nicely
        return fig

    def build_portfolio_figure(
        self,
        portfolio_id,
```

```
        ticker_label=None,
        range_label="ALL",
        include_forecast=True,
        future_days=30,
    ):
        """
        Builds a portfolio figure with optional regression forecast.
        """
        self.load_from_sqlite(portfolio_id=portfolio_id)
        if self.df.empty:
            return self.build_figure(range_label=range_label,
ticker_label=ticker_label)

        forecast_df = None
        if include_forecast:
            try:
                model = Regression(
                    db_path=self.db_path,
                    portfolio_id=portfolio_id,
                    future_days=future_days,
                )
                forecast_df, _ = model.run()
            except Exception as exc:
                self.last_status = f"{self.last_status} Regression skipped: {exc}"

        return self.build_figure(range_label=range_label, ticker_label=ticker_label,
forecast_df=forecast_df)

    def show_plot(self, range_label="ALL", ticker="AAPL"):
        """
        Displays the candlestick plot for the specified range and ticker.
        """
        self.load_from_sqlite()
        if self.df.empty:
            self.fetch_from_yfinance(ticker)

        fig = self.build_figure(range_label=range_label, ticker_label=ticker)
        fig.show()
        plt.show()


def prompt_range_label():
    """
    Prompts the user to select a date range label.
    """
    valid_ranges = {"1M", "6M", "YTD", "1Y", "ALL"}
    selected = input("Choose date range (1M, 6M, YTD, 1Y, ALL): ").strip().upper()
    if selected in valid_ranges:
```

```
        return selected
    print("Invalid range, defaulting to ALL.")
    return "ALL"


if __name__ == "__main__":
    plotter = StockDataPlotter()
    chosen_range = prompt_range_label()
    plotter.show_plot(range_label=chosen_range, ticker="AAPL")
```

B5

Regression

```
import sqlite3
import os
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from pandas.tseries.holiday import USFederalHolidayCalendar
from pandas.tseries.offsets import CustomBusinessDay

class Regression:
    """
    The class for performing linear regression on stock closing prices to predict
future values.
    """
    def __init__(self, db_path=None, portfolio_id=1, future_days=30):
        """
        Initializes the regression model with database path, portfolio ID, and
prediction days.
        """
        self.db_path = db_path or self._default_db_path()
        self.conn = sqlite3.connect(self.db_path)
        self.future_days = future_days
        self.portfolio_id = portfolio_id
        self.query = """
        SELECT DateID, Close
        FROM STOCK_PLOTTING
        WHERE portfolioID = ?
        ORDER BY DateID ASC
        """
        self.trading_day_offset =
CustomBusinessDay(calendar=USFederalHolidayCalendar())  # I use this to generate only
trading days for predictions
```

```
    def _default_db_path(self):
        """
        Determines the default database path by checking local and parent directories.
        """
        local = os.path.join(os.getcwd(), "marketanalysis_2.db")
        parent = os.path.abspath(os.path.join(os.path.dirname(__file__), "..",
"marketanalysis_2.db"))
        if os.path.exists(local):
            return local
        return parent
    def preprocessing(self):
        """
        Loads and preprocesses stock data from the database, adding previous close
prices.
        """
        self.df = pd.read_sql_query(self.query, self.conn,
params=(self.portfolio_id,))
        self.conn.close()
        if self.df.empty:
            raise ValueError("No stock data found for the selected portfolio.")
        self.df["DateID"] = pd.to_datetime(self.df["DateID"], errors="coerce")
        self.df = self.df.dropna(subset=["DateID"]).reset_index(drop=True)
        self.df['Prev_Close'] = self.df['Close'].shift(1)  # I shift close prices to
create lag feature for regression
        self.df = self.df.dropna().reset_index(drop=True)
        if len(self.df) < 2:
            raise ValueError("Not enough rows to train the regression model.")
    def model_training(self):
        """
        Trains a linear regression model on the preprocessed data and evaluates it.
        """
        X = self.df[['Prev_Close']].values  # I use previous close as the feature for
simplicity
        y = self.df['Close'].values
        split_index = int(len(X) * 0.8)  # 80-20 train-test split, a common choice
        if split_index == 0 or split_index >= len(X):
            raise ValueError("Not enough data to create train/test split.")
        X_train, X_test = X[:split_index], X[split_index:]
        y_train, y_test = y[:split_index], y[split_index:]

        self.model = LinearRegression()  # I choose linear regression for its
simplicity and interpretability
        self.model.fit(X_train, y_train)
        y_pred = self.model.predict(X_test)
        mse = mean_squared_error(y_test, y_pred)
        r2 = r2_score(y_test, y_pred)
        print("Model Evaluation")
        print(f"Mean Squared Error: {mse:.4f}")
```

```
        print(f"R^2 Score: {r2:.4f}")
        return {"mse": mse, "r2": r2}
    def closingprice_predict(self):
        """
        Generates future closing price predictions using recursive forecasting.
        """
        self.future_predictions = []
        last_close = self.df['Close'].iloc[-1]
        for _ in range(self.future_days):
            next_price = self.model.predict(np.array([[last_close]]))[0]  # I use
recursive prediction to chain forecasts
            self.future_predictions.append(next_price)
            last_close = next_price  # Update for next prediction

    def get_future_prediction_frame(self):
        """
        Creates a DataFrame with future dates and predicted closing prices.
        """
        if not hasattr(self, "future_predictions"):
            raise ValueError("Run closingprice_predict() before requesting prediction
data.")
        last_date = self.df['DateID'].iloc[-1]
        self.future_dates = pd.date_range(
            start=last_date + self.trading_day_offset,
            periods=self.future_days,
            freq=self.trading_day_offset  # I use business days to skip weekends and
holidays
        )
        return pd.DataFrame({
            "DateID": self.future_dates,
            "PredictedClose": self.future_predictions,
        })

    def run(self):
        """
        Executes the full regression pipeline: preprocessing, training, prediction,
and returns results.
        """
        self.preprocessing()
        metrics = self.model_training()
        self.closingprice_predict()
        predictions = self.get_future_prediction_frame()
        return predictions, metrics

    def print_predictions(self):
        """
        Prints the future predicted closing prices in a readable format.
        """
```

```
        if not hasattr(self, "future_dates"):
            self.get_future_prediction_frame()
        print(f"\nNext {self.future_days} Trading Days Predicted Closing Prices:")
        for i, (date, price) in enumerate(zip(self.future_dates,
self.future_predictions), start=1):
            print(f"Day {i} ({date.date()}): {price:.2f}")

if __name__ == "__main__":
    model = Regression()
    model.run()
    model.print_predictions()
```

B6

Volatility_calculator

```
import math
import statistics
import sqlite3

class Volatlity:
    '''Class for calculating volatility from database prices.'''
    def __init__(self, db_path, ticker):
        '''Initialize the class with database path and ticker.'''
        # Set database path and ticker
        self.db_path = db_path
        self.ticker = ticker
        # Load closing prices from database
        self.closing_prices = self._load_closing_prices_from_db()
        # Calculate logarithmic returns
        self.log_returns = self._calculate_log_returns()

    def _load_closing_prices_from_db(self):
        '''Load closing prices from database, trying different tables if needed.'''
        # Connect to database
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        # Check if 'stock_prices' table exists
        cursor.execute(
            "SELECT 1 FROM sqlite_master WHERE type='table' AND name='stock_prices';"
        )
        has_stock_prices = cursor.fetchone() is not None

        if has_stock_prices:
            # Query the stock_prices table
            cursor.execute(
                """
```

```
                SELECT close_price
                FROM stock_prices
                WHERE ticker = ?
                ORDER BY date ASC
                """,
                (self.ticker,),
            )
            prices = [row[0] for row in cursor.fetchall()]
        else:
            # Fall back to STOCK_PLOTTING table
            cursor.execute(
                """
                SELECT s.Close
                FROM STOCK_PLOTTING s
                JOIN PORTFOLIO p ON p.portfolioID = s.portfolioID
                WHERE p.ticker = ?
                ORDER BY s.DateID ASC
                """,
                (self.ticker,),
            )
            prices = [row[0] for row in cursor.fetchall()]

        # Close connection
        conn.close()
        if not prices:
            raise ValueError(
                f"No closing prices found for ticker '{self.ticker}' in database
'{self.db_path}'."
            )
        return prices

    def _calculate_log_returns(self):
        '''Compute logarithmic returns from closing prices.'''
        returns = []
        for i in range(1, len(self.closing_prices)):
            # Skip invalid prices
            if self.closing_prices[i - 1] <= 0 or self.closing_prices[i] <= 0:
                continue
            # Calculate log return
            r = math.log(self.closing_prices[i] / self.closing_prices[i - 1])
            returns.append(r)
        return returns


    @staticmethod
    def _log_returns_from_prices(prices):
        '''Calculate log returns from a list of prices.'''
        returns = []
```

```
        for i in range(1, len(prices)):
            prev_price = prices[i - 1]
            curr_price = prices[i]
            if prev_price <= 0 or curr_price <= 0:
                continue
            returns.append(math.log(curr_price / prev_price))
        return returns

    @staticmethod
    def rolling_from_prices(prices, window=30):
        '''Compute rolling volatilities from prices using specified window.'''
        log_returns = Volatlity._log_returns_from_prices(prices)
        if len(log_returns) < window:
            return []

        volatilities = []
        for i in range(window, len(log_returns) + 1):
            window_slice = log_returns[i - window:i]
            if len(window_slice) < 2:
                continue
            std_dev = statistics.stdev(window_slice)
            annualised_vol = std_dev * math.sqrt(252)
            volatilities.append(annualised_vol)
        return volatilities

    @staticmethod
    def latest_from_prices(prices, window=30):
        '''Get the latest volatility from prices.'''
        vols = Volatlity.rolling_from_prices(prices, window=window)
        return vols[-1] if vols else None

    def rolling_volatility(self, window=30):
        '''Calculate rolling historical volatility using moving window.'''
        volatilities = []

        if len(self.log_returns) < window:
            return []

        for i in range(window, len(self.log_returns) + 1):
            window_slice = self.log_returns[i - window:i]

            if len(window_slice) < 2:
                continue

            std_dev = statistics.stdev(window_slice)

            # Annualise the volatility
            annualised_vol = std_dev * math.sqrt(252)
```

```
            volatilities.append(annualised_vol)

        return volatilities

    def latest_volatility(self, window=30):
        '''Return the most recent rolling volatility value.'''
        vols = self.rolling_volatility(window)
        return vols[-1] if vols else None


if __name__ == "__main__":
    # Set database path and ticker for testing
    DATABASE_PATH = "marketanalysis_2.db"
    TICKER = "AAPL"

    # Create instance of the class
    vol = Volatlity(DATABASE_PATH, TICKER)

    # Print some results
    print("Loaded prices:", len(vol.closing_prices))
    print("Latest annualised volatility:", vol.latest_volatility())
    print("All rolling volatilities:", vol.rolling_volatility())
```

Bibliography

The fetching using yfinance module – https://medium.com/@anjalivemuri97/day-4-fetching-historical-stock-data-with-yfinance-f45f3bd8b9c6

Ticker name to company name conversion - https://stackoverflow.com/questions/38967533/retrieve-company-name-with-ticker-symbol-input-yahoo-or-google-api

pandas as SQLite export - https://www.youtube.com/watch?v=aC6KYMsK23A

plotly module for candle stick- https://plotly.com/python/candlestick-charts/

convert SQLite to pandas data frame - https://stackoverflow.com/questions/36028759/how-to-open-and-convert-sqlite-database-to-pandas-dataframe

the volatility module inspiration - https://medium.com/@polanitzer/volatility-calculation-in-python-estimate-the-annualized-volatility-of-historical-stock-prices-db937366a54d

volatility module inspiration https://github.com/cryptarbitrage-code/historical-volatility-calculations

integration of matplot lib with tkinter https://www.youtube.com/watch?v=lVTC8CvScQo

Hashlib module https://www.geeksforgeeks.org/python/hashlib-module-in-python/

linear regression module

https://www.geeksforgeeks.org/machine-learning/python-linear-regression-using-sklearn/
hashlib algorihtm
https://www.youtube.com/watch?v=i-h0CtKde6w